

共享设备与医生约束下的超声预约联合优化

摘 要

超声预约不是单机排队问题：同一患者可能包含多个检查项目，项目对设备能力存在选择限制，检查过程还同时占用医生；空腹、憋尿、床旁检查和跨室转运又改变了可用时间窗。针对这些特点，本文以题目给出的住院、门诊、体检记录及 22 台设备资料为基础，建立“参数标定—统一任务表示—连续日历排程—留出期评价”的患者级联合模型。模型参数只由 2019 年 3 月 1 日至 2024 年 3 月 31 日训练期估计，2024 年 4 月 1 日至 2025 年 3 月 31 日仅用于时间顺序检验；由于住院事件的最大截止期为 48 小时，只在留出边界前加 2 日预热，用于携入真正可跨边界的在制任务。

对于问题一，先以“患者编号+精确开单时刻”构造申请事件，去除打印、预约等非扫查项目，并按互斥优先级将服务项目归入 11 类。题目没有扫查起止时刻，故不把报告间隔解释为实测检查时长，而以题给时长区间的中点作为名义值、上界作为复杂病例值；单项目相邻报告间隔和医生日吞吐仅用于形成相对修正。训练期得到 297 656 个住院、245 614 个门诊和 239 569 个体检事件，形成 42 336 个医生—项目完整调度时长估计，其中 6 416 个组合具有直接间隔证据。设备能力以表 1 项目文本为直接证据来源，建立项目级严格匹配、类别回退和床旁明确三层证据；22 台设备中床旁检查仅 7 号机可承担，是确定性唯一瓶颈。医生容量则由表 6 训练日首末活动时段重构为星期一 5 分钟槽并发上限，主情景为 5 至 20 人，而非把每日医生总数复制到每个机房。

对于问题二，将每位住院患者的全部服务项目作为一个不可拆分完成目标，在 5 分钟离散连续日历上同时决定患者顺序、项目内部顺序、开始时刻与机房。模型约束包括设备互斥、医生共享并发、项目—设备兼容、患者自身不重叠、48 小时截止、空腹 10 时前完成、憋尿准备、床旁 7 号机、同室优先及跨室至少 10 分钟转运。采用“共同机房优先—多候选项目序列—最早可行时隙”的构造算法，并分别比较共享先到先服务、最小松弛度保护和联合稀缺度策略。

对于问题三，把门诊和体检的历史报告日期作为回顾性计划日压力场景，不读取历史报告时刻、机房和医生派单；释放时刻不早于真实开单，三类患者在同一年度日历中竞争相同房间与医生容量。主情景将留出期中按整数事件最接近 5% 的 8 341 个事件设为复杂病例，其项目采用时长上界。留出期共有 70 771 个住院事件和 96 046 个门诊/体检事件。题目未给出“满足门诊、体检需求”的数值阈值，故以共享 FCFS 的计划日服务率作为不劣基线，再按住院 48 小时率选择主方案。资源可行 FCFS 完成 45 260 个住院事件，48 小时率 63.95%，并完成 57 341 个门诊/体检计划日事件，完成率 59.70%；联合稀缺度把背景率提高到 61.61%，却使住院率降至 63.31%，因此只作为帕累托备选。独立程序重构 151 020 条排程任务，对房间重叠、医生超容量、患者冲突、兼容、转运、准备约束、时长和结果聚合等 18 项检查，违规数均为 0。敏感性分析表明，医生容量降至训练日下四分位时住院率下降 1.99 个百分点；仅保留严格设备证据会使背景完成率下降 10.03 个百分点，说明上线前必须由超声科确认回退项目清单。

本文最终给出可直接执行的患者—项目—时段—机房推荐表。由于数据缺少真实预约时刻、扫查结束、报告提交时刻、未来医生身份和项目资质矩阵，本文把“排程中最后一个项目结束”作为报告完成代理，结果应理解为预约决策方案与能力压力评估，而不是对临床报告时效的因果结论。

关键词：超声预约；患者级排程；共享资源；48 小时完成率；设备兼容；敏感性分析

1 问题重述

1.1 问题背景

超声科每天同时接收住院、门诊和体检患者。三类患者的到达方式和服务承诺不同：住院患者由临床开单触发，题目要求尽可能在 48 小时内完成全部超声项目并出具报告；门诊和体检患者更接近预约制，需要在计划日期内完成。检查过程还受到三类耦合约束：其一，同一患者可能一次申请多个项目，既要确定患者间顺序，又要确定患者内部项目顺序；其二，不同设备能够完成的项目集合不同，床旁、产科高等级、儿科等项目的可替代性明显不一致；其三，设备有空并不等于可以开检，检查还要占用具备相应能力的医生。医疗预约研究通常把容量设计与患者级派位视为相互连接的两个层次 [1, 2]，多项目患者还会引入组合预约与转运协调 [3, 4]。

题目附件主要记录开单和报告环节，没有真实预约时刻、扫查开始、扫查结束和报告提交时刻。因此，本题的关键并非套用某一种经典排队公式，而是先回答“每个字段能够支持什么主张”。若把报告时间直接当作扫查结束，便会把候检、图像采集、报告撰写和提交等环节混合；若按历史机房复现派单，则又会把旧策略的信息泄漏到评价期。本文据此把证据边界置于模型之前：题目给定区间决定绝对服务时长，历史报告间隔只提供相对负担与医生吞吐证据，表 1 决定设备能力，表 6 提供医生活动时段代理。

1.2 三问的数学任务

问题一要求从历史数据中提取科室需求、项目时长、设备能力和医生效率，并据此配置设备与项目。本文将其分解为四个参数层：科室一来源需求分布、互斥项目族与服务时长、项目一设备可行集合、星期一时隙医生并发容量。平峰、高峰和 5% 特殊病例均由同一参数层派生，避免每个情景使用不同口径。

问题二要求为住院患者给出检查室和检查时段的最佳排序，并最大化 48 小时内完成全部项目的患者比例。因此决策变量必须下沉到“患者一项目一时隙一房间”，不能只计算总分分钟容量。对每位患者，全部服务项目完成才记为成功；任何一个项目超时或无可用设备，均不能计入 48 小时完成分子。

问题三要求联合安排门诊、体检和住院患者。本文把前两类的历史报告日期解释为回顾性预约日压力场景，把住院开单时刻解释为释放时刻。三类患者进入同一连续日历，设备互斥和医生并发约束完全共享；联合模型既输出总体策略比较，也输出住院患者的项目次序、时段和机房推荐表。

1.3 总体技术路线

本文采用严格时间顺序设计。训练期为 2019 年 3 月 1 日至 2024 年 3 月 31 日，用于分类规则、设备能力证据、时长、医生容量和调度规则标定；留出期为 2024 年 4 月 1 日至 2025 年 3 月 31 日，只用于评价。为了避免在 4 月 1 日把系统错误地清空，仿真从 2024 年 3 月 30 日启动。住院任务的最长截止期是 48 小时，因而前 2 日已足以携入所有可能跨评价边界的在制任务；这 2 日只预热日历，不进入指标分母。整个年度只建立一次房间占用和医生负荷状态，不按月重置。

从原始记录到可执行预约方案的建模链

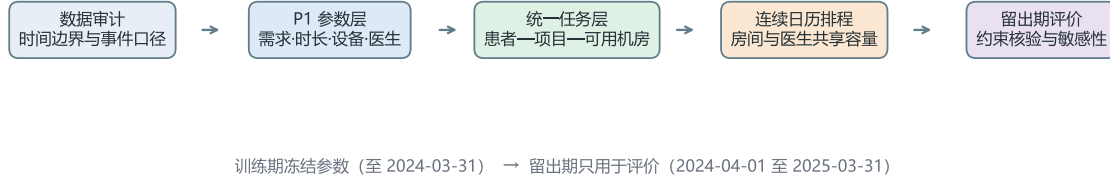


图 1 从原始记录到可执行预约方案的建模链

如图 1 所示，先进行数据语义审计，再完成问题一的参数标定；随后把不同来源记录转换为统一患者任务，最后在连续日历上执行三种排程策略并独立核验。该路线使模型的每一项硬约束都能追溯到题目条件或明确假设，也使留出期结果不依赖历史派单答案。

1.4 本文的主要特点

本文的主要特点有三点。第一，设备能力从“项目族矩阵”进一步细化为项目级证据层；项目族只作为表 1 文本不足时的显式回退，不把“某机器做过该类别中的一个项目”扩大为“能够完成全类别”。第二，问题二与问题三使用同一患者级约束内核，避免一个模型允许跨室而另一个模型禁止跨室、或一个模型计医生而另一个模型忽略医生的可行域错位。第三，结果同时报告住院 48 小时率、门诊/体检预约日率、等待分位数、跨室率和证据回退占比，避免用一个加权总分掩盖服务取舍。

2 问题分析

2.1 字段语义与数据口径

住院、门诊、体检三张明细均以项目行为基本单位，同一患者同一开单时刻可以出现多行。若直接把明细行视为患者，会重复计算多项目申请；若只按患者编号聚合，又会把同一患者不同时期的多次申请合并。因此定义申请事件键

$$e = (s, p, a), \quad (1)$$

其中 s 为来源， p 为患者编号， a 为精确开单时刻；患者编号缺失时以来源行号构造只在该表内唯一的代理键。对同一事件内完全相同的服务项目去重，而不同项目全部保留。打印、预约和图文报告等不占用超声扫描设备的记录被标记为非服务项，不进入工作量。

时间字段分为到达与完成两类。住院释放时刻取开单时刻 a_e ，截止时刻为 $d_e = a_e + 48\text{h}$ 。门诊和体检缺少真实预约日期，本文只在回顾性评价中使用报告日期构造计划日 g_e ；其日内窗口为上午 $[8:00, 12:00]$ 和下午 $[13:00, 17:00]$ 。历史报告的具体时分、历史机房和历史医生不进入决策。官方质量指标强调以申请单为分母评价完成情况 [5]，但附件又缺少独立的报告提交时刻，因此本文将排程中最后一个项目结束时刻作为“完成检查并可进入报告环节”的代理，全文不把它表述为真实报告提交时间。

表 1 统一连续模拟与留出期评价规模

患者来源	连续模拟事件数	项目任务数	留出期评价事件数
住院	71,029	141,429	70,771
门诊	55,512	69,756	55,226
体检	40,820	117,701	40,820
合计	167,361	328,886	166,817

连续模拟规模见表 1。其中住院事件在 2 日预热期和一年留出期均保留精确开单时刻；门诊与体检只使用计划日压力，不使用历史派单结果。这一处理使问题三能够使用三类患者的真实项目组合和日需求，同时避免把不可获得的未来机房、医生当作模型输入。

2.2 项目归类与资源标记

项目归类采用互斥优先级词典。首先识别非服务项；随后依次识别产科 III/IV 级、产科 I/II 级、心脏、介入/定位、血管、腔内、腹部/胃肠、浅表器官、儿科专项和一般其他。优先级解决关键词重叠，例如“心脏造影”先被识别为心脏项目，不因包含“造影”而误入介入类别。每个项目族的时长边界直接依据题给区间，并保留项目原名用于设备匹配，不以类别名替代项目名。

题目中的检查准备要求转换为三个事件级标记：包含“空腹”的项目必须在 10 时前完成；包含“充盈膀胱”等词的事件在主情景中以开单后 60 分钟作为准备完成时刻，另以 45 和 90 分钟作敏感性；若门诊/体检在计划日开班前已满足提前量，则从工作块起点即可排入；明确“床旁”的检查只允许 7 号床边 B 超机。若一个事件同时包含多个标记，所有相应约束共同生效。

2.3 时间边界和中文名称泄漏控制

用于建模的项目目录只统计训练期出现的项目名称。留出期项目在调度输入中按训练期确定的归类规则处理；若新名称无法获得项目级设备证据，才进入已标注的类别回退。另输出全期项目目录只用于检查名称漂移，不参与留出期派位。医生效率和报告间隔同样只使用 2024 年 3 月 31 日以前的报告记录，避免利用留出期完成时刻修正未来调度速度。

2.4 需求变化与高峰特征

图 2 给出三类患者月度日均开单量。住院需求总体平稳上升；门诊呈较强年度和节假日波动；体检最不稳定，在团检月份形成尖峰。因而用全年平均值配置日容量会同时低估体检高峰和住院周一压力。本文在问题一中分别计算来源内日中位数、90% 分位和 95% 分位，并保留星期结构，不把“高峰”定义为简单事件数最多的若干天。

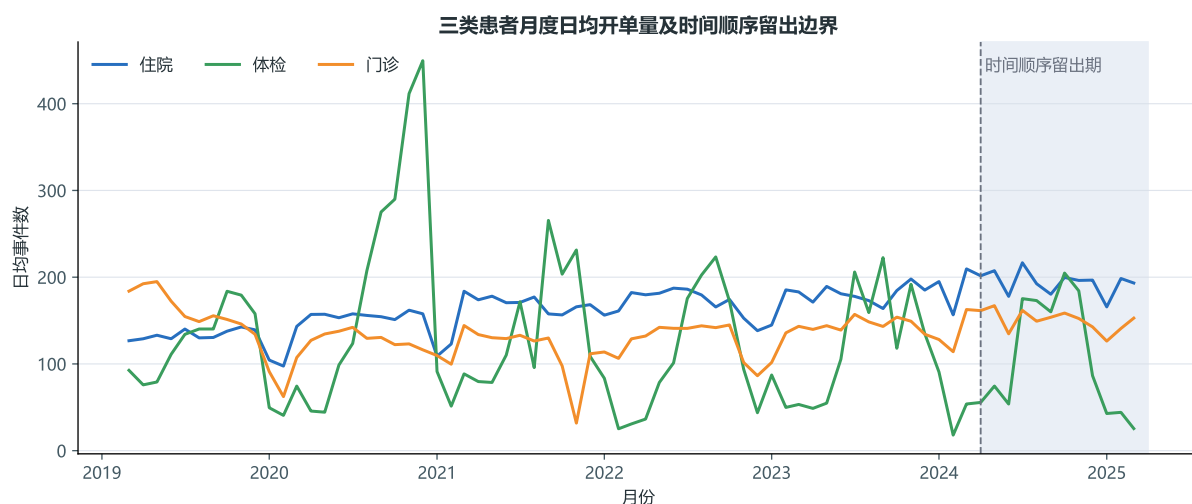


图2 三类患者月度日均开单量及时间顺序留出边界

训练期科室需求见表2。住院需求分散于87个可识别科室，前6科室占35.45%；门诊需求集中度更高，前6科室占62.95%；体检记录只有一个科室编码，不能据此推断其临床科室组成。科室字段缺失的事件数量很少，但模型仍在结果中保留“缺失”类别，不使用众数强行填补。

表2 训练期科室需求集中度

来源	训练期事件数	可识别科室数	前6科室占比	前12科室占比
住院	297,656	87	35.45%	56.72%
门诊	245,614	133	62.95%	81.33%
体检	239,569	1	100.00%	100.00%

2.5 数据能够支持与不能支持的结论

数据能够支持：事件到达规模、项目组合、题给时长边界、项目文本对应的设备能力、历史医生活动时段以及留出期回顾性压力测试。数据不能直接支持：真实扫查时长、报告撰写时长、未来医生身份与项目资质、真实预约时刻、未完成患者的真实后续处置。过程挖掘中的相邻时间戳不自动等于活动持续时间 [6]，影像报告时间的测量也必须区分图像采集、阅片和报告环节 [7]。因此，本文所有“时长估计”均称为调度占用参数，所有“完成率”均注明代理边界。

3 模型假设

结合题目条件和字段边界，作如下假设。

- A1.** 每台设备在任一5分钟时隙内至多执行一个项目；一个项目执行期间连续占用同一台设备和一名医生，不允许中断。该假设对应超声检查的基本互斥资源特征。
- A2.** 仿真日历采用每日8:00–12:00、13:00–17:00两个标准日班工作块，项目不跨午休或日界。该时窗是根据表6训练期首末报告活动重建的可复现容量模板，不是A医院未来正式班表；医院实施时应直接换入当日班表。
- A3.** 同一患者的多个项目必须串行完成；若相邻项目更换房间，主情景预留10分钟转运和重新准备时间，并用5与15分钟检验边界。附件没有院内转运用时，因而10分钟不被写成医院事实。同一房间可连续衔接，转运时间为0。题目“通常安排在同一时段、同一诊室”被表达为优先规则而非绝对硬约束，因为部分项目没有共同兼容设备。

- A4.** 项目服务时长采用题给区间中点作为名义时长，并向上取整到 5 分钟；固定随机种子在评价期和预热期分别抽取最接近 5% 的整数个事件作为特殊病例，其全部项目采用题给上界。多项目患者仍逐项累计时长而不并行重叠：附件没有给出同次检查的固定节约比例，且同一设备和医生不能同时完成两个项目。该处理不虚构额外倍率或合并收益。
- A5.** 空腹项目必须在当天 10 时前完成；憋尿事件主情景按“开单时刻后 60 分钟准备完成”计算，45 和 90 分钟用于边界分析。门诊/体检若在计划日开始前已经完成该提前量，可从日工作块起点进入排程，不再机械追加 60 分钟。该值不来自本题附件；国内医院公开超声流程中存在“提前 1 小时喝水憋尿”的临床准备说明 [8]，但 A 医院上线时仍应读取患者实际准备完成时刻。床旁项目只在 7 号机安排。
- A6.** 门诊和体检的报告日期作为回顾性计划日压力场景。该假设用于检验“若当日需求需在当日完成，资源是否可行”，不等同于拥有真实预约记录。
- A7.** 未排入截止窗口的事件保留为未分配，并进入指标分母；不把这些患者从等待分布和完成率中删除。等待分位数只对成功排入者计算，同时单列未分配人数，以免条件分位数掩盖失败患者。
- A8.** 设备购置日期和型号不用于扩大项目能力集合。设备能否执行项目首先由表 1 文本证据决定；同刻候选仍并列时，程序只用累计负荷、训练期稀缺权重和设备编号作确定性择优。

假设 A1–A3 定义排程可行域；A4–A5 定义题给不确定性和准备要求；A6 是问题三在缺少真实预约字段时的必要情景假设；A7–A8 控制评价与能力解释。为防止新增设定混入附件事实，表 3 逐项给出来源和使用边界。

表 3 主要设定的来源与现实边界

设定/参数	事实来源	模型中的限定
住院 48 小时全项目完成	赛题直接给定	事件硬截止时刻
5% 特殊检查取上界时长	赛题直接给定比例；附件无个体标签	评价/预热分别按最近整数抽取 5%
项目时长区间与空腹 10 点窗口	赛题直接给定	名义/上界时长与硬时窗
平峰 $Q_{0.45}$ - $Q_{0.55}$ 、高峰 $Q_{0.90}$	运营分位阈值，非赛题指定日期	用真实日需求识别代表性容量水平
22 台设备能力、床旁 7 号机	附件表 1 与设备表文本证据	项目-设备兼容集
50 个项目采用类别级能力回退	项目名称与表 1 逐字/语义核心匹配失败时，仅沿用表 1 同项目族具体证据	主情景逐项留痕；严格证据情景禁用回退
08:00-12:00、13:00-17:00 日班模板	附件表 6 训练期首末报告活动重建	可复现容量模板，非未来正式排班
星期-5 分钟医生并发容量	训练日重构活动区间的同刻人数中位数	仅为总并发代理；不声称掌握未来实名班表或专项资质
相邻报告间隔 (0, 60] 分钟	扫查起止缺失下的局部代理阈值	只用于问题一相对吞吐证据，不进入主排程时长
5 分钟时间网格	计算离散化，附件无此字段	时长向上取整，不缩短检查
跨室转运 10 分钟	附件无转运时间	主情景；5/15 分钟边界检验
憋尿准备 60 分钟	附件无时长；医院公开流程建议提前 1 小时 [8]	主情景；45/90 分钟边界检验
门诊/体检报告日作为计划日	附件无历史预约日	回溯压力情景，不声称还原历史预约
最后任务结束代理报告提交	附件无扫查起止与报告书写时长	官方 KPI 的乐观代理，明示适用边界
2 日预热与 2 日截止尾窗	由最大 48 小时截止期推出	只携入跨边界在制任务并覆盖 3 月 31 日开单截止

敏感性分析对附件不能直接标定的医生容量、项目能力回退、憋尿准备和跨室转运每次只改变一个因素，不把补充检验伪装成赛题指定情景。

4 符号说明

表 4 主要集合、参数与变量

符号	含义
$e \in \mathcal{E}$	患者申请事件，含来源、患者、释放时刻和截止时刻
$j \in \mathcal{J}_e$	事件 e 的服务项目； $n_e = \mathcal{J}_e $
$r \in \mathcal{R}$	超声设备或对应检查房间， $ \mathcal{R} = 22$
$t \in \mathcal{T}$	5 分钟离散时隙；每天上午、下午各 48 个
$q_e \in \{I, O, P\}$	事件来源：住院、门诊、体检
a_e, d_e	事件释放时刻与截止时刻
$p_{ej}^{(0)}, p_{ej}^{(1)}$	项目名义时长和特殊病例上界时长（时隙数）
z_e	特殊病例标记， $\sum_e z_e / \mathcal{E} = 5\%$
$\mathcal{R}_{ej}^S, \mathcal{R}_{ej}^F$	项目的严格兼容设备集合与类别回退设备集合
$\mathcal{R}_{ej}$	主情景可用设备集合；床旁项目固定为 $\{7\}$
c_{wt}	星期 w 、日内时隙 t 的医生并发容量
b_{ej}, f_{ej}, u_{ej}	床旁、空腹、憋尿标记
τ	相邻项目跨室转运时长，主情景为 10 分钟
x_{ejrt}	项目 (e, j) 是否在设备 r 、时隙 t 开始
y_e	事件 e 是否全部项目在截止前完成
C_e, S_e	事件完成时刻与首个项目开始时刻
$L_r(t)$	时刻 t 前设备 r 的累计占用时隙数
κ_r	设备 r 的需求稀缺度权重

为避免与自然常数混淆，正文用 e 表示一个具体事件，用 $\mathcal{E}$ 表示事件集合；集合上标 S 和 F 分别表示严格证据与回退证据。完成率均按事件计算而非项目行计算：只有 n_e 个服务项目全部排入且最后完成时刻不晚于 d_e ，才有 $y_e = 1$ 。

5 模型的建立与求解

三个子问共用同一个患者事件、项目任务、设备能力和医生时隙容量内核。问题一估计需求与资源参数；问题二建立住院患者级排程；问题三只在共享日历中加入门诊和体检计划日约束，不另造一套不相容的容量模型。全年整数模型的变量规模过大，因而本文用整数规划明确数学可行域，用共享同一约束的确定性构造算法完成全年求解。

5.1 问题一：需求、时长与资源配置模型

5.1.1 需求预测与峰平判定

对来源 s 、日期 d ，记开单事件数为 N_{sd} 。为检验星期与趋势结构，比较三个不使用留出期信息的基准模型：7 日季节朴素、最近 8 周同星期中位数和日历趋势岭回归。后者写为

$$\hat{N}_{sd} = \beta_0 + \beta_1 t_d + \sum_{w=1}^6 \gamma_w I(w_d = w) + \sum_{m=2}^{12} \eta_m I(m_d = m) + \rho_1 \sin \frac{2\pi t_d}{365.25} + \rho_2 \cos \frac{2\pi t_d}{365.25}. \quad (2)$$

岭参数只在训练期设计矩阵上确定。采用

$$\text{WAPE} = \frac{\sum_d |N_{sd} - \hat{N}_{sd}|}{\sum_d N_{sd}} \quad (3)$$

作为主评价，因为体检低需求日会使逐日百分比误差不稳定。留出期结果表明：住院和门诊开单量均由最近 8 周同星期中位数取得最低 WAPE，分别为 16.04% 和 14.04%；体检开单量由日历趋势模型取得最低 WAPE，但仍为 69.83%，说明团检尖峰不能靠常规日历变量充分预测。三类合计开单量的最低 WAPE 为 24.92%。医院日需求预测中显式建模星期结构并滚动验证是必要步骤 [9, 10]，但体检业务还需要接入未来团检计划，不能只依赖历史时间序列。

对留出期每日需求分布，以中位数附近 $[Q_{0.45}, Q_{0.55}]$ 定义平峰，以 $Q_{0.90}$ 定义高峰，以 $Q_{0.95}$ 定义压力日。住院开单日平峰为 197 人，高峰阈值 283 人；门诊分别为 156 和 198 人；体检因团检尖峰，平峰 48 人而 $Q_{0.90}$ 达到 308.2 人。故设备配置采用“固定能力底座 + 高峰弹性块”，而不是按体检全年均值永久占用机房。

5.1.2 服务时长的分层估计

设项目 j 属于类别 $k(j)$ ，题给时长区间为 $[l_k, u_k]$ ，名义时长

$$p_j^{(0)} = 5 \left\lceil \frac{(l_k + u_k)/2}{5} \right\rceil, \quad p_j^{(1)} = 5 \left\lceil \frac{u_k}{5} \right\rceil. \quad (4)$$

5% 特殊事件使用 $p_j^{(1)}$ ，其余使用 $p_j^{(0)}$ 。各项目族参数见表 5。心脏、产科 III/IV 级的绝对占用显著高于一般项目，是高峰配置中必须保护的长任务。

同一申请含多个项目时，主排程按项目逐项累计上述时长；在同一兼容机房连续完成可以省去转运，但不把不同项目视为同时扫描。训练期另以 0.8、0.9 和 1.0 三种合并系数检查需求量口径，本文不把缺少现场证据的节约比例写入患者级可行域。

表 5 项目族调度时长及训练期间隔证据

项目族	下界/min	名义/min	上界/min	训练期代理样本数
产科 III/IV 级	30	35	40	13,451
产科 I/II 级	15	17.5	20	20,095
心脏	10	12.5	15	141,017
介入/定位	5	7.5	10	6,782
血管	5	7.5	10	59,006
经阴道/腔内	5	7.5	10	21,244
泌尿/经腹妇科	5	7.5	10	75,137
腹部/胃肠	5	7.5	10	99,719
浅表器官	5	7.5	10	67,825
儿科专项	5	7.5	10	3,950
一般其他	5	7.5	10	3,400

为描述医生异质性，先选取同一医生、同一天的单项目事件，计算相邻报告间隔 g_n ，仅保留 $0 < g_n \leq 60$ 分钟。对医生 i ，由有效工作日吞吐量得到对数相对因子 h_i ，再作经验贝叶斯收缩：

$$\tilde{h}_i = \omega_i h_i + (1 - \omega_i) \mu_h, \quad \omega_i = \frac{m_i}{m_i + \lambda}, \quad (5)$$

其中 m_i 为有效工作日数， μ_h 为总体对数吞吐均值。医生一项目组合的调度时长取直接间隔中位数与题给名义时长经医生因子修正后的收缩组合，并截断在 $[l_k, u_k]$ 内。该方法形成 42 336 个完整组合、6 416 个直接证据组合。由于相邻报告间隔包含非扫描活动， $\tilde{h}_i$ 只用于相对速度与能力分析，患者级主排程仍采用题给绝对时长，以免把代理噪声固化为硬约束。基于检查类别预测超声检查时长具有现实依据 [11]，但本文对字段缺失作了更保守的解释。

5.1.3 项目—设备能力证据

设备能力从表 1 的检查项目文本逐项建立。对项目名称 q 和设备 r 定义三级匹配：

$$A_{qr} = \begin{cases} 2, & \text{项目名称精确、包含或语义核心匹配;} \\ 1, & \text{无项目级证据但项目族有具体能力证据;} \\ 3, & \text{文本明确为床旁且 } r = 7; \\ 0, & \text{否则。} \end{cases} \quad (6)$$

主情景允许 $A_{qr} \geq 1$ ，同时记录 $A_{qr} = 1$ 的回退任务；严格敏感性只允许 $A_{qr} \in \{2, 3\}$ 。宽泛描述如“常规项目”不能单独建立某个项目族的硬能力。图 3 显示，住院、门诊任务的严格或床旁证据分别达到 98.2% 和 91.2%，体检只有 69.4%，其余 30.6% 依赖类别回退。这一差异决定了问题三不能只报告总兼容率。

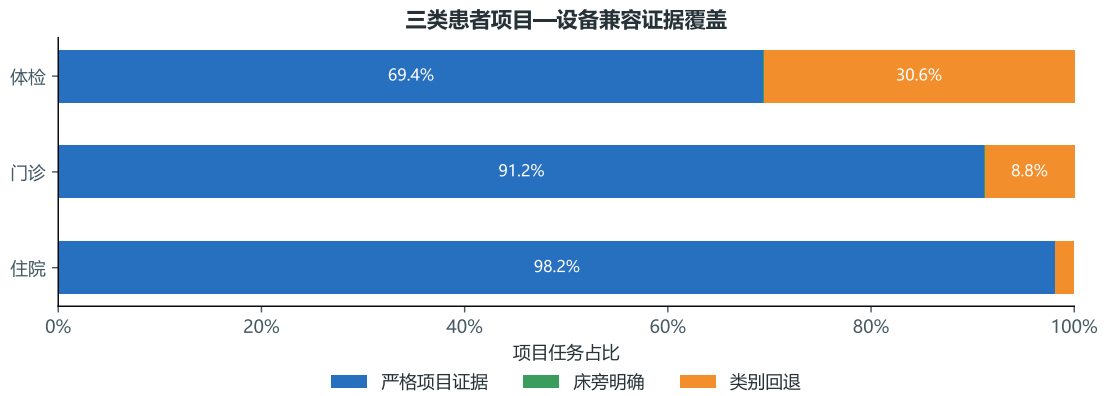


图 3 三类患者项目—设备兼容证据覆盖

关键资源见表 6。床旁仅 7 号机可执行；产科 III/IV 级集中于 64、43、65 号三台设备；儿科专项仅 45、44 号两台。心脏、血管等项目虽有较多可用设备，但长时长和高需求仍会形成负荷瓶颈。

表 6 关键资源的设备覆盖与结构判断

关键资源	可用设备数	设备编号	结构判断
床旁	1	7	唯一瓶颈
心脏	5	73 113 48 45 49	多房间
血管	12	40 69 72 71 41 42 46 47 44 74 75 77	多房间
III/IV 产科	3	64 43 65	少数房间
I/II 产科	13	40 41 42 46 47 44 64 43 65 160 74 75 77	多房间
儿科专项	2	45 44	少数房间
介入/定位	15	40 69 72 41 42 46 47 44 64 43 65 160 74 75 77	多房间

5.1.4 医生共享时隙容量

表 6 提供医生每日项目数及首末活动时刻。对训练日 d 和 5 分钟槽 t ，若 t 位于医生 i 重构活动区间内，则 $I_{idt} = 1$ 。星期 w 的主容量和偏紧容量分别为

$$c_{wt}^{(0)} = \min \left\{ 22, \text{Med}_{d: w(d)=w} \sum_i I_{idt} \right\}, \quad c_{wt}^{(L)} = \min \left\{ 22, Q_{0.25, d: w(d)=w} \sum_i I_{idt} \right\}. \quad (7)$$

首活动在 7–9 时之间归整为 8 时开班，12:30–14 时归整为 13 时开班；末活动在 11–12:30 归整为 12 时，在 16–19 时归整为 17 时，其他情况保留观察区间并向前扩 10 分钟。该规则只修复报告时间代理对班次边界的轻微偏移，不把单个医生复制成全天可用。

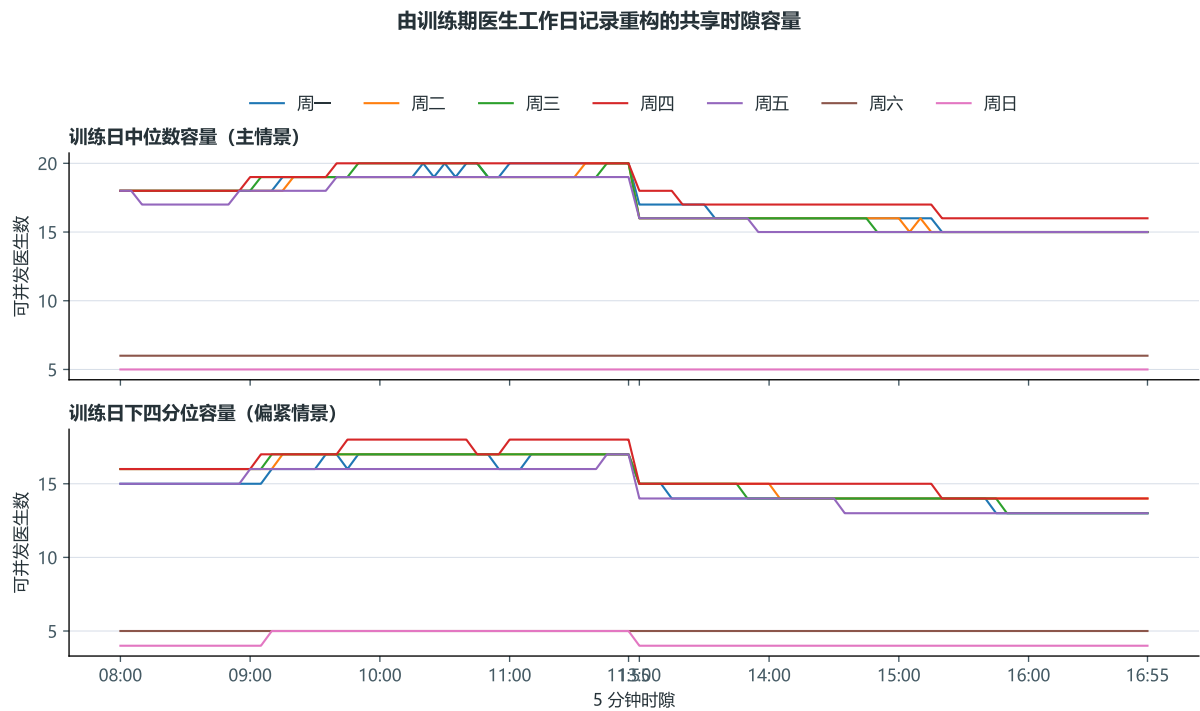


图 4 训练期重构的星期一时隙医生共享容量

表 7 各星期医生并发容量范围

星期	主情景			偏紧情景		
	最小	中位	最大	最小	中位	最大
周一	15	17.5	20	13	15	17
周二	15	17	20	14	15.5	17
周三	15	17	20	13	15.5	17
周四	16	18	20	14	15.5	18
周五	15	16.5	19	13	14.5	17
周六	6	6	6	5	5	5
周日	5	5	5	4	4	5

工作日主容量为 15–20 人，周六为 6 人，周日为 5 人；实际可同时运行的设备数取设备数 22 与 c_{wt} 的较小值。图 4 表明午后容量普遍略低，周末更明显。医生下四分位情景将用于检验人力不足风险。

5.1.5 峰平配置方案

综合需求、能力和医生容量，提出以下配置原则。

- 7 号机全天保留床旁任务，非床旁项目只有在确定不会挤占床旁窗口时才可回填；这是唯一确定性硬瓶颈。
- 64、43、65 号设备形成产科 III/IV 级保护池；45、44 号形成儿科保护池。其余兼容机房按项目稀缺度分配，灵活项目优先使用覆盖面较广且当前负荷较低的设备。

3. 平峰按训练期中位需求开放固定能力底座；高峰不固定新增某台“万能机”，而是在上午空腹块和下午非空腹块分别释放弹性时隙。体检团检高峰需提前导入团检名单，单独预留成块容量。
4. 5% 特殊病例不另设固定机房，而在每个工作块保留与上界时长差对应的缓冲。医生并发低于设备数时，先补医生班次而不是继续开放设备。

表 8 把上述原则落实为 22 台现有设备的运行配置。这里的“开放”表示允许排程占用，并不等于临时购置设备；任一时刻真正可并行工作的台数仍受 c_{wt} 限制。

表 8 22 台现有设备的峰平运行配置

资源池	台数	平峰配置	高峰/压力配置
床旁 7 号机	1	床旁任务保护，确认无待服务床旁任务时才回填	保持床旁专用，不以普通任务填满
产科 III/IV 级（64、43、65）	3	保留专项能力，按实际产科项目启用	三台均进入专项候选，空余槽才承接兼容任务
儿科专项（45、44）	2	保留儿科能力，空余槽可按证据回填	两台均保护儿科窗口，避免一般项目提前占用
其余兼容设备	16	按 $Q_{0.45}-Q_{0.55}$ 工作量滚动开放	按 $Q_{0.90}$ 需求和已知团检名单释放成块时隙； $Q_{0.95}$ 压力日先增医生班次

该方案既回答了“哪些机器配置哪些项目”，又保留了患者级模型根据当日项目组合动态选择具体机房的空间；固定配置负责保护稀缺能力，滚动排程负责利用可替代能力。

5.2 问题二：住院患者 48 小时排程模型

5.2.1 患者级离散时间模型

对住院事件 e ，项目 $j \in \mathcal{J}_e$ 需要 p_{ej} 个 5 分钟时隙，可用设备集合为 $\mathcal{R}_{ej}$ 。若项目在设备 r 、时隙 t 开始，令 $x_{ejrt} = 1$ 。每个项目至多开始一次：

$$\sum_{r \in \mathcal{R}_{ej}} \sum_{t \in \mathcal{T}_{ej}} x_{ejrt} \leq 1, \quad e \in \mathcal{E}, j \in \mathcal{J}_e, \quad (8)$$

其中 $\mathcal{T}_{ej}$ 已剔除释放前、截止后、跨午休和跨日的开始时隙。事件是否全部完成由

$$n_e y_e \leq \sum_{j \in \mathcal{J}_e} \sum_{r, t} x_{ejrt}, \quad y_e \in \{0, 1\} \quad (9)$$

连接；因此多项目患者缺少任一项目时 $y_e = 0$ 。

设备互斥约束为

$$\sum_{e, j} \sum_{t': t' \leq t < t' + p_{ej}} x_{ejrt'} \leq 1, \quad r \in \mathcal{R}, t \in \mathcal{T}. \quad (10)$$

医生是所有机房共享的并发资源：

$$\sum_{e, j, r} \sum_{t': t' \leq t < t' + p_{ej}} x_{ejrt'} \leq c_{w(t), s(t)}, \quad t \in \mathcal{T}, \quad (11)$$

其中 $w(t)$ 为星期， $s(t)$ 为日内槽。式 (11) 防止把同一批医生分别复制给 22 台设备。

5.2.2 患者内部顺序与准备约束

同一患者的项目必须串行。若项目 j 在 j' 之前, 记 $o_{ejj'} = 1$, 则

$$S_{ej'} \geq S_{ej} + p_{ej} + \tau v_{ejj'} - M(1 - o_{ejj'}), \quad (12)$$

$$S_{ej} \geq S_{ej'} + p_{ej'} + \tau v_{ej'j} - Mo_{ejj'}, \quad (13)$$

其中 $v_{ejj'} = 1$ 表示相邻项目不在同一设备, $\tau = 2$ 个时隙。对 $n_e > 2$ 的事件, 精确整数模型还需引入序位变量消除子环。大规模求解中直接构造一个完整项目序列, 天然满足式 (13)。

空腹项目的完成时刻满足

$$S_{ej} + p_{ej} \leq 10:00, \quad f_{ej} = 1. \quad (14)$$

憋尿事件的有效释放时刻为 $a'_e = a_e + \delta_e$, 主情景 $\delta_e = 60$ 分钟; 床旁项目令 $\mathcal{R}_{ej} = \{7\}$ 。其他项目不得因为患者包含床旁标记而自动绕过项目兼容, 每个项目仍按自身能力集合安排。

5.2.3 目标函数

问题二的首要目标是最大化 48 小时内完成全部项目的患者数:

$$\max Z_1 = \sum_{e \in \mathcal{E}_I} y_e. \quad (15)$$

在 Z_1 固定后, 依次最小化首检等待、跨室次数和设备负荷不均衡:

$$\min Z_2 = \sum_{e: y_e=1} (S_e - a'_e), \quad (16)$$

$$\min Z_3 = \sum_{e, j, j'} v_{ejj'}, \quad (17)$$

$$\min Z_4 = \sum_{r \in \mathcal{R}} \left| L_r - \frac{1}{|\mathcal{R}|} \sum_{r'} L_{r'} \right|. \quad (18)$$

采用词典序而非 $Z_1 - \alpha Z_2 - \beta Z_3$, 是为了避免任意权重让若干分钟等待抵消一个患者的 48 小时成功。带患者优先级和时限保证的动态诊断资源调度也常采用服务水平优先的结构 [12, 13]。

5.2.4 大规模构造算法

全年事件和项目数使全时域 MILP 规模过大。本文保留上述整数模型作为结构定义, 再采用与其共享可行域的确定性构造算法。

对每个事件, 先计算项目的可用设备数 $|\mathcal{R}_{ej}|$ 和工作量 p_{ej} , 形成四个候选项目序列: 空腹优先且稀缺优先、空腹优先且长任务优先、纯稀缺优先、原始项目序。若所有项目存在共同设备, 先在累计负荷最低的三个共同设备中依次寻找完整方案; 共同设备快捷搜索不可行时, 逐个候选序列在全部兼容设备中寻找最早可行时隙, 并在换室处插入 10 分钟转运。因此“三个”只是全年构造算法的同室优先搜索宽度, 不是现实中的设备数量限制, 也不从可行集合中删除其他设备。只有事件全部项目都找到时才提交占用; 任一项目失败则释放本次试排的全部槽, 不产生“半个患者已排”的状态。

设备选择不仅看当前负荷，还定义背景需求稀缺权重

$$\kappa_r = \sum_{e,j:r \in \mathcal{R}_{ej}} \frac{p_{ej}}{|\mathcal{R}_{ej}|}. \quad (19)$$

式中求和只使用 2019 年 3 月 1 日至 2024 年 3 月 31 日训练期形成的项目目录与时长，不读取留出期项目构成。在同一最早时刻有多台设备可行时，先保留患者上一个房间，再选累计负荷低、 κ_r 较小的设备；这使灵活任务尽量不占用稀缺设备。

5.2.5 策略定义

比较三种策略：

1. 共享 **FCFS**：门诊/体检按计划日和释放顺序排入，住院按释放时刻排序；设备先取最早可行时刻，同刻再按编号选择。
2. 松弛度保护：背景部分与共享 FCFS 相同；住院按 $\sigma_e = d_e - p_e$ 排序，其中 $p_e = \sum_j p_{ej}$ ，优先处理剩余余量小的多项目患者。
3. 联合稀缺度：背景事件先按计划日，再按空腹、最小可用设备数和工作量排序；住院采用松弛度排序，具体设备按负荷与稀缺度选择。

三种策略使用完全相同的事件、特殊病例、时长、设备集合和医生容量，唯一差异是排序与同刻设备择优。因此策略差异可以归因于排程规则，而不是输入变化。

5.2.6 问题二结果与推荐方案

在问题三的门诊/体检共享压力下，三种策略结果见表 9 和图 5。单纯松弛度保护的住院 48 小时率为 63.75%，低于共享 FCFS 的 63.95%，说明提前放置长且稀缺的多项目患者会增加局部碎片。联合稀缺度把背景预约日率由 59.70% 提高到 61.61%，但住院率降至 63.31%；它与共享 FCFS 互不支配，而不是双目标占优。

表 9 主情景三种患者级策略的留出期结果

策略	住院 48h 完成数	住院率	背景完成数	背景率	等待 P90/h
共享 FCFS	45,260	63.95%	57,341	59.70%	40.42
松弛度保护	45,116	63.75%	57,341	59.70%	40.25
联合稀缺度	44,805	63.31%	59,175	61.61%	41.50

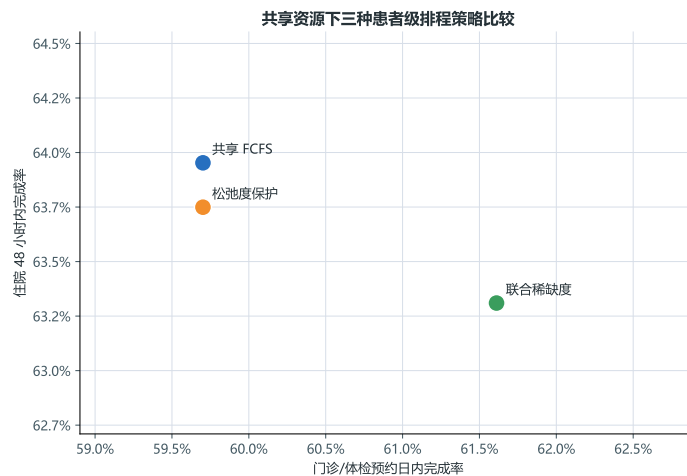


图 5 共享资源下三种排程策略比较

共享 FCFS 成功排入 45 260 个留出期住院事件，未分配 25 511 个。成功者首检等待中位数 4.50 小时、P90 为 40.42 小时、P95 为 45.58 小时；多项目患者跨室率为 19.96%。它比松弛度保护多完成 144 个住院事件；相对联合稀缺度则多完成 455 个住院事件，但少完成 1 834 个门诊/体检计划日事件。由于赛题把住院 48 小时率设为唯一考核指标，且共享 FCFS 的背景率就是本文定义的不劣基线，最终推荐资源可行 FCFS；若医院把背景服务下限提高到 61.61% 附近，则应切换到联合稀缺度帕累托方案，而不能声称一套策略同时改善所有指标。

表 10 给出推荐清单的开头部分。完整清单逐事件给出患者序位、项目序位、项目族、开始与结束时刻、设备、准备标记、转运时间和兼容证据层，可直接转换为预约系统导入表。

表 10 留出期住院患者推荐排程样例

患者序位	事件编号	项目序位	项目族	时段	设备
1	IP2024-000303	1	泌尿/经腹妇科	04-01 11:05–11:15	门 16(65)
2	IP2024-000310	1	产科 I/II 级	04-01 11:05–11:25	门 17(160)
3	IP2024-000265	1	浅表器官	04-01 11:10–11:20	病 3(72)
4	IP2024-000281	1	浅表器官	04-01 11:10–11:20	妇门 1(74)
5	IP2024-000261	1	浅表器官	04-01 11:15–11:25	妇门 3(77)
6	IP2024-000267	1	浅表器官	04-01 11:15–11:25	病 1(69)
7	IP2024-000315	1	产科 I/II 级	04-01 11:20–11:40	妇门 1(74)
8	IP2024-000332	1	一般其他	04-01 11:20–11:30	妇门 2(75)

5.3 问题三：三类患者联合排程

5.3.1 共享压力场景的构造

门诊和体检数据没有预约时刻。为回答“在三类患者同时占用设备时，住院 48 小时率能达到多少”，本文把两类患者的历史报告日期作为回顾性计划日，构造必须在该日工作块内完成的背景任务。该场景使用患者项目组合和日期负荷，但剔除报告时分、历史机房与历史医生。因此，它反映的是“当日需求压力下的可排程度”，不是对历史派单的重放。

背景事件的基础释放时刻为历史开单时刻与计划日工作块起点的较晚者，截止为计划日工作块终点；空腹任务只能进入上午并在 10 时前结束。对憋尿任务，准备完成时刻取开单时刻加准备提前量，最终释放为该时刻与计划日工作块起点的较晚者；这允许已提前准备的门诊/体检患者在开班时进入排程，而不会一律延迟到 9 时。住院事件仍使用精确开单时刻、准备约束和 48 小时截止。连续日历覆盖 2024 年 3 月 30 日至 2025 年 4 月 2 日：边界前 2 日承接尚未逾期的在制任务，边界后 2 日只用于完成 3 月 31 日开单患者的 48 小时观察窗；评价仅统计 2024 年 4 月 1 日至 2025 年 3 月 31 日释放或计划的事件。

5.3.2 背景服务下限与住院主目标

题面要求在满足门诊、体检需求的前提下最大化住院 48 小时率，但未给出背景服务的数值阈值。若直接把背景完成数放在住院指标之前，会在背景多排少量患者时牺牲题面唯一考核指标；若完全忽略背景，又不符合业务前提。因此本文以共享 FCFS 得到的背景计划日完成数 B_0 作为“不比基线差”的可审计下限：

$$Z_B = \sum_{e \in \mathcal{E}_O \cup \mathcal{E}_P} y_e \geq B_0. \quad (20)$$

在满足该下限的候选策略中，首先最大化住院 48 小时完成数

$$\max Z_I = \sum_{e \in \mathcal{E}_I} y_e. \quad (21)$$

随后才比较背景增量、住院等待、跨室和设备负荷差。共享 FCFS 和联合稀缺度均满足 $Z_B \geq B_0$ ，前者住院指标更高，故作为主推荐；后者背景服务率更高，作为医院提高背景下限时的帕累托备选。若医院能给出正式门诊/体检服务水平或急诊等级，应以该阈值和急诊硬保护替代 B_0 ，不能长期把历史基线当成临床制度。

主策略先按计划日与释放时刻安排背景患者，再按住院释放时刻（由于截止均为开单后 48 小时，亦等价于最早截止顺序）进入共享日历；每个患者内部仍采用能力稀缺优先、同室优先和最早可行时隙搜索。因此“FCFS”只描述患者间主序，不等于忽略设备能力的简单排队。联合稀缺度备选则先按空腹标记、项目最小可用设备数和工作量安排背景任务，再按住院松弛度排入余量。两种过程都只初始化一次房间占用和医生负荷状态。

5.3.3 联合排程结果

主情景中，主推荐策略在留出期完成 57 341 个背景事件，计划日完成率 59.70%；其中门诊完成 42 871 个，体检完成 14 470 个。住院完成 45 260 个，48 小时率 63.95%。联合稀缺度备选完成 59 175 个背景事件，背景率 61.61%，但住院只完成 44 805 个、住院率 63.31%。两个比例的分母均包含未分配事件，未因无可行设备或搜索失败而删除。

图 6 给出 14 日滚动完成率；14 日只用于压低日级随机波动、辨认趋势，不进入排程规则。住院率大部分时间在 60%–75% 之间，门诊/体检在 2024 年秋季出现明显下降、2025 年初回升，与体检团检波动和背景总负荷一致。年度平均值不能替代日级监控；实际部署时应同时查看完成率、未来 48 小时已知需求和稀缺项目分钟数，再决定是否调班。

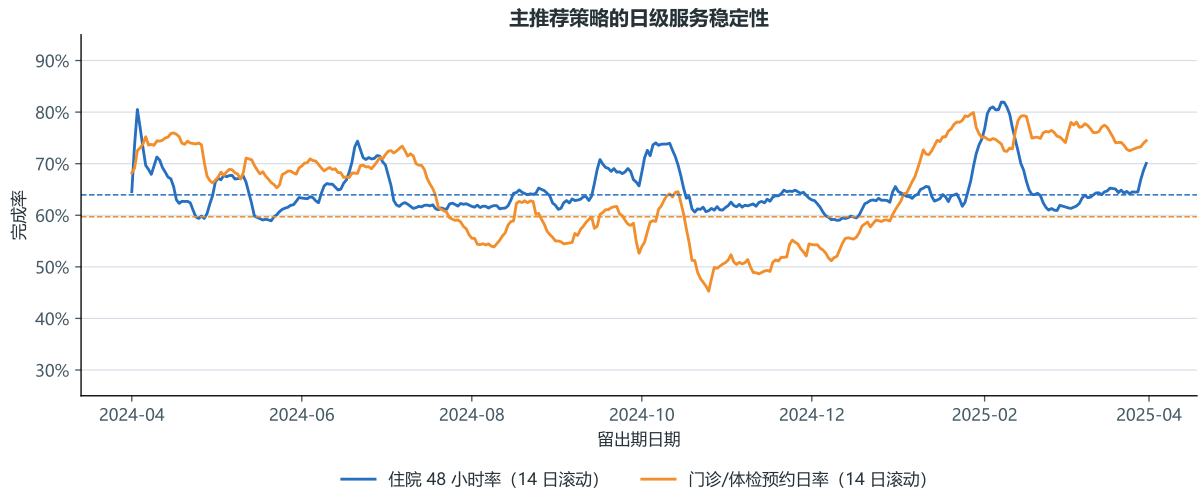


图 6 主推荐策略的日级服务稳定性

图 7 展示主推荐策略留出期工作量最高的 2024 年 10 月 14 日。排程在 12–13 时保持午休断点，三类患者在各机房交错安排；床边 B 超主要服务住院患者，产科和门诊机房在上午承接体检与门诊，下午逐步释放给住院任务。图中每个色块均来自一条实际患者项目记录，不是总容量示意。

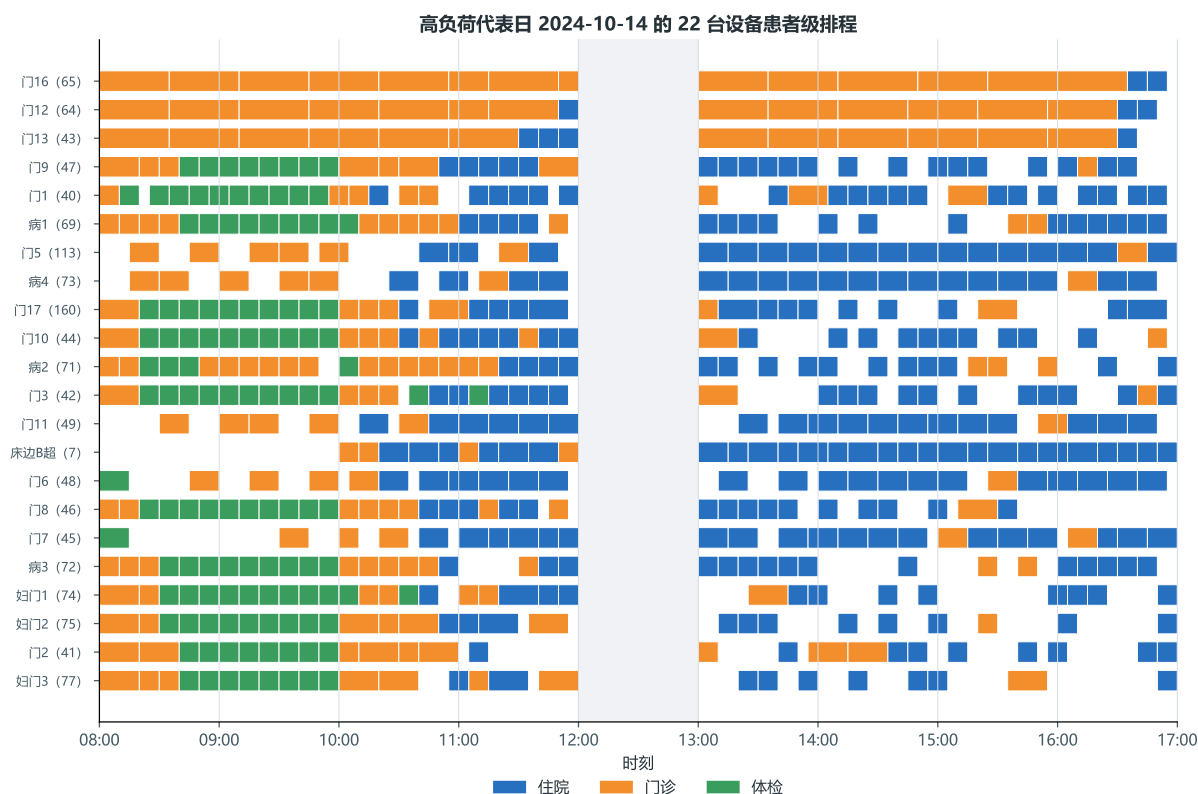


图 7 高负荷代表日的 22 台设备患者级排程

设备日均占用率见图 8. 7 号床边 B 超达到 73.95%，显著高于其他设备的 36.16%–57.05%，与表 6 的唯一床旁能力一致。其余普通设备存在一定负荷差异；产科 III/IV 级设备占用相对较低并不表示可以撤销，因为它们承担的是不可替代的专项能力。

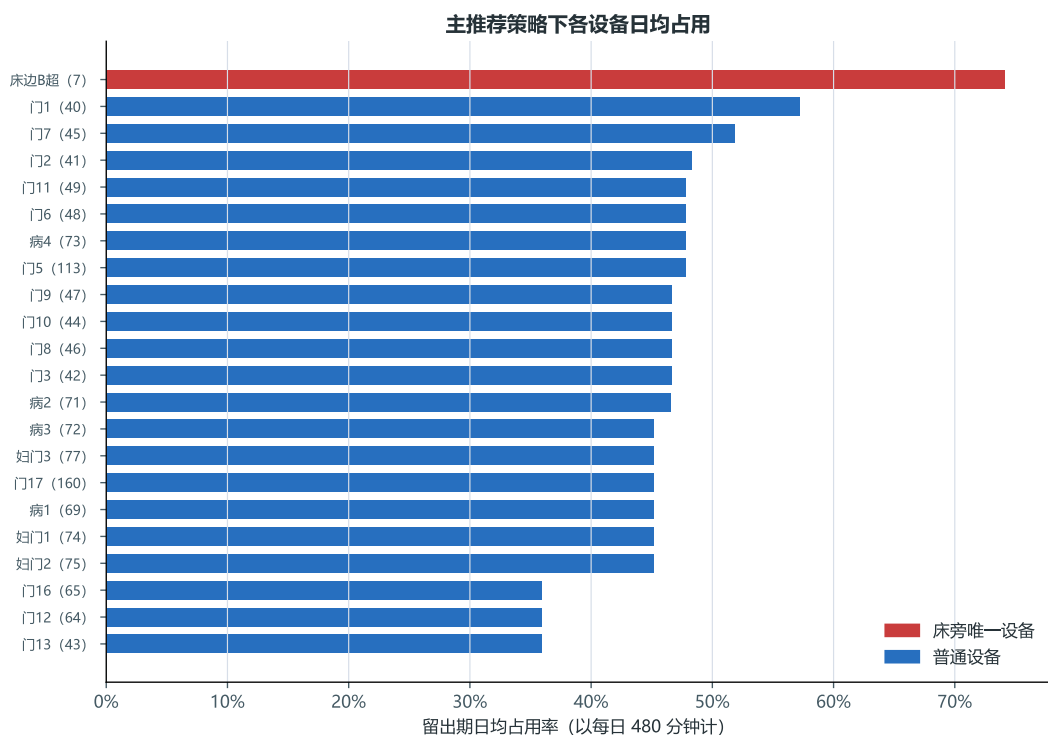


图 8 主推荐策略下各设备日均占用率

5.3.4 联合方案的实施建议

由结果得到四项实施建议。

- 1. 每次发布下一轮计划前，用未来 48 小时已知门诊、体检计划和住院在制队列重新构造滚动日历；已确认预约作为冻结层，新增住院患者只在余量中插入。
- 2. 7 号床边 B 超按未来 48 小时床旁任务上界分钟保护容量；当剩余容量已不能覆盖床旁需求或最紧迫床旁事件时，停止非床旁回填，并由医院决定是否启用具备床旁能力的备用设备或移动班次。
- 3. 对 64、43、65 号产科高等级设备和 45、44 号儿科设备保留能力池，不以短期占用率为唯一裁撤依据；灵活项目优先落到能力覆盖广的普通设备。
- 4. 若预计医生并发达到 c_{wt} ，优先调整医生班次而不是继续开放空机房。新增设备只有与相应医生资质和班次同时增加，才能转化为完成率。

6 模型的检验

6.1 独立可行性核验

为避免调度器用自身逻辑证明自身正确，另写独立核验程序，只读取统一输入、最终排程和结果表，重新计算任务时长、占用区间和医生并发。核验对象为 151 020 条项目任务、102 986 个已排事件。检查组见表 11，所有违规数均为 0；最大观测医生并发为 20，等于主容量最大值而未超出。

表 11 最终联合排程的独立核验结果

核验项目	违规数	结论
任务键、输入对应、时长	0	通过
设备兼容、床旁位置	0	通过
释放、截止、工作时段	0	通过
房间、患者、转运冲突	0	通过
医生并发容量	0	通过
结果表聚合一致性	0	通过

除表中项目外，还检查了 5 分钟网格、项目序位连续、空腹 10 时前完成、床旁 7 号机和输出聚合标记。所有成功事件的项目数均等于输入服务项目数；所有未成功事件在结果表中保留失败原因“截止前无完整可行方案”。因此 63.95% 不是项目级完成比例，也不是删除未排患者后的条件比例。

6.2 补充敏感性情景

赛题明确要求考虑峰/平配置和约 5% 复杂病例；二者已进入主模型。为审核数据证据与假设边界，本文另设置七个补充检验，它们不是赛题指定情景：取消 5% 时长上浮、医生容量采用训练日下四分位、只允许严格项目级设备证据、憋尿准备 45/90 分钟、跨室转运 5/15 分钟。每次只改变一个因素，结果见表 12 和图 9。

表 12 主推荐策略的补充敏感性检验

情景	住院 48h 率	背景预约日率	条件等待 P90/h	排入分钟
主情景：5% 上界时长	63.95%	59.70%	40.42	1,819,800
无特殊时长上浮	63.97%	59.70%	40.42	1,819,660
医生容量下四分位	61.96%	58.95%	42.00	1,769,275
仅严格设备兼容	71.57%	49.67%	45.33	1,664,965
憋尿准备 45 分钟	63.89%	59.88%	40.25	1,821,400
憋尿准备 90 分钟	64.12%	59.17%	40.17	1,815,485
跨室转运 5 分钟	63.93%	59.71%	40.50	1,821,800
跨室转运 15 分钟	63.99%	59.76%	40.33	1,819,020

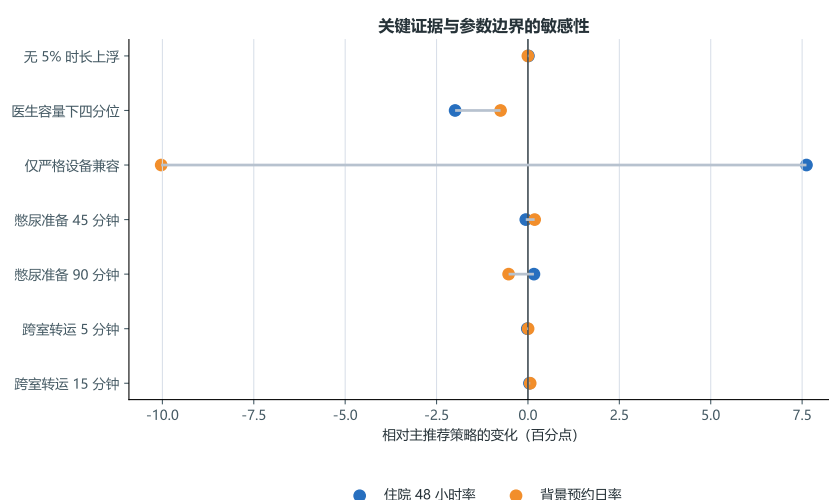


图 9 补充证据与参数边界相对主情景的变化

取消特殊时长上浮后住院率只增加 0.013 个百分点，背景率几乎不变。原因是多数项目上界与名义值在 5 分钟离散后差异有限，且系统瓶颈主要来自项目兼容和峰值到达。该结果不否定赛题的 5% 条件，而是说明其在当前时长分档下边际影响较小。

医生容量降至下四分位后，住院率由 63.95% 降至 61.96%，背景率由 59.70% 降至 58.95%，住院首检等待中位数由 4.50 小时升至 8.08 小时，表明住院 48 小时目标对医生班次明显敏感。相比之下，单纯增加普通设备若没有医生同步到位，改善有限。

只允许严格设备证据时，背景率降至 49.67%，住院率反升至 71.57%。这不是模型变好，而是大量体检和门诊项目因缺少严格项目级匹配被直接判为不可排，释放了容量给住院。该现象说明项目能力覆盖必须用两类指标联合解释；部署前应由超声科逐条确认 50 个回退项目的可用设备，确认后再冻结正式能力矩阵。

憋尿准备 45 分钟时住院率为 63.89%，90 分钟时为 64.12%，背景率分别为 59.88% 和 59.17%。准备更晚会使部分背景事件无法在计划日完成，却为住院释放少量容量；准备更早又会改变排序与碎片。因此准备时长不是单调优化旋钮，应由预约通知、到院时间和实际准备状态共同决定。

跨室转运 5、10 和 15 分钟时，住院率分别为 63.93%、63.95% 和 63.99%，背景率分别为 59.71%、59.70% 和 59.76%。完成率在三点上变化较小但非单调，这是离散时隙、候选顺序和局部容量碎片共同作用的结果，不表示更慢转运能提升临床效率。上线时应用实测诊室动线替代该参数。

6.3 结构合理性分析

本文结果与诊断资源调度文献中的两个结论一致：一是混合患者资源应把事前容量保护与动态优先级结合 [14, 15]；二是超声预约应同时考虑检查室分配和随机时长，而不是只给患者时间不给房间 [16]。不同之处在于，本文没有用仿真分布代替题目缺失字段，而是以题给区间构造可解释上下界，并把项目能力证据强弱直接放入敏感性。

7 模型的评价

7.1 模型优点

1. 口径完整。事件、项目、设备和医生均有明确单位；未分配患者保留在分母，避免把项目完成率冒充患者完成率。
2. 约束统一。问题二和问题三共用同一房间、医生、患者、兼容、准备和转运规则，主模型与回放模型不存在可行域差异。
3. 证据可追溯。设备严格匹配、床旁明确和类别回退分别记录；训练期与留出期物理隔离，未来报告时刻和历史派单不进入决策。
4. 方案可执行。输出下沉到每位住院患者的项目次序、开始时刻和机房，并给出代表日完整甘特图，而非仅报告总容量或理论上界。
5. 权衡透明。联合策略同时报告两类服务率和等待分位数；严格兼容情景的住院虚增被识别为服务覆盖下降，而不是误写为性能提升。

7.2 模型局限

第一，历史数据缺少真实预约时刻、扫查起止和报告提交时刻。本文用题给区间确定绝对时长，用最后项目结束代理报告完成，因此无法分离扫查与报告环节。第二，未来医生只有总并发容量，没有姓名、项目资质和班表；模型保证总人数不超限，但不能保证某一时刻恰有具备专项资质的医生。第三，门诊和体检只构造回顾性计划日压力，不能还原真实预约提前期、迟到、爽约和插队。第四，题面没有给出背景服务下限，本文只能以共享 FCFS 服务率构造不劣基线；若医院给出更高阈值，策略选择会沿帕累托前沿改变。第五，构造算法在给定策略族中求最早可行方案，未证明对全年大规模整数模型达到全局最优；本文的“最佳”只指三种已比较策略中住院主指标最高且背景不低于基线。

8 模型的改进与推广

8.1 数据与求解方法的改进

若医院能够补充扫查开始、扫查结束、报告提交、预约时刻和医生项目资质，模型可作三项升级：其一，分别估计扫查时长和报告时长，以双阶段队列替代完成代理；其二，以医生一项目资质矩阵替代总并发容量，形成设备与医生双匹配；其三，用真实预约、迟到与爽约分布进行滚动随机优化，并把已确认预约冻结、未确认预约柔性调整。运行层面的刷新频率和冻结窗口应由叫号提前量、患者到院状态及系统响应时间标定，不在缺少现场数据时预设固定分钟数。

8.2 医院内部推广与跨机构迁移

模型的可推广部分是“事件—项目—双资源—时间窗”结构，而不是 A 医院的具体房号、日班时间或 10 分钟转运。在 A 医院内部，心电、CT、MRI 等多资源检查可沿用患者完整性、能力兼容和截止期约束，但必须重新定义服务时长、准备要求和人员资质。迁移到其他医院时，应先以当地设备清单、班表和患者动线替换配置参数，再用本地时间顺序留出期评价；不应直接移植本文的 63.95% 结果。

部署可采用“计划层—执行层—反馈层”闭环。计划层每日生成未来 48 小时容量保护；执行层读取到院、准备完成、急诊插入和设备停机状态；反馈层按日监控两类完

成率、未排原因、等待分位数和改号次数。只有在前瞻试点中同时保持门诊/体检服务和住院 48 小时指标，才将新策略固化为正式规则。

8.3 主要结论

本文完成了超声预约问题的患者级联合建模。问题一得到训练期科室需求、11 类项目时长、22 台设备能力证据和星期一 5 分钟医生容量；床旁 7 号机是最明确的单点瓶颈。问题二在完整共享资源约束下输出住院患者项目顺序、时段和机房；问题三把门诊、体检与住院置于同一连续年度日历。以共享 FCFS 背景率为不劣基线时，资源可行 FCFS 在留出期取得住院 63.95%、背景 59.70% 的完成率，是三个候选策略中住院主指标最高者；联合稀缺度以住院率降至 63.31% 为代价把背景率提高到 61.61%，构成另一帕累托点。独立核验未发现约束违规，但医生容量和设备项目证据对结果影响明显。因而实施路线不是单纯增加设备，而是先确认正式背景服务阈值和项目能力清单、保护床旁与专项设备、保障高峰医生并发，再沿可接受的服务权衡选择患者级滚动排程策略。

9 参考文献

- [1] GUPTA D, DENTON B. Appointment scheduling in health care: Challenges and opportunities[J]. IIE Transactions, 2008, 40(9): 800–819. DOI: <https://doi.org/10.1080/07408170802165880>.
- [2] AHMADI-JAVID A, JALALI Z, KLASSEN K J. Outpatient appointment systems in healthcare: A review of optimization studies[J]. European Journal of Operational Research, 2017, 258(1): 3–34. DOI: <https://doi.org/10.1016/j.ejor.2016.06.064>.
- [3] MARYNISSEN J, DEMEULEMEESTER E. Literature review on multi-appointment scheduling problems in hospitals[J]. European Journal of Operational Research, 2019, 272(2): 407–419. DOI: <https://doi.org/10.1016/j.ejor.2018.03.001>.
- [4] APERGI L A, BARAS J S, GOLDEN B L, et al. An optimization model for multi-appointment scheduling in an outpatient cardiology setting[J]. Operations Research for Health Care, 2020, 26: 100267. DOI: <https://doi.org/10.1016/j.orhc.2020.100267>.
- [5] 国家卫生健康委员会办公厅. 国家卫生健康委办公厅关于印发超声诊断等 5 个专业医疗质量控制指标（2022 年版）的通知：国卫办医函（2022）161 号 [EB/OL]. (2022-05-27)[2026-08-24]. <https://www.nhc.gov.cn/yzygj/c100068/202205/164e6556f6d34c33a3344fec9a2c5074.shtml>.
- [6] VAN DER AALST W M P, SCHONENBERG M H, SONG M. Time prediction based on process mining[J]. Information Systems, 2011, 36(2): 450–475. DOI: <https://doi.org/10.1016/j.is.2010.09.001>.
- [7] SEXAUER R, BESTLER C. Time is money: Considerations for measuring the radiological reading time[J]. Journal of Imaging, 2022, 8(8): 208. DOI: <https://doi.org/10.3390/jimaging8080208>.
- [8] 华容区人民医院. 超声科流程及注意事项 [EB/OL]. (2025-08-21)[2026-08-24]. http://www.hbhr.gov.cn/hrqxxgk/xxgkml/ggqsydwxxgk/wsjk/hrqrmyy/202508/t20250821_720866.html.
- [9] LUO L, LUO L, ZHANG X, et al. Hospital daily outpatient visits forecasting using a combinatorial model based on ARIMA and SES models[J]. BMC Health Services Research, 2017, 17: 469. DOI: <https://doi.org/10.1186/s12913-017-2407-9>.
- [10] LIN C K Y. Dynamic appointment scheduling with forecasting and priority-specific access time service level standards[J]. Computers & Industrial Engineering, 2019, 135: 970–986. DOI: <https://doi.org/10.1016/j.cie.2019.06.049>.
- [11] LAI C H, LU Y J, CHEN P S. Using data-driven techniques to predict outpatient ultrasound examination time for the multi-clinic outpatient appointment scheduling problem[J]. Communications in Statistics-Simulation and Computation, 2025, 54(9): 3358–3376. DOI: <https://doi.org/10.1080/03610918.2024.2349168>.
- [12] PATRICK J, PUTERMAN M L, QUEYRANNE M. Dynamic multipriority patient scheduling for a diagnostic resource[J]. Operations Research, 2008, 56(6): 1507–1525. DOI: <https://doi.org/10.1287/opre.1080.0590>.

- [13] BAUERHENNE C, KOLISCH R, SCHULZ A S. Robust appointment scheduling with waiting time guarantees[J]. *Manufacturing & Service Operations Management*, 2026, 28(3): 995–1009. DOI: <https://doi.org/10.1287/msom.2024.0852>.
- [14] GREEN L V, SAVIN S, WANG B. Managing patient service in a diagnostic medical facility[J]. *Operations Research*, 2006, 54(1): 11–25. DOI: <https://doi.org/10.1287/opre.1060.0242>.
- [15] PATRICK J, PUTERMAN M L. Improving resource utilization for diagnostic services through flexible inpatient scheduling: A method for improving resource utilization[J]. *Journal of the Operational Research Society*, 2007, 58(2): 235–245. DOI: <https://doi.org/10.1057/palgrave.jors.2602242>.
- [16] CHEN P S, CHEN G Y H, LIU L W, et al. Using simulation optimization to solve patient appointment scheduling and examination room assignment problems for patients undergoing ultrasound examination[J]. *Healthcare*, 2022, 10(1): 164. DOI: <https://doi.org/10.3390/healthcare10010164>.

A 交付结果与复现顺序

本附录属于正文之外的支撑材料。完整复现顺序为：原始工作簿转存为 Parquet；运行问题一主分析、科室需求、设备能力和医生一项目时长估计；构建统一患者事件；运行联合调度；运行独立核验；最后生成论文图表。随机种子在代码中固定，评价期与预热期分别按事件数最近整数抽取 5% 复杂病例。

主结果包括三种策略与七个补充敏感性情景的指标表、最终患者任务排程、患者结果、住院推荐表、设备日占用和独立核验报告。`final_patient_task_schedule.csv` 为本文唯一最终患者级方案；联合稀缺度只作为门诊/体检服务率更高的帕累托备选，旧的总容量线性模型和按月重置回放不作为本文证据。

B 问题一主分析完整程序

Listing 1 项目分类、需求趋势、时长与医生效率分析

```
1 """P1 demand, project-family, and workload-proxy analysis.
2
3 The source workbooks and frozen Parquet files are read-only inputs. Results and
4 figures are written below supporting_materials by default.
5 """
6
7 from __future__ import annotations
8
9 import argparse
10 import json
11 import math
12 import re
13 from pathlib import Path
14 from typing import Any
15
16 import matplotlib
17
18 matplotlib.use("Agg")
19 import matplotlib.pyplot as plt
20 import numpy as np
21 import pandas as pd
22
23
24 ROOT = Path(__file__).resolve().parents[2]
25 SUPPORTING_DIR = ROOT / "supporting_materials"
26 SEED = 20260823
27 START_DATE = pd.Timestamp("2019-03-01")
28 END_DATE = pd.Timestamp("2025-03-31")
29 VALIDATION_START = pd.Timestamp("2024-04-01")
30 VALIDATION_END = pd.Timestamp("2025-03-31")
31
32 DETAIL_RENAME = {
33     "开单日期（年月日）": "order_date_raw",
34     "开单时间（00:00）": "order_time_raw",
35     "患者类型": "patient_type",
36     "病人ID": "patient_id",
37     "开单医生ID": "order_doctor_id",
38     "科室ID": "department_id",
39     "医嘱系统开单内容": "ordered_project",
40     "检查系统开单内容": "exam_project",
41     "检查报告出具日期（年月日）": "report_date_raw",
42     "检查报告出具时间（00:00）": "report_time_raw",
43     "检查人ID": "exam_doctor_id",
44     "检查超声的机器ID": "machine_id",
45 }
46
```

```

47 SOURCE_FILES = {
48     "住院": "inpatient.parquet",
49     "门诊": "outpatient.parquet",
50     "体检": "physical_exam.parquet",
51 }
52
53 NON_SERVICE_PATTERN = (
54     r"^(?:床旁(?:B超检查|彩色多普勒超声检查)加收|"
55     r"彩色胶片打印|超声计算机图文报告)$"
56 )
57
58 # The first matching rule wins. Obstetric rules precede cardiac because
59 # "胎儿心脏" is an obstetric level-IV item rather than a routine cardiac item.
60 CATEGORY_RULES = [
61     {
62         "category": "非服务计费/报告项",
63         "pattern": NON_SERVICE_PATTERN,
64         "basis": "整条内容仅为床旁加收、胶片打印或图文报告，不单独占用扫查时间；床旁标记仍保留",
65         "lower": 0.0,
66         "nominal": 0.0,
67         "upper": 0.0,
68     },
69     {
70         "category": "产科III/IV级",
71         "pattern": r"(?:III|IV|\u2162|\u2163|三|四)级产科|胎儿心脏|系统产前超声",
72         "basis": "题面明确III/IV级产科为30--40分钟；胎儿心脏归入IV级资源族",
73         "lower": 30.0,
74         "nominal": 35.0,
75         "upper": 40.0,
76     },
77     {
78         "category": "产科I/II级",
79         "pattern": r"(?:I|II|\u2160|\u2161|一|二)级产科|早孕|胎儿生物物理|母胎医学|宫颈管长度|产科超声",
80         "basis": "题面明确I/II级产科为15--20分钟；其余产科筛查按同资源族管理",
81         "lower": 15.0,
82         "nominal": 17.5,
83         "upper": 20.0,
84     },
85     {
86         "category": "心脏",
87         "pattern": r"心脏|心包|心动图|冠脉|室壁|瓣膜|经食道|经食管",
88         "basis": "题面明确心脏超声为10--15分钟；心脏声学造影仍属于心脏项目，故优先于一般“造影”
89         规则",
90         "lower": 10.0,
91         "nominal": 12.5,
92         "upper": 15.0,
93     },
94     {
95         "category": "介入/定位",
96         "pattern": r"穿刺|活检|置管|造影|定位|介入|引导",
97         "basis": "在排除心脏项目后，操作性项目按介入/定位资源管理；题面未另给时长，采用其他项目区
98         间",
99         "lower": 5.0,
100        "nominal": 7.5,
101        "upper": 10.0,
102    },
103    {
104        "category": "血管",
105        "pattern": r"血管|动脉|静脉|大隐|小隐|血流图|加血流",
106        "basis": "由项目名称识别血管专用资源族；题面未另给时长，采用其他项目区间",
107        "lower": 5.0,
108        "nominal": 7.5,
109        "upper": 10.0,
110    }
111 ]

```

```

108     },
109     {
110         "category": "经阴道/腔内",
111         "pattern": r"经阴道|腔内彩超|妇科B超（经阴道）|妇科B超\\（经阴道\\)",
112         "basis": "由检查路径识别腔内资源族；题面未另给时长，采用其他项目区间",
113         "lower": 5.0,
114         "nominal": 7.5,
115         "upper": 10.0,
116     },
117     {
118         "category": "泌尿/经腹妇科",
119         "pattern": r"泌尿|输尿管|膀胱|前列腺|双肾|肾彩超|尿量|经腹妇科|妇科B超（经腹）|妇科B超\\（经腹\\）|盆底",
120         "basis": "题面要求泌尿系和经腹妇科考虑充盈膀胱约束；时长采用其他项目区间",
121         "lower": 5.0,
122         "nominal": 7.5,
123         "upper": 10.0,
124     },
125     {
126         "category": "腹部/胃肠",
127         "pattern": r"腹部|腹水|腹盆|肝|胆|胰|脾|胃|肠道|阑尾|肠系膜|腹腔|腹膜后|膜腹后|肾上腺",
128         "basis": "题面要求腹部及肠系膜项目空腹并优先上午；时长采用其他项目区间",
129         "lower": 5.0,
130         "nominal": 7.5,
131         "upper": 10.0,
132     },
133     {
134         "category": "浅表器官",
135         "pattern": r"甲状腺|甲状旁腺|乳腺|淋巴结|体表|包块|阴囊|睾丸|附睾|涎腺|浅表|腹股沟|双眼",
136         "basis": "由项目部位识别通用浅表资源族；时长采用其他项目区间",
137         "lower": 5.0,
138         "nominal": 7.5,
139         "upper": 10.0,
140     },
141     {
142         "category": "儿科专项",
143         "pattern": r"婴儿头颅|新生儿|髋关节",
144         "basis": "设备清单显示儿科专项能力差异；题面未另给时长，采用其他项目区间",
145         "lower": 5.0,
146         "nominal": 7.5,
147         "upper": 10.0,
148     },
149     {
150         "category": "一般其他",
151         "pattern": r".*",
152         "basis": "未命中前述资源族，按题面其他项目5--10分钟",
153         "lower": 5.0,
154         "nominal": 7.5,
155         "upper": 10.0,
156     },
157 ]
158
159 CATEGORY_ORDER = [rule["category"] for rule in CATEGORY_RULES]
160 CATEGORY_RANK = {category: index for index, category in enumerate(reversed(
161     CATEGORY_ORDER))}
162 DURATION = {
163     rule["category"]: (rule["lower"], rule["nominal"], rule["upper"])
164     for rule in CATEGORY_RULES
165 }
166
167 FLAG_PATTERNS = {
168     "床旁标记": r"床旁",

```

```

168 "空腹上午标记": r"腹部|肝胆|胃肠|肠道|阑尾|肠系膜|胰|脾",
169 "充盈膀胱标记": r"泌尿|输尿管|膀胱|前列腺|经腹妇科|妇科B超(经腹)|妇科B超\((经腹\)|盆底",
170 }
171
172
173 def clean_text(series: pd.Series) -> pd.Series:
174     return series.astype("string").str.strip()
175
176
177 def clean_id(series: pd.Series) -> pd.Series:
178     return (
179         clean_text(series)
180         .str.replace(r"^\['']+|[\']+$", "", regex=True)
181         .str.replace(r"\.0$", "", regex=True)
182     )
183
184
185 def combine_datetime(date_series: pd.Series, time_series: pd.Series) -> pd.Series:
186     dates = pd.to_datetime(date_series, errors="coerce", format="mixed").dt.normalize()
187     parts = clean_text(time_series).str.extract(
188         r"(?P<hour>\d{1,2}):(?P<minute>\d{2})(?::(?P<second>\d{2}(?:\.\d+)??))?",
189         expand=True,
190     )
191     seconds = (
192         pd.to_numeric(parts["hour"], errors="coerce") * 3600
193         + pd.to_numeric(parts["minute"], errors="coerce") * 60
194         + pd.to_numeric(parts["second"], errors="coerce").fillna(0)
195     )
196     return dates + pd.to_timedelta(seconds, unit="s")
197
198
199 def normalize_project(series: pd.Series) -> pd.Series:
200     result = clean_text(series).fillna("")
201     for old, new in {
202         "(": "(",
203         ")": ")",
204         ",": ",",
205         " ": " ",
206         "\u2163": "IV",
207         "\u2162": "III",
208         "\u2161": "II",
209         "\u2160": "I",
210     }.items():
211         result = result.str.replace(old, new, regex=False)
212     result = result.str.replace(r"^\[|\]$ ", "", regex=True)
213     result = result.str.replace(r"(?:憋尿彩超|心脏彩超|彩超|B超)$", "", regex=True)
214     return result.replace("", pd.NA)
215
216
217
218 def classify_projects(series: pd.Series) -> pd.DataFrame:
219     normalized = normalize_project(series)
220     category = pd.Series("一般其他", index=series.index, dtype="string")
221     unmatched = pd.Series(True, index=series.index)
222     for rule in CATEGORY_RULES[:-1]:
223         hit = normalized.str.contains(rule["pattern"], case=False, regex=True, na=
False) & unmatched
224         category.loc[hit] = rule["category"]
225         unmatched.loc[hit] = False
226     result = pd.DataFrame({"project_norm": normalized, "category": category})
227     result["non_service_item"] = normalized.str.fullmatch(NON_SERVICE_PATTERN, na=
False)
228     for flag, pattern in FLAG_PATTERNS.items():

```

```

229         result[flag] = normalized.str.contains(pattern, regex=True, na=False)
230     result["duration_lower_min"] = result["category"].map(lambda x: DURATION[x][0])
231     result["duration_nominal_min"] = result["category"].map(lambda x: DURATION[x][1])
232     result["duration_upper_min"] = result["category"].map(lambda x: DURATION[x][2])
233     return result
234
235
236 def load_detail(path: Path, source: str) -> pd.DataFrame:
237     frame = pd.read_parquet(path).rename(columns=DETAIL_RENAME)
238     for column in [
239         "patient_id",
240         "order_doctor_id",
241         "department_id",
242         "exam_doctor_id",
243         "machine_id",
244     ]:
245         frame[column] = clean_id(frame[column])
246     frame["order_dt"] = combine_datetime(frame["order_date_raw"], frame["
order_time_raw"])
247     frame["report_dt"] = combine_datetime(frame["report_date_raw"], frame["
report_time_raw"])
248     frame["source"] = source
249     classes = classify_projects(frame["ordered_project"])
250     frame = pd.concat([frame, classes], axis=1)
251     frame["category_rank"] = frame["category"].map(CATEGORY_RANK).fillna(0).astype(int)
252     frame["category_score"] = frame["duration_nominal_min"] * 100 + frame["
category_rank"]
253     return frame
254
255
256 def make_project_catalog(
257     frame: pd.DataFrame,
258     start_date: pd.Timestamp = START_DATE,
259     end_date: pd.Timestamp = END_DATE,
260 ) -> pd.DataFrame:
261     frame = frame[
262         frame["order_dt"].notna()
263         & frame["order_dt"].dt.normalize().between(start_date, end_date)
264     ].copy()
265     columns = [
266         "source",
267         "ordered_project",
268         "project_norm",
269         "category",
270         "床旁标记",
271         "空腹上午标记",
272         "充盈膀胱标记",
273         "non_service_item",
274         "duration_lower_min",
275         "duration_nominal_min",
276         "duration_upper_min",
277     ]
278     return (
279         frame.groupby(columns, dropna=False)
280         .size()
281         .rename("record_count")
282         .reset_index()
283         .sort_values(["source", "record_count"], ascending=[True, False])
284     )
285
286
287 def make_order_episodes(frame: pd.DataFrame) -> pd.DataFrame:
288     valid = frame[

```

```

289         frame["order_dt"].notna()
290         & frame["order_dt"].dt.normalize().between(START_DATE, END_DATE)
291     ].copy()
292     valid["patient_key"] = valid["patient_id"].fillna("缺失_")
293     missing = valid["patient_id"].isna()
294     valid.loc[missing, "patient_key"] = "缺失_" + valid.loc[missing, "source_row"].
astype(str)
295     keys = ["source", "patient_key", "order_dt"]
296     summary = (
297         valid.groupby(keys, dropna=False)
298         .agg(
299             project_row_count=("source_row", "size"),
300             category_count=("category", "nunique"),
301             nonservice_record_count=("non_service_item", "sum"),
302             bedside=("床旁标记", "max"),
303             fasting_before_10=("空腹上午标记", "max"),
304             requires_bladder=("充盈膀胱标记", "max"),
305         )
306         .reset_index()
307     )
308     unique_projects = valid.drop_duplicates(keys + ["project_norm"]).copy()
309     service_counts = (
310         unique_projects[~unique_projects["non_service_item"]]
311         .groupby(keys, dropna=False)
312         .size()
313         .rename("unique_project_count")
314         .reset_index()
315     )
316     nonservice_counts = (
317         unique_projects[unique_projects["non_service_item"]]
318         .groupby(keys, dropna=False)
319         .size()
320         .rename("unique_nonservice_item_count")
321         .reset_index()
322     )
323     workload = (
324         unique_projects.groupby(keys, dropna=False)
325         .agg(
326             workload_lower_min=("duration_lower_min", "sum"),
327             nominal_workload_min=("duration_nominal_min", "sum"),
328             workload_upper_min=("duration_upper_min", "sum"),
329         )
330         .reset_index()
331     )
332     primary = (
333         valid.sort_values("category_score")
334         .drop_duplicates(keys, keep="last")[keys + ["category"]]
335         .rename(columns={"category": "primary_category"})
336     )
337     result = (
338         summary.merge(service_counts, on=keys, how="left")
339         .merge(nonservice_counts, on=keys, how="left")
340         .merge(workload, on=keys, how="left")
341         .merge(primary, on=keys, how="left")
342     )
343     result[["unique_project_count", "unique_nonservice_item_count"]] = result[
344         ["unique_project_count", "unique_nonservice_item_count"]
345     ].fillna(0).astype(int)
346     return result
347
348
349 def make_report_episodes(frame: pd.DataFrame) -> pd.DataFrame:
350     valid = frame[
351         frame["order_dt"].notna()

```

```

352         & frame["report_dt"].notna()
353         & frame["exam_doctor_id"].notna()
354         & frame["machine_id"].notna()
355     ].copy()
356     valid["patient_key"] = valid["patient_id"].fillna("缺失_")
357     missing = valid["patient_id"].isna()
358     valid.loc[missing, "patient_key"] = "缺失_" + valid.loc[missing, "source_row"].
359     astype(str)
360     keys = ["source", "patient_key", "order_dt", "exam_doctor_id", "machine_id"]
361     summary = (
362         valid.groupby(keys, dropna=False)
363         .agg(
364             report_dt=("report_dt", "max"),
365             project_row_count=("source_row", "size"),
366             category_count=("category", "nunique"),
367             nonservice_record_count=("non_service_item", "sum"),
368             bedside=("床旁标记", "max"),
369             fasting_before_10=("空腹上午标记", "max"),
370             requires_bladder=("充盈膀胱标记", "max"),
371         )
372         .reset_index()
373     )
374     unique_projects = valid.drop_duplicates(keys + ["project_norm"]).copy()
375     service_counts = (
376         unique_projects[~unique_projects["non_service_item"]]
377         .groupby(keys, dropna=False)
378         .size()
379         .rename("unique_project_count")
380         .reset_index()
381     )
382     nonservice_counts = (
383         unique_projects[unique_projects["non_service_item"]]
384         .groupby(keys, dropna=False)
385         .size()
386         .rename("unique_nonservice_item_count")
387         .reset_index()
388     )
389     workload = (
390         unique_projects.groupby(keys, dropna=False)
391         .agg(
392             workload_lower_min=("duration_lower_min", "sum"),
393             nominal_workload_min=("duration_nominal_min", "sum"),
394             workload_upper_min=("duration_upper_min", "sum"),
395         )
396         .reset_index()
397     )
398     primary = (
399         valid.sort_values("category_score")
400         .drop_duplicates(keys, keep="last")[keys + ["category"]]
401         .rename(columns={"category": "primary_category"})
402     )
403     result = (
404         summary.merge(service_counts, on=keys, how="left")
405         .merge(nonservice_counts, on=keys, how="left")
406         .merge(workload, on=keys, how="left")
407         .merge(primary, on=keys, how="left")
408     )
409     result[["unique_project_count", "unique_nonservice_item_count"]] = result[
410         ["unique_project_count", "unique_nonservice_item_count"]
411     ].fillna(0).astype(int)
412     return result
413
414 def multiproject_sensitivity(episodes: pd.DataFrame, basis: str) -> pd.DataFrame:

```

```

415 rows: list[dict[str, Any]] = []
416 for (source,), group in episodes.groupby(["source"]):
417     multiple = group["unique_project_count"].gt(1)
418     for coefficient in [1.0, 0.9, 0.8]:
419         adjusted = group["nominal_workload_min"].where(
420             ~multiple,
421             group["nominal_workload_min"] * coefficient,
422         )
423         rows.append(
424             {
425                 "source": source,
426                 "basis": basis,
427                 "multiproject_coefficient": coefficient,
428                 "episode_count": int(len(group)),
429                 "multiproject_episode_count": int(multiple.sum()),
430                 "multiproject_episode_share": float(multiple.mean()),
431                 "total_nominal_workload_min": float(adjusted.sum()),
432                 "mean_nominal_workload_per_episode_min": float(adjusted.mean()),
433             }
434         )
435     return pd.DataFrame(rows)
436
437 def complete_daily(
438     daily: pd.DataFrame,
439     source: str,
440     basis: str,
441 ) -> pd.DataFrame:
442     calendar = pd.DataFrame({"date": pd.date_range(START_DATE, END_DATE, freq="D")})
443     out = calendar.merge(daily, on="date", how="left")
444     out["episode_count"] = out["episode_count"].fillna(0).astype(int)
445     out["project_row_count"] = out["project_row_count"].fillna(0).astype(int)
446     out.insert(0, "basis", basis)
447     out.insert(0, "source", source)
448     return out
449
450
451 def aggregate_trends(daily: pd.DataFrame, frequency: str) -> pd.DataFrame:
452     frame = daily.copy()
453     if frequency == "weekly":
454         frame["period"] = frame["date"].dt.to_period("W-SUN").dt.start_time
455     elif frequency == "monthly":
456         frame["period"] = frame["date"].dt.to_period("M").dt.start_time
457     elif frequency == "annual":
458         frame["period"] = frame["date"].dt.to_period("Y").dt.start_time
459     else:
460         raise ValueError(frequency)
461     result = (
462         frame.groupby(["source", "basis", "period"], as_index=False)
463         .agg(
464             episode_count=("episode_count", "sum"),
465             project_row_count=("project_row_count", "sum"),
466             calendar_days=("date", "size"),
467             mean_daily_episodes=("episode_count", "mean"),
468             median_daily_episodes=("episode_count", "median"),
469             p90_daily_episodes=("episode_count", lambda x: x.quantile(0.9)),
470         )
471     )
472     if frequency == "annual":
473         result["full_calendar_year"] = result["period"].dt.year.between(2020, 2024)
474     return result
475
476
477 def calendar_design(dates: pd.Series, origin: pd.Timestamp) -> np.ndarray:
478

```

```

479 dates = pd.Series(pd.to_datetime(dates)).reset_index(drop=True)
480 n = len(dates)
481 trend = (dates - origin).dt.days.to_numpy(dtype=float) / 365.25
482 day_of_year = dates.dt.dayofyear.to_numpy(dtype=float)
483 columns: list[np.ndarray] = [
484     np.ones(n),
485     trend,
486     np.sin(2 * np.pi * day_of_year / 365.25),
487     np.cos(2 * np.pi * day_of_year / 365.25),
488 ]
489 weekday = dates.dt.dayofweek.to_numpy()
490 month = dates.dt.month.to_numpy()
491 columns.extend((weekday == value).astype(float) for value in range(1, 7))
492 columns.extend((month == value).astype(float) for value in range(2, 13))
493 return np.column_stack(columns)
494
495 def metric_row(actual: np.ndarray, predicted: np.ndarray) -> dict[str, float]:
496     residual = predicted - actual
497     abs_error = np.abs(residual)
498     denominator = np.maximum(np.abs(actual) + np.abs(predicted), 1e-9)
499     return {
500         "MAE": float(np.mean(abs_error)),
501         "RMSE": float(np.sqrt(np.mean(residual**2))),
502         "WAPE": float(abs_error.sum() / max(np.abs(actual).sum(), 1e-9)),
503         "sMAPE": float(np.mean(2 * abs_error / denominator)),
504         "mean_bias": float(np.mean(residual)),
505     }
506
507
508 def chronological_validation(daily: pd.DataFrame) -> pd.DataFrame:
509     results: list[dict[str, Any]] = []
510     series_frames: list[tuple[str, str, pd.DataFrame]] = []
511     for (source, basis), group in daily.groupby(["source", "basis"]):
512         series_frames.append((source, basis, group.sort_values("date").copy()))
513     for basis, group in daily.groupby("basis"):
514         combined = (
515             group.groupby("date", as_index=False)["episode_count"].sum().sort_values("
516             date")
517         )
518         series_frames.append(("三类合计", basis, combined))
519
520     for source, basis, frame in series_frames:
521         frame = frame.sort_values("date").reset_index(drop=True)
522         y = frame["episode_count"].to_numpy(dtype=float)
523         dates = frame["date"]
524         train_mask = dates.lt(VALIDATION_START).to_numpy()
525         test_mask = dates.between(VALIDATION_START, VALIDATION_END).to_numpy()
526         actual = y[test_mask]
527
528         predictions: dict[str, np.ndarray] = {}
529         predictions["7日季节朴素"] = pd.Series(y).shift(7).to_numpy()[test_mask]
530
531         same_weekday_median = np.full(len(frame), np.nan)
532         for index in np.flatnonzero(test_mask):
533             history_index = [index - 7 * lag for lag in range(1, 9) if index - 7 * lag
534             >= 0]
535             same_weekday_median[index] = np.median(y[history_index])
536         predictions["过去8周同星期中位数"] = same_weekday_median[test_mask]
537
538         design_train = calendar_design(dates[train_mask], START_DATE)
539         response_train = np.log1p(y[train_mask])
540         ridge = 1e-4 * np.eye(design_train.shape[1])
541         ridge[0, 0] = 0

```

```

541     coefficient = np.linalg.solve(
542         design_train.T @ design_train + ridge,
543         design_train.T @ response_train,
544     )
545     design_test = calendar_design(dates[test_mask], START_DATE)
546     predictions["日历趋势对数岭回归"] = np.maximum(np.expm1(design_test @
coefficient), 0)

547
548     for model, predicted in predictions.items():
549         finite = np.isfinite(predicted) & np.isfinite(actual)
550         metrics = metric_row(actual[finite], predicted[finite])
551         results.append(
552             {
553                 "source": source,
554                 "basis": basis,
555                 "model": model,
556                 "train_start": str(dates[train_mask].min().date()),
557                 "train_end": str(dates[train_mask].max().date()),
558                 "validation_start": str(dates[test_mask].min().date()),
559                 "validation_end": str(dates[test_mask].max().date()),
560                 "validation_days": int(finite.sum()),
561                 **metrics,
562             }
563         )
564     result = pd.DataFrame(results)
565     result["best_WAPE_within_series"] = result.groupby(["source", "basis"])["WAPE"].
transform("min").eq(result["WAPE"])
566     return result
567
568
569 def peak_flat_summary(daily: pd.DataFrame) -> pd.DataFrame:
570     frames: list[tuple[str, str, pd.DataFrame]] = []
571     for (source, basis), group in daily.groupby(["source", "basis"]):
572         frames.append((source, basis, group.copy()))
573     for basis, group in daily.groupby("basis"):
574         frames.append(
575             (
576                 "三类合计",
577                 basis,
578                 group.groupby("date", as_index=False)["episode_count"].sum(),
579             )
580         )
581     rows: list[dict[str, Any]] = []
582     weekdays = ["周一", "周二", "周三", "周四", "周五", "周六", "周日"]
583     for source, basis, frame in frames:
584         recent = frame[frame["date"].between(VALIDATION_START, VALIDATION_END)].copy()
585         recent["weekday"] = recent["date"].dt.dayofweek
586         recent["month"] = recent["date"].dt.month
587         values = recent["episode_count"]
588         q45, q50, q55, q90, q95 = values.quantile([0.45, 0.5, 0.55, 0.9, 0.95])
589         peak_candidates = recent[recent["episode_count"].ge(q90)].copy()
590         peak_candidates["distance"] = (peak_candidates["episode_count"] - q90).abs()
591         flat_candidates = recent[recent["episode_count"].between(q45, q55, inclusive="
both")].copy()
592         flat_candidates["distance"] = (flat_candidates["episode_count"] - q50).abs()
593         peak_row = peak_candidates.sort_values(["distance", "date"]).iloc[0]
594         flat_row = flat_candidates.sort_values(["distance", "date"]).iloc[0]
595         weekday_median = recent.groupby("weekday")["episode_count"].median()
596         month_median = recent.groupby("month")["episode_count"].median()
597         rows.append(
598             {
599                 "source": source,
600                 "basis": basis,
601                 "period": f"{VALIDATION_START.date()}至{VALIDATION_END.date()}",

```

```

602         "q45": float(q45),
603         "median_flat_level": float(q50),
604         "q55": float(q55),
605         "q90_peak_threshold": float(q90),
606         "q95_stress_threshold": float(q95),
607         "representative_peak_date": str(peak_row["date"].date()),
608         "representative_peak_count": int(peak_row["episode_count"]),
609         "representative_flat_date": str(flat_row["date"].date()),
610         "representative_flat_count": int(flat_row["episode_count"]),
611         "peak_weekday": weekdays[int(weekday_median.idxmax())],
612         "peak_weekday_median": float(weekday_median.max()),
613         "peak_calendar_month": int(month_median.idxmax()),
614         "peak_month_daily_median": float(month_median.max()),
615     }
616 )
617 return pd.DataFrame(rows)
618
619
620 def median_absolute_deviation(series: pd.Series) -> float:
621     values = series.dropna().to_numpy(dtype=float)
622     if len(values) == 0:
623         return math.nan
624     median = np.median(values)
625     return float(np.median(np.abs(values - median)))
626
627
628 def report_gap_proxy(episodes: pd.DataFrame) -> tuple[pd.DataFrame, dict[str, Any], pd
        .DataFrame]:
629     frame = episodes[
630         episodes["report_dt"].dt.normalize().between(START_DATE, END_DATE)
631         & episodes["exam_doctor_id"].notna()
632         & episodes["nominal_workload_min"].gt(0)
633     ].copy()
634     frame["report_date"] = frame["report_dt"].dt.normalize()
635     frame = frame.sort_values(["exam_doctor_id", "report_date", "report_dt"])
636     frame["previous_report_dt"] = frame.groupby(["exam_doctor_id", "report_date"])["
        report_dt"].shift(1)
637     frame["report_gap_min"] = (frame["report_dt"] - frame["previous_report_dt"]).dt.
        total_seconds() / 60
638     hour = frame["report_dt"].dt.hour + frame["report_dt"].dt.minute / 60
639     valid = frame["report_gap_min"].gt(0) & frame["report_gap_min"].le(60) & hour.
        between(6, 22)
640     frame["valid_gap_proxy"] = valid
641     valid_frame = frame[valid].copy()
642     category_proxy_frame = valid_frame[valid_frame["unique_project_count"].eq(1)].copy
        ()
643     grouped = (
644         category_proxy_frame.groupby("primary_category")["report_gap_min"]
645         .agg(
646             proxy_n="size",
647             proxy_median_min="median",
648             proxy_q25_min=lambda x: x.quantile(0.25),
649             proxy_q75_min=lambda x: x.quantile(0.75),
650             proxy_mad_min=median_absolute_deviation,
651         )
652         .reset_index()
653     )
654     overall_median = float(category_proxy_frame["report_gap_min"].median())
655     grouped["relative_burden_index"] = grouped["proxy_median_min"] / overall_median
656     diagnostics = {
657         "all_report_episodes": int(len(frame)),
658         "episodes_with_previous_same_doctor_day": int(frame["report_gap_min"].notna().
        sum()),
659         "zero_or_negative_gap_count": int(frame["report_gap_min"].le(0).sum()),

```

```

660         "gap_over_60_min_count": int(frame["report_gap_min"].gt(60).sum()),
661         "valid_gap_proxy_count": int(valid.sum()),
662         "valid_gap_proxy_share_of_all_episodes": float(valid.mean()),
663         "single_project_valid_gap_proxy_count": int(len(category_proxy_frame)),
664         "single_project_valid_gap_proxy_share": float(len(category_proxy_frame) / max(
valid.sum(), 1)),
665         "overall_single_project_valid_gap_median_min": overall_median,
666         "interpretation": "类别代理仅使用单项目事件；相邻报告间隔仍含扫描、书写、等待和批量出报
告效应，只能作为相对负荷代理，不能解释为服务时长。",
667     }
668     return grouped, diagnostics, frame
669
670
671 def doctor_efficiency(
672     episodes_with_gaps: pd.DataFrame,
673     doctor_day_path: Path,
674 ) -> tuple[pd.DataFrame, dict[str, Any], pd.DataFrame]:
675     frame = episodes_with_gaps.copy()
676     frame["report_date"] = frame["report_dt"].dt.normalize()
677     frame = frame.sort_values(["exam_doctor_id", "report_date", "report_dt"])
678     day = (
679         frame.groupby(["exam_doctor_id", "report_date"], as_index=False)
680         .agg(
681             episode_count=("patient_key", "size"),
682             project_row_count=("project_row_count", "sum"),
683             nominal_workload_min=("nominal_workload_min", "sum"),
684             first_report=("report_dt", "min"),
685             last_report=("report_dt", "max"),
686             first_episode_nominal=("nominal_workload_min", "first"),
687         )
688     )
689     day["report_span_min"] = (day["last_report"] - day["first_report"]).dt.
total_seconds() / 60
690     day["effective_window_min"] = day["report_span_min"] + day["first_episode_nominal"]
691     day["proxy_productivity"] = day["nominal_workload_min"] / day["
effective_window_min"]
692     valid = (
693         day["episode_count"].ge(3)
694         & day["report_span_min"].gt(0)
695         & day["report_span_min"].le(16 * 60)
696         & np.isfinite(day["proxy_productivity"])
697         & day["proxy_productivity"].gt(0)
698     )
699     valid_day = day[valid].copy()
700     raw_log = np.log(valid_day["proxy_productivity"])
701     low, high = raw_log.quantile([0.01, 0.99])
702     valid_day["log_productivity_winsor"] = raw_log.clip(low, high)
703
704     pooled_within_variance = float(
705         valid_day.groupby("exam_doctor_id")["log_productivity_winsor"].var().median()
706     )
707     if not math.isfinite(pooled_within_variance) or pooled_within_variance <= 0:
708         pooled_within_variance = float(valid_day["log_productivity_winsor"].var())
709     doctors = (
710         valid_day.groupby("exam_doctor_id")
711         .agg(
712             valid_days=("report_date", "size"),
713             total_episodes=("episode_count", "sum"),
714             mean_log_productivity=("log_productivity_winsor", "mean"),
715             variance_log_productivity=("log_productivity_winsor", "var"),
716             raw_proxy_productivity=("proxy_productivity", "median"),
717         )
718     ).reset_index()

```

```

719 )
720 doctors["variance_log_productivity"] = doctors["variance_log_productivity"].fillna
(pooled_within_variance)
721 doctors["sampling_variance"] = doctors["variance_log_productivity"] / doctors["
valid_days"]
722 mu0 = float(np.average(doctors["mean_log_productivity"], weights=doctors["
valid_days"]))
723 between_observed = float(doctors["mean_log_productivity"].var(ddof=1))
724 mean_sampling = float(doctors["sampling_variance"].mean())
725 tau2 = max(between_observed - mean_sampling, 1e-6)
726 doctors["shrinkage_weight"] = tau2 / (tau2 + doctors["sampling_variance"])
727 doctors["posterior_log_productivity"] = (
728     doctors["shrinkage_weight"] * doctors["mean_log_productivity"]
729     + (1 - doctors["shrinkage_weight"]) * mu0
730 )
731 doctors["posterior_variance"] = 1 / (1 / tau2 + 1 / doctors["sampling_variance"])
732 doctors["raw_relative_throughput_factor"] = np.exp(doctors["mean_log_productivity"
] - mu0)
733 doctors["shrunk_relative_throughput_factor"] = np.exp(doctors["
posterior_log_productivity"] - mu0)
734 posterior_sd = np.sqrt(doctors["posterior_variance"])
735 doctors["factor_ci95_lower"] = np.exp(doctors["posterior_log_productivity"] - mu0
- 1.96 * posterior_sd)
736 doctors["factor_ci95_upper"] = np.exp(doctors["posterior_log_productivity"] - mu0
+ 1.96 * posterior_sd)
737 doctors = doctors.sort_values("shrunk_relative_throughput_factor", ascending=False
)
738
739 raw_doctor_day = pd.read_parquet(doctor_day_path).rename(
740     columns={
741         "检查人ID": "exam_doctor_id",
742         "检查日期": "report_date",
743         "部位数": "listed_project_count",
744         "最早检查时间": "listed_first_report",
745         "最晚检查时间": "listed_last_report",
746     }
747 )
748 raw_doctor_day["exam_doctor_id"] = clean_id(raw_doctor_day["exam_doctor_id"])
749 raw_doctor_day["report_date"] = pd.to_datetime(raw_doctor_day["report_date"],
errors="coerce", format="mixed").dt.normalize()
750 raw_doctor_day["listed_project_count"] = pd.to_numeric(raw_doctor_day["
listed_project_count"], errors="coerce")
751 raw_doctor_day["listed_first_report"] = pd.to_datetime(raw_doctor_day["
listed_first_report"], errors="coerce", format="mixed")
752 raw_doctor_day["listed_last_report"] = pd.to_datetime(raw_doctor_day["
listed_last_report"], errors="coerce", format="mixed")
753 reconciliation = raw_doctor_day.merge(
754     day[
755         [
756             "exam_doctor_id",
757             "report_date",
758             "project_row_count",
759             "episode_count",
760             "first_report",
761             "last_report",
762         ]
763     ],
764     on=["exam_doctor_id", "report_date"],
765     how="left",
766 )
767 matched = reconciliation[
768     reconciliation["listed_project_count"].notna()
769     & reconciliation["project_row_count"].notna()
770 ].copy()

```

```

771 first_abs_minutes = (matched["listed_first_report"] - matched["first_report"]).dt.
total_seconds().abs() / 60
772 last_abs_minutes = (matched["listed_last_report"] - matched["last_report"]).dt.
total_seconds().abs() / 60
773 diagnostics = {
774     "doctor_day_rows": int(len(day)),
775     "valid_doctor_days_for_efficiency": int(len(valid_day)),
776     "doctor_count_estimated": int(len(doctors)),
777     "daily_validity_rule": "至少3个报告事件，报告跨度大于0且不超过16小时；日对数生产率在1%
与99%分位温莎化。",
778     "empirical_bayes_prior_log_mean": mu0,
779     "empirical_bayes_between_doctor_variance": tau2,
780     "table6_matched_doctor_days": int(len(matched)),
781     "table6_project_count_correlation": float(matched["listed_project_count"].corr
(matched["project_row_count"])),
782     "table6_project_count_MAE": float((matched["listed_project_count"] - matched["
project_row_count"]).abs().mean()),
783     "table6_first_time_median_abs_difference_min": float(first_abs_minutes.median
()),
784     "table6_last_time_median_abs_difference_min": float(last_abs_minutes.median())
785 ,
786     "interpretation": "因报告时刻包含非扫查时间，因子仅表示历史报告吞吐代理；当前医生名单未
给出时不能直接映射为未来个人速度。",
787 }
788 return doctors, diagnostics, reconciliation
789
790 def duration_table(gap_proxy: pd.DataFrame, category_volume: pd.DataFrame) -> pd.
DataFrame:
791 rows = []
792 for priority, rule in enumerate(CATEGORY_RULES, start=1):
793     rows.append(
794         {
795             "priority": priority,
796             "category": rule["category"],
797             "keyword_regex": rule["pattern"],
798             "classification_basis": rule["basis"],
799             "duration_lower_min": rule["lower"],
800             "duration_nominal_min": rule["nominal"],
801             "duration_upper_min": rule["upper"],
802             "robust_schedule_parameter_min": rule["upper"],
803             "special_case_frequency_from_problem": 0.05,
804             "special_case_extra_duration_known": False,
805             "multiproject_default_coefficient": 1.0,
806             "multiproject_sensitivity_lower": 0.8,
807             "multiproject_sensitivity_upper": 1.0,
808         }
809     )
810 result = pd.DataFrame(rows)
811 total_volume = category_volume.groupby("category")["record_count"].sum().rename("
all_source_record_count")
812 result = result.merge(total_volume, on="category", how="left")
813 result = result.merge(gap_proxy, left_on="category", right_on="primary_category",
how="left").drop(columns="primary_category")
814 return result
815
816
817 def finite_json(value: Any) -> Any:
818     if isinstance(value, dict):
819         return {str(key): finite_json(item) for key, item in value.items()}
820     if isinstance(value, list):
821         return [finite_json(item) for item in value]
822     if isinstance(value, (np.integer,)):
823         return int(value)

```

```

824     if isinstance(value, (np.floating, float)):
825         return float(value) if math.isfinite(float(value)) else None
826     if isinstance(value, (pd.Timestamp,)):
827         return value.isoformat()
828     return value
829
830
831 def set_plot_style() -> None:
832     plt.rcParams.update(
833         {
834             "font.sans-serif": ["SimHei", "Microsoft YaHei", "Arial Unicode MS", "
DejaVu Sans"],
835             "axes.unicode_minus": False,
836             "font.size": 10,
837             "axes.spines.top": False,
838             "axes.spines.right": False,
839             "figure.dpi": 150,
840             "savefig.dpi": 220,
841         }
842     )
843
844
845 def make_figures(
846     daily: pd.DataFrame,
847     monthly: pd.DataFrame,
848     peak_flat: pd.DataFrame,
849     durations: pd.DataFrame,
850     doctors: pd.DataFrame,
851     figures_dir: Path,
852 ) -> None:
853     set_plot_style()
854     colors = {"住院": "#1f77b4", "门诊": "#d95f02", "体检": "#2a9d8f"}
855
856     fig, ax = plt.subplots(figsize=(11, 5.2))
857     order = daily[daily["basis"].eq("开单日")].copy()
858     for source, group in order.groupby("source"):
859         group = group.sort_values("date")
860         rolling = group["episode_count"].rolling(28, center=True, min_periods=14).mean()
861         ax.plot(group["date"], rolling, label=f"{source} (28日中心均值)", color=colors[
source], linewidth=1.4)
862     ax.axvline(VALIDATION_START, color="#555555", linestyle="--", linewidth=1, label="
时间顺序验证起点")
863     ax.set_ylabel("每日开单事件数")
864     ax.set_title("三类患者开单需求长期趋势")
865     ax.legend(ncol=2, frameon=False)
866     ax.grid(axis="y", alpha=0.2)
867     fig.tight_layout()
868     fig.savefig(figures_dir / "daily_order_trend.png", bbox_inches="tight")
869     plt.close(fig)
870
871     fig, axes = plt.subplots(2, 1, figsize=(11, 7.2), sharex=True)
872     for axis, basis in zip(axes, ["开单日", "报告日"]):
873         selected = monthly[monthly["basis"].eq(basis)]
874         for source, group in selected.groupby("source"):
875             axis.plot(group["period"], group["mean_daily_episodes"], label=source,
color=colors[source], linewidth=1.3)
876         axis.set_ylabel("月内日均事件数")
877         axis.set_title(f"{basis}口径")
878         axis.grid(axis="y", alpha=0.2)
879     axes[0].legend(ncol=3, frameon=False)
880     axes[-1].set_xlabel("月份")
881     fig.suptitle("开单需求与报告吞吐的月度趋势（口径分离）", y=0.995)
882     fig.tight_layout()

```

```

883 fig.savefig(figures_dir / "monthly_order_report_trend.png", bbox_inches="tight")
884 plt.close(fig)
885
886 recent_order = daily[
887     daily["basis"].eq("开单日") & daily["date"].between(VALIDATION_START,
888     VALIDATION_END)
889 ].copy()
890 recent_order["weekday"] = recent_order["date"].dt.dayofweek
891 profile = recent_order.groupby(["source", "weekday"])["episode_count"].median().
892 reset_index()
893 fig, ax = plt.subplots(figsize=(9.5, 4.8))
894 labels = ["周一", "周二", "周三", "周四", "周五", "周六", "周日"]
895 for source, group in profile.groupby("source"):
896     ax.plot(group["weekday"], group["episode_count"], marker="o", label=source,
897     color=colors[source])
898 ax.set_xticks(range(7), labels)
899 ax.set_ylabel("日开单事件中位数")
900 ax.set_title("验证期星期负荷剖面")
901 ax.legend(frameon=False)
902 ax.grid(axis="y", alpha=0.2)
903 fig.tight_layout()
904 fig.savefig(figures_dir / "weekday_order_profile.png", bbox_inches="tight")
905 plt.close(fig)
906
907 plot_duration = durations.sort_values("duration_nominal_min")
908 fig, ax = plt.subplots(figsize=(10, 6))
909 y = np.arange(len(plot_duration))
910 error = np.vstack(
911     [
912         plot_duration["duration_nominal_min"] - plot_duration["duration_lower_min"]
913     ],
914     [
915         plot_duration["duration_upper_min"] - plot_duration["duration_nominal_min"]
916     ],
917 )
918 ax.errorbar(
919     plot_duration["duration_nominal_min"],
920     y,
921     xerr=error,
922     fmt="o",
923     color="#1f77b4",
924     ecolor="#7f8c8d",
925     capsize=3,
926     label="题面下界/中点/上界",
927 )
928 proxy_scaled = plot_duration["relative_burden_index"] * 7.5
929 ax.scatter(proxy_scaled, y, marker="x", color="#d95f02", label="报告间隔相对指数×7.5
930 (非时长)")
931 ax.set_yticks(y, plot_duration["category"])
932 ax.set_xlabel("分钟或相对指数缩放值")
933 ax.set_title("调度时长参数与报告间隔代理的边界")
934 ax.legend(frameon=False)
935 ax.grid(axis="x", alpha=0.2)
936 fig.tight_layout()
937 fig.savefig(figures_dir / "duration_and_gap_proxy.png", bbox_inches="tight")
938 plt.close(fig)
939
940 fig, ax = plt.subplots(figsize=(9.5, 4.8))
941 factors = doctors["shrunk_relative_throughput_factor"].sort_values().reset_index(
942 drop=True)
943 ax.plot(np.arange(1, len(factors) + 1), factors, marker="o", markersize=3,
944 linewidth=1, color="#2a9d8f")
945 ax.axhline(1, color="#555555", linestyle="--", linewidth=1)
946 ax.set_xlabel("历史医生 (按收缩因子排序)")

```

```

939 ax.set_ylabel("相对报告吞吐因子")
940 ax.set_title("经验贝叶斯收缩后的医生历史吞吐代理")
941 ax.grid(axis="y", alpha=0.2)
942 fig.tight_layout()
943 fig.savefig(figures_dir / "doctor_throughput_factors.png", bbox_inches="tight")
944 plt.close(fig)
945
946
947 def markdown_table(frame: pd.DataFrame, columns: list[str], limit: int | None = None)
    -> str:
948     selected = frame[columns].copy()
949     if limit is not None:
950         selected = selected.head(limit)
951     header = "|" + "|".join(columns) + "|"
952     divider = "|" + "|".join(["---"] * len(columns)) + "|"
953     rows = []
954     for _, row in selected.iterrows():
955         values = []
956         for value in row:
957             if pd.isna(value):
958                 values.append("不适用")
959             elif isinstance(value, float):
960                 values.append(f"{value:.3f}")
961             else:
962                 values.append(str(value))
963         rows.append("|" + "|".join(values) + "|")
964     return "\n".join([header, divider, *rows])
965
966
967 def write_report(
968     output_path: Path,
969     summary: dict[str, Any],
970     peak_flat: pd.DataFrame,
971     validation: pd.DataFrame,
972     durations: pd.DataFrame,
973     doctors: pd.DataFrame,
974     weekly: pd.DataFrame,
975     annual: pd.DataFrame,
976     multiproject: pd.DataFrame,
977     nonservice: pd.DataFrame,
978 ) -> None:
979     combined_order = peak_flat[
980         peak_flat["source"].eq("三类合计") & peak_flat["basis"].eq("开单日")
981     ].iloc[0]
982     best = validation[validation["best_WAPE_within_series"]].sort_values(["source", "basis"])
983     duration_view = durations[
984         [
985             "category",
986             "duration_lower_min",
987             "duration_nominal_min",
988             "duration_upper_min",
989             "proxy_n",
990             "proxy_median_min",
991             "relative_burden_index",
992         ]
993     ]
994     factor_quantiles = doctors["shrunk_relative_throughput_factor"].quantile([0.05, 0.5, 0.95])
995     full_years = annual[annual["full_calendar_year"]].copy()
996     full_years["year"] = full_years["period"].dt.year
997     annual_view = full_years[
998         ["source", "basis", "year", "episode_count", "mean_daily_episodes"]
999     ]

```

```

1000     weekly_peak = (
1001         weekly.sort_values("mean_daily_episodes", ascending=False)
1002         .groupby(["source", "basis"], as_index=False)
1003         .first()[["source", "basis", "period", "episode_count", "mean_daily_episodes"
1004     ]]
1005     )
1006     weekly_peak["period"] = weekly_peak["period"].dt.date.astype(str)
1007     multi_view = multiproject[
1008         [
1009             "source",
1010             "basis",
1011             "multiproject_coefficient",
1012             "multiproject_episode_share",
1013             "total_nominal_workload_min",
1014             "mean_nominal_workload_per_episode_min",
1015         ]
1016     ]
1017     nonservice_view = nonservice[
1018         ["source", "ordered_project", "record_count", "bedside_record_count"]
1019     ]
1020     text = f"""# P1 数据分析技术报告
1021
1022 ## 1. 结论摘要
1023
1024 本分析覆盖 2019-03-01 至 2025-03-31。三类患者按开单日合计的验证期平峰中位数为 {combined_order
1025     ['median_flat_level']:.0f} 个开单事件/日，90% 高峰阈值为 {combined_order['
1026     q90_peak_threshold']:.0f} 个/日，95% 压力阈值为 {combined_order['
1027     q95_stress_threshold']:.0f} 个/日；代表性平峰日为 {combined_order['
1028     representative_flat_date']} ({combined_order['representative_flat_count']} 个)，代
1029     表性高峰日为 {combined_order['representative_peak_date']} ({combined_order['
1030     representative_peak_count']} 个)。这些阈值来自固定在最后12个月的时间顺序验证窗，不以全样本
1031     最大值定义高峰。
1032
1033 项目分类采用“互斥项目族 + 非互斥资源标记”：项目族决定基础时长与设备能力，床旁、空腹上午、充盈膀胱
1034     三个标记决定地点或时段约束。规则逐条按优先级匹配，项目字典保留原名、规范名、命中类别和记录量，能
1035     够逐项追溯。
1036
1037 独立计费/报告条目不作为检查项目排程。只有规范化文本完整匹配“床旁B超检查加收”“床旁彩色多普勒超声检
1038     查加收”“彩色胶片打印”或“超声计算机图文报告”时才置0分钟；组合文本只要还含真实扫查项目，就按
1039     真实项目分类和计时。床旁加收虽然为0分钟，仍保留床旁资源标记并与同开单事件中的真实项目绑定。正式
1040     期命中数量如下：
1041
1042 {markdown_table(nonservice_view, ['source', 'ordered_project', 'record_count', '
1043     bedside_record_count'])}
1044
1045 题目数据没有检查开始时刻。开单到报告时长包含排队、预约、检查、书写和报告发布，不能当作服务时长。本分
1046     析只把单项目事件中、同一医生同日报告事件的相邻间隔作为类别相对工作量代理，并将其限制在大于0且不
1047     超过60分钟；最终调度分钟参数仍直接取题面时长区间的中点，稳健情景取上界。5%特殊病例频率来自题
1048     面，但题面未给额外时长倍率，因此不人为设置倍率。
1049
1050 历史医生的收缩后相对报告吞吐因子 5%、50%、95% 分位分别为 {factor_quantiles.loc[0.05]:.3f}、{
1051     factor_quantiles.loc[0.5]:.3f}、{factor_quantiles.loc[0.95]:.3f}。该因子已按有效医生
1052     日做经验贝叶斯收缩，但仍是报告吞吐代理，不是临床扫查速度，也不能在缺少当前医生ID映射时直接用于点
1053     名排班。
1054
1055 ## 2. 数据口径
1056
1057 - 开单事件：同一患者、同一精确开单时刻合并为一个事件；同一事件的服务项目按规范名称去重，纯计费/报告
1058     条目不计服务项目数。
1059 - 报告事件：同一患者、同一开单时刻、同一检查医生和同一机器构成一个连续检查事件，完成时刻取该组最后报
1060     告时刻；事件内按规范项目名去重后逐项累加时长。
1061 - 开单日序列描述到达需求；报告日序列描述历史完成吞吐。两者分别输出，未把报告日吞吐误称为到达需求。
1062 - 日序列补齐完整自然日，未出现记录的日期按0处理；周、月、年均由完整日序列汇总。
1063 - 2019年仅从3月开始，2025年仅到3月结束，年度总量横向比较应限于2020--2024完整年份。

```

3. 项目分类与调度时长

```
{markdown_table(duration_view, list(duration_view.columns))}
```

说明：`proxy\_median\_min` 是单项目报告事件间隔的稳健中位数，含等待和书写；`relative\_burden\_index` 仅用于检验类别相对负荷，不替代题面时长。多项目事件默认按规范项目去重后逐项累加题面参数，并在同一兼容检查室连续安排。题面没有给出同次检查的固定节约比例，因此默认合并系数为1.0；0.8和0.9只作为敏感性情景：

```
{markdown_table(multi_view, ['source', 'basis', 'multiproject_coefficient', 'multiproject_episode_share', 'total_nominal_workload_min', 'mean_nominal_workload_per_episode_min'])}
```

4. 高峰和平峰识别

```
{markdown_table(peak_flat, ['source', 'basis', 'median_flat_level', 'q90_peak_threshold', 'q95_stress_threshold', 'representative_flat_date', 'representative_peak_date', 'peak_weekday', 'peak_calendar_month'])}
```

高峰阈值取验证期日事件数90%分位，压力阈值取95%分位；平峰水平取中位数，代表日选取最接近对应阈值的实际日期。月份用月内“每日事件数中位数”排序，避免月份天数不同造成总量偏差。

完整日序列同时按周、月、年汇总。各序列在全观察期内的日均负荷最高周如下；周首日期为周一：

```
{markdown_table(weekly_peak, ['source', 'basis', 'period', 'episode_count', 'mean_daily_episodes'])}
```

完整年份的年度结果如下。2019年从3月开始、2025年到3月结束，因此未混入完整年份比较：

```
{markdown_table(annual_view, ['source', 'basis', 'year', 'episode_count', 'mean_daily_episodes'])}
```

5. 时间顺序验证

训练集固定为 2019-03-01 至 2024-03-31，验证集固定为 2024-04-01 至 2025-03-31，未随机打乱。比较7日季节朴素、滚动一步的过去8周同星期中位数、只使用日历和长期趋势的对数岭回归。滚动一步模型在每个验证日只使用该日以前的真实观测。下表列出每条序列按WAPE最优的基线：

```
{markdown_table(best, ['source', 'basis', 'model', 'MAE', 'RMSE', 'WAPE', 'sMAPE', 'mean_bias'])}
```

此验证用于判断季节/星期结构能否在未来时间段复现，不把其中任何一个基线当成最终排程需求预测器。最终模型应至少以最优基线为对照，并在滚动窗口中重估。

6. 工作量代理与医生收缩估计

报告间隔代理先按“同一医生、同一报告日”排序，仅保留 `[0, 60]` 分钟间隔，并只用单项目事件形成类别统计。医生日生产率定义为去重项目题面名义时长之和除以有效报告窗口；多项目默认系数为1.0。有效窗口为最晚减最早报告时刻，再补首个事件名义时长。至少3个事件、跨度不超过16小时的医生日进入估计，日对数生产率在1%和99%分位温莎化。

对每名医生的平均对数生产率采用正态层次模型的经验贝叶斯收缩：先由医生间方差扣除平均抽样方差估计先验方差，再按各医生的日内波动和有效天数形成数据权重。输出包括原始相对因子、收缩因子和95%区间。该处理降低少量医生日导致的极端值，但不消除报告流程和病例组合造成的混杂。

表6复核结果：匹配医生日 `{summary['doctor_efficiency']['table6_matched_doctor_days']}` 条，明细项目记录数与表6部位数相关系数为 `{summary['doctor_efficiency']['table6_project_count_correlation']:.4f}`，两端报告时刻中位绝对差分别为 `{summary['doctor_efficiency']['table6_first_time_median_abs_difference_min']:.2f}` 和 `{summary['doctor_efficiency']['table6_last_time_median_abs_difference_min']:.2f}` 分钟。该一致性支持“报告事件重建”用于吞吐分析，但不改变其非服务时长属性。

7. 可用于后续模型的参数与边界

```

1082
1083 1. 基础时长使用 `service_duration_parameters.csv` 的中点；同一事件各去重服务项目默认逐项求和并
    连续同室安排，容量压力测试使用逐项上界之和；`non_service_item=True` 的独立条目为0分钟。
1084 2. 多项目合并系数默认1.0，0.8--1.0只用于敏感性分析，不将“同室连续”解释为零增量同时扫描。
1085 3. 5%特殊病例单独占用上界槽位或设情景变量；在没有额外时长证据时不外推倍率。
1086 4. 医生收缩因子只适合历史敏感性分析。若后续获得当前33名医生ID对应和班表，才可逐人应用；否则使用总体
    因子1并以因子分布做情景。
1087 5. 体检历史机器ID几乎不能对应现行诊室时，项目族分析仍可用于总工作量，但不能据此恢复旧房间的精确能
    力。
1088 6. 报告事件分组依赖“患者+精确开单时刻+医生+机器”。同分钟多张申请单可能被合并，故排程应对事件粒度
    另做口径敏感性分析。
1089
1090 ## 8. 复现
1091
1092 在工作区根目录执行：
1093
1094 ```powershell
1095 python supporting_materials/code/analyze_p1.py
1096 ```
1097
1098 随机种子固定为 {SEED}。脚本只读取 `supporting_materials/processed_data/raw_parquet/`，结果
    写入 `supporting_materials/results/p1/`，图件写入 `supporting_materials/figures/p1/`
1099 。
1100
1101 """
1102     output_path.write_text(text, encoding="utf-8")
1103
1104 def main() -> None:
1105     parser = argparse.ArgumentParser()
1106     parser.add_argument(
1107         "--raw-dir",
1108         type=Path,
1109         default=SUPPORTING_DIR / "processed_data" / "raw_parquet",
1110     )
1111     parser.add_argument(
1112         "--output-dir",
1113         type=Path,
1114         default=SUPPORTING_DIR / "results" / "p1",
1115     )
1116     parser.add_argument(
1117         "--figures-dir",
1118         type=Path,
1119         default=SUPPORTING_DIR / "figures" / "p1",
1120     )
1121     args = parser.parse_args()
1122     np.random.seed(SEED)
1123     results_dir = args.output_dir
1124     figures_dir = args.figures_dir
1125     results_dir.mkdir(parents=True, exist_ok=True)
1126     figures_dir.mkdir(parents=True, exist_ok=True)
1127
1128     all_catalogs_train: list[pd.DataFrame] = []
1129     all_catalogs_full: list[pd.DataFrame] = []
1130     all_daily: list[pd.DataFrame] = []
1131     all_report_episodes: list[pd.DataFrame] = []
1132     all_multiproject_sensitivity: list[pd.DataFrame] = []
1133     source_diagnostics: dict[str, Any] = {}
1134
1135     for source, filename in SOURCE_FILES.items():
1136         frame = load_detail(args.raw_dir / filename, source)
1137         official_frame = frame[
1138             frame["order_dt"].notna()
1139             & frame["order_dt"].dt.normalize().between(START_DATE, END_DATE)
1140         ].copy()

```

```

1140     all_catalogs_train.append(
1141         make_project_catalog(frame, START_DATE, VALIDATION_START - pd.Timedelta(
1142             days=1))
1143     )
1144     all_catalogs_full.append(make_project_catalog(frame, START_DATE, END_DATE))
1145     order_episodes = make_order_episodes(frame)
1146     report_episodes = make_report_episodes(frame)
1147     order_episodes = order_episodes[order_episodes["nominal_workload_min"].gt(0)].
1148     copy()
1149     report_episodes = report_episodes[report_episodes["nominal_workload_min"].gt
1150     (0)].copy()
1151     all_report_episodes.append(report_episodes)
1152     all_multiproject_sensitivity.append(multiproject_sensitivity(order_episodes, "
1153     开单事件"))
1154     official_report_episodes = report_episodes[
1155         report_episodes["report_dt"].dt.normalize().between(START_DATE, END_DATE)
1156     ].copy()
1157     all_multiproject_sensitivity.append(multiproject_sensitivity(
1158     official_report_episodes, "报告事件"))
1159
1160     order_daily = (
1161         order_episodes.assign(date=order_episodes["order_dt"].dt.normalize())
1162         .groupby("date", as_index=False)
1163         .agg(episode_count=("patient_key", "size"), project_row_count=("
1164     project_row_count", "sum"))
1165     )
1166     report_daily = (
1167         report_episodes.assign(date=report_episodes["report_dt"].dt.normalize())
1168         .loc[lambdax: x["date"].between(START_DATE, END_DATE)]
1169         .groupby("date", as_index=False)
1170         .agg(episode_count=("patient_key", "size"), project_row_count=("
1171     project_row_count", "sum"))
1172     )
1173     all_daily.append(complete_daily(order_daily, source, "开单日"))
1174     all_daily.append(complete_daily(report_daily, source, "报告日"))
1175     source_diagnostics[source] = {
1176         "detail_rows": int(len(frame)),
1177         "official_detail_rows": int(len(official_frame)),
1178         "official_order_events": int(len(order_episodes)),
1179         "official_report_events": int(len(official_report_episodes)),
1180         "unique_ordered_project_names": int(official_frame["ordered_project"].
1181     nunique(dropna=True)),
1182         "unclassified_general_other_share": float(official_frame["category"].eq("
1183     一般其他").mean()),
1184         "non_service_item_rows": int(official_frame["non_service_item"].sum()),
1185         "non_service_item_share": float(official_frame["non_service_item"].mean())
1186     },
1187     {
1188         "bedside_row_share": float(official_frame["床旁标记"].mean()),
1189         "fasting_constraint_row_share": float(official_frame["空腹上午标记"].mean())
1190     },
1191     {
1192         "bladder_constraint_row_share": float(official_frame["充盈膀胱标记"].mean())
1193     },
1194     }
1195     del frame, official_frame, order_episodes, official_report_episodes
1196
1197     catalog = pd.concat(all_catalogs_train, ignore_index=True)
1198     catalog_full = pd.concat(all_catalogs_full, ignore_index=True)
1199     daily = pd.concat(all_daily, ignore_index=True)
1200     report_episodes_all = pd.concat(all_report_episodes, ignore_index=True)
1201     report_episodes_training = report_episodes_all[
1202         report_episodes_all["report_dt"].dt.normalize().lt(VALIDATION_START)
1203     ].copy()
1204     multiproject = pd.concat(all_multiproject_sensitivity, ignore_index=True)

```

```

1192 category_volume = (
1193     catalog_full.groupby(["source", "category"], as_index=False)["record_count"].
1194     sum()
1195 )
1196 category_volume["share_within_source"] = category_volume["record_count"] /
1197 category_volume.groupby("source")["record_count"].transform("sum")
1198 nonservice = (
1199     catalog_full[catalog_full["non_service_item"]]
1200     .groupby(["source", "ordered_project"], as_index=False)
1201     .agg(
1202         record_count=("record_count", "sum"),
1203         bedside_record_count=("record_count", lambda x: x.sum()),
1204     )
1205 )
1206 nonservice["bedside_record_count"] = np.where(
1207     nonservice["ordered_project"].str.contains("床旁", na=False),
1208     nonservice["record_count"],
1209     0,
1210 )
1211 weekly = aggregate_trends(daily, "weekly")
1212 monthly = aggregate_trends(daily, "monthly")
1213 annual = aggregate_trends(daily, "annual")
1214 validation = chronological_validation(daily)
1215 peak_flat = peak_flat_summary(daily)
1216 # Scheduling-time and doctor-efficiency estimates are model inputs, so they
1217 # must be frozen before the chronological holdout begins. Full-period
1218 # report episodes remain available only for descriptive trend outputs.
1219 gap_proxy, gap_diagnostics, episode_gap_frame = report_gap_proxy(
1220     report_episodes_training)
1221 doctors, doctor_diagnostics, reconciliation = doctor_efficiency(
1222     episode_gap_frame,
1223     args.raw_dir / "doctor_day.parquet",
1224 )
1225 durations = duration_table(gap_proxy, category_volume)
1226
1227 rules = pd.DataFrame(
1228     [
1229         {
1230             "priority": index,
1231             "category": rule["category"],
1232             "keyword_regex": rule["pattern"],
1233             "classification_basis": rule["basis"],
1234             "duration_lower_min": rule["lower"],
1235             "duration_nominal_min": rule["nominal"],
1236             "duration_upper_min": rule["upper"],
1237         }
1238         for index, rule in enumerate(CATEGORY_RULES, start=1)
1239     ]
1240 )
1241 flags = pd.DataFrame(
1242     [
1243         {"flag": flag, "keyword_regex": pattern}
1244         for flag, pattern in FLAG_PATTERNS.items()
1245     ]
1246 )
1247
1248 # The authoritative catalog stops before the chronological holdout. The
1249 # full-period catalog is descriptive only and must not be joined into
1250 # holdout dispatch inputs.
1251 catalog.to_csv(results_dir / "project_category_catalog.csv", index=False, encoding="utf-8-sig")
1252 catalog.to_csv(results_dir / "project_category_catalog_train.csv", index=False, encoding="utf-8-sig")

```

```

1251 catalog_full.to_csv(
1252     results_dir / "project_category_catalog_full_audit.csv",
1253     index=False,
1254     encoding="utf-8-sig",
1255 )
1256 rules.to_csv(results_dir / "category_rules.csv", index=False, encoding="utf-8-sig"
1257 )
1258 flags.to_csv(results_dir / "resource_flag_rules.csv", index=False, encoding="utf
1259 -8-sig")
1260 category_volume.to_csv(results_dir / "category_volume_summary.csv", index=False,
1261 encoding="utf-8-sig")
1262 daily.to_csv(results_dir / "daily_trends.csv", index=False, encoding="utf-8-sig")
1263 weekly.to_csv(results_dir / "weekly_trends.csv", index=False, encoding="utf-8-sig"
1264 )
1265 monthly.to_csv(results_dir / "monthly_trends.csv", index=False, encoding="utf-8-
1266 sig")
1267 annual.to_csv(results_dir / "annual_trends.csv", index=False, encoding="utf-8-sig"
1268 )
1269 validation.to_csv(results_dir / "chronological_validation.csv", index=False,
1270 encoding="utf-8-sig")
1271 peak_flat.to_csv(results_dir / "peak_flat_summary.csv", index=False, encoding="utf
1272 -8-sig")
1273 gap_proxy.to_csv(results_dir / "report_gap_proxy_by_category.csv", index=False,
1274 encoding="utf-8-sig")
1275 durations.to_csv(results_dir / "service_duration_parameters.csv", index=False,
1276 encoding="utf-8-sig")
1277 doctors.to_csv(results_dir / "doctor_efficiency_shrunk.csv", index=False, encoding
1278 ="utf-8-sig")
1279 reconciliation.to_csv(results_dir / "doctor_day_reconciliation.csv", index=False,
1280 encoding="utf-8-sig")
1281 multiproject.to_csv(results_dir / "multiproject_coefficient_sensitivity.csv",
1282 index=False, encoding="utf-8-sig")
1283 nonservice.to_csv(results_dir / "non_service_item_summary.csv", index=False,
1284 encoding="utf-8-sig")
1285
1286 summary = {
1287     "seed": SEED,
1288     "period": {"start": str(START_DATE.date()), "end": str(END_DATE.date())},
1289     "validation_period": {
1290         "start": str(VALIDATION_START.date()),
1291         "end": str(VALIDATION_END.date()),
1292         "split_type": "strict chronological holdout",
1293     },
1294     "catalog_boundary": {
1295         "authoritative_project_catalog_end": str((VALIDATION_START - pd.Timedelta(
1296 days=1)).date()),
1297         "full_period_catalog_output": "project_category_catalog_full_audit.csv",
1298         "holdout_names_used_for_dispatch_join": False,
1299     },
1300     "doctor_time_boundary": {
1301         "authoritative_report_gap_end": str((VALIDATION_START - pd.Timedelta(days
1302 =1)).date()),
1303         "holdout_report_times_used_for_doctor_or_duration_estimation": False,
1304     },
1305     "source_diagnostics": source_diagnostics,
1306     "report_gap_proxy": gap_diagnostics,
1307     "doctor_efficiency": doctor_diagnostics,
1308     "multiproject_sensitivity": multiproject.to_dict(orient="records"),
1309     "non_service_items": nonservice.to_dict(orient="records"),
1310     "modeling_boundary": {
1311         "has_exam_start_timestamp": False,
1312         "turnaround_used_as_service_time": False,
1313         "duration_parameter_basis": "problem-statement intervals; midpoint for
1314 nominal, upper bound for robust scenario",

```

```

1298         "special_case_frequency": 0.05,
1299         "special_case_extra_duration_multiplier": None,
1300     },
1301 }
1302 (results_dir / "analysis_summary.json").write_text(
1303     json.dumps(finite_json(summary), ensure_ascii=False, indent=2),
1304     encoding="utf-8",
1305 )
1306 make_figures(daily, monthly, peak_flat, durations, doctors, figures_dir)
1307 write_report(
1308     results_dir / "P1_DATA_ANALYSIS_REPORT.md",
1309     summary,
1310     peak_flat,
1311     validation,
1312     durations,
1313     doctors,
1314     weekly,
1315     annual,
1316     multiproject,
1317     nonservice,
1318 )
1319
1320 print(json.dumps(finite_json(summary), ensure_ascii=False, indent=2))
1321
1322
1323 if __name__ == "__main__":
1324     main()

```

C 训练期科室需求完整程序

Listing 2 训练期与全期审计科室需求统计

```

1  """Build a reproducible department-level demand summary for Problem 1.
2
3  The source workbooks are not modified.  Department names are unavailable, so
4  all outputs retain the anonymous department identifiers supplied in the data.
5  """
6
7  from __future__ import annotations
8
9  import argparse
10 import json
11 from pathlib import Path
12
13 import pandas as pd
14
15 from analyze_p1 import END_DATE, SOURCE_FILES, START_DATE, VALIDATION_START,
16     load_detail
17
18 def choose_event_department(service_rows: pd.DataFrame) -> tuple[pd.DataFrame, int]:
19     """Assign each application proxy to one department without double counting."""
20     event_keys = ["source", "patient_key", "order_dt"]
21     candidate_counts = service_rows.groupby(event_keys, dropna=False)["department_id"]
22     .nunique()
23     mixed_department_events = int((candidate_counts > 1).sum())
24
25     # Preserve the source ordering and use the first actual service project's
26     # department.  This avoids letting exact duplicate rows vote multiple times.
27     chosen = (
28         service_rows.sort_values(event_keys + ["source_row"])
29         .drop_duplicates(event_keys, keep="first")
30         [event_keys + ["department_id"]]

```

```

30 )
31 return chosen, mixed_department_events
32
33
34 def build_summary(
35     raw_dir: Path, end_date: pd.Timestamp
36 ) -> tuple[pd.DataFrame, pd.DataFrame, dict]:
37     all_departments: list[pd.DataFrame] = []
38     overview_rows: list[dict] = []
39     metadata: dict = {
40         "date_range": [START_DATE.date().isoformat(), end_date.date().isoformat()],
41         "event_proxy": "source + patient_id + exact order datetime",
42         "department_attribution": "department on the first source-ordered service
project row",
43         "sources": {},
44     }
45
46     for source, filename in SOURCE_FILES.items():
47         frame = load_detail(raw_dir / filename, source)
48         frame = frame[
49             frame["order_dt"].notna()
50             & frame["order_dt"].dt.normalize().between(START_DATE, end_date)
51         ].copy()
52         frame["patient_key"] = frame["patient_id"]
53         missing_patient = frame["patient_key"].isna()
54         frame.loc[missing_patient, "patient_key"] = (
55             "missing_row_" + frame.index[missing_patient].astype(str)
56         )
57         frame["department_id"] = frame["department_id"].fillna("缺失")
58
59         event_keys = ["source", "patient_key", "order_dt"]
60         service_rows = frame[~frame["non_service_item"]].copy()
61         chosen, mixed_count = choose_event_department(service_rows)
62
63         unique_projects = service_rows.drop_duplicates(event_keys + ["project_norm"])
64         event_workload = (
65             unique_projects.groupby(event_keys, dropna=False)
66             .agg(
67                 unique_service_projects=("project_norm", "size"),
68                 nominal_workload_min=("duration_nominal_min", "sum"),
69                 upper_workload_min=("duration_upper_min", "sum"),
70             )
71             .reset_index()
72         )
73         events = chosen.merge(event_workload, on=event_keys, how="inner", validate="
one_to_one")
74
75         summary = (
76             events.groupby(["source", "department_id"], dropna=False)
77             .agg(
78                 service_event_count=("order_dt", "size"),
79                 unique_service_projects=("unique_service_projects", "sum"),
80                 nominal_workload_min=("nominal_workload_min", "sum"),
81                 upper_workload_min=("upper_workload_min", "sum"),
82             )
83             .reset_index()
84         )
85         total_events = int(summary["service_event_count"].sum())
86         summary["event_share"] = summary["service_event_count"] / total_events
87         summary["mean_nominal_min_per_event"] = (
88             summary["nominal_workload_min"] / summary["service_event_count"]
89         )
90         summary = summary.sort_values(
91             ["service_event_count", "department_id"], ascending=[False, True]

```

```

92     ).reset_index(drop=True)
93     summary["rank"] = summary.index + 1
94     summary["cumulative_event_share"] = summary["event_share"].cumsum()
95     summary = summary[
96         [
97             "source",
98             "rank",
99             "department_id",
100             "service_event_count",
101             "event_share",
102             "cumulative_event_share",
103             "unique_service_projects",
104             "nominal_workload_min",
105             "upper_workload_min",
106             "mean_nominal_min_per_event",
107         ]
108     ]
109     all_departments.append(summary)
110
111     top = summary.head(12)
112     known_departments = summary[summary["department_id"] != "缺失"]
113     missing_department_events = int(
114         summary.loc[summary["department_id"] == "缺失", "service_event_count"].sum()
115     )
116     overview_rows.append(
117         {
118             "source": source,
119             "service_event_count": total_events,
120             "known_department_count": int(known_departments["department_id"].nunique()),
121             "missing_department_event_count": missing_department_events,
122             "top_6_event_share": float(summary.head(6)["event_share"].sum()),
123             "top_12_event_share": float(top["event_share"].sum()),
124             "top_6_department_ids": ", ".join(summary.head(6)["department_id"].astype(str)),
125             "top_6_event_counts": ", ".join(
126                 summary.head(6)["service_event_count"].astype(str)
127             ),
128             "mixed_department_event_count": mixed_count,
129         }
130     )
131     metadata["sources"][source] = {
132         "official_rows": int(len(frame)),
133         "service_rows": int(len(service_rows)),
134         "service_events": total_events,
135         "known_department_count": int(known_departments["department_id"].nunique()),
136     },
137     {
138         "missing_department_events": missing_department_events,
139         "mixed_department_events": mixed_count,
140     }
141
142     return pd.concat(all_departments, ignore_index=True), pd.DataFrame(overview_rows), metadata
143
144 def main() -> None:
145     parser = argparse.ArgumentParser()
146     root_default = Path(__file__).resolve().parents[2]
147     parser.add_argument("--raw-dir", type=Path, default=root_default / "supporting_materials/processed_data/raw_parquet")
148     parser.add_argument("--output-dir", type=Path, default=root_default / "supporting_materials/results/p1")
149     args = parser.parse_args()

```

```

149     args.output_dir.mkdir(parents=True, exist_ok=True)
150
151     training_end = VALIDATION_START - pd.Timedelta(days=1)
152     summary, overview, metadata = build_summary(args.raw_dir, training_end)
153     full_summary, full_overview, full_metadata = build_summary(args.raw_dir, END_DATE)
154     summary.to_csv(args.output_dir / "department_demand_summary.csv", index=False,
155                   encoding="utf-8-sig")
156     overview.to_csv(args.output_dir / "department_demand_overview.csv", index=False,
157                    encoding="utf-8-sig")
158     summary.to_csv(
159         args.output_dir / "department_demand_summary_train.csv", index=False, encoding
160         ="utf-8-sig"
161     )
162     overview.to_csv(
163         args.output_dir / "department_demand_overview_train.csv", index=False,
164         encoding="utf-8-sig"
165     )
166     full_summary.to_csv(
167         args.output_dir / "department_demand_summary_full_audit.csv",
168         index=False,
169         encoding="utf-8-sig",
170     )
171     full_overview.to_csv(
172         args.output_dir / "department_demand_overview_full_audit.csv",
173         index=False,
174         encoding="utf-8-sig",
175     )
176     metadata["boundary"] = {
177         "authoritative_training_end": training_end.date().isoformat(),
178         "full_audit_end": END_DATE.date().isoformat(),
179         "holdout_used_for_configuration": False,
180     }
181     metadata["full_period_audit"] = full_metadata
182     with (args.output_dir / "department_demand_metadata.json").open("w", encoding="utf
183         -8") as handle:
184         json.dump(metadata, handle, ensure_ascii=False, indent=2)
185
186     print(overview.to_string(index=False))
187
188 if __name__ == "__main__":
189     main()

```

D 设备能力证据审计完整程序

Listing 3 表 1 项目—设备能力证据与独立检查

```

1  """Build and audit the current-room capability matrix from Table 1 only."""
2
3  from __future__ import annotations
4
5  import argparse
6  import json
7  import math
8  import re
9  from pathlib import Path
10 from typing import Any
11
12 import numpy as np
13 import pandas as pd
14
15
16 ROOT = Path(__file__).resolve().parents[2]

```

```

17 SUPPORTING_DIR = ROOT / "supporting_materials"
18 SERVICE_FAMILIES = [
19     "产科III/IV级",
20     "产科I/II级",
21     "介入/定位",
22     "心脏",
23     "血管",
24     "经阴道/腔内",
25     "泌尿/经腹妇科",
26     "腹部/胃肠",
27     "浅表器官",
28     "儿科专项",
29     "一般其他",
30 ]
31
32 SPECIAL_RESOURCES = [
33     ("床旁", None),
34     ("心脏", "心脏"),
35     ("血管", "血管"),
36     ("III/IV产科", "产科III/IV级"),
37     ("I/II产科", "产科I/II级"),
38     ("儿科专项", "儿科专项"),
39     ("介入/定位", "介入/定位"),
40 ]
41
42
43 def clean_text(series: pd.Series) -> pd.Series:
44     return series.astype("string").str.strip()
45
46
47 def clean_id(series: pd.Series) -> pd.Series:
48     return (
49         clean_text(series)
50         .str.replace(r"^[\']+|[\']+$", "", regex=True)
51         .str.replace(r"\.0$", "", regex=True)
52     )
53
54
55 def normalize_project_text(value: str) -> str:
56     text = str(value).strip()
57     for old, new in {
58         "(": "(",
59         ")": ")",
60         ",": ",",
61         " ": " ",
62         "\u2163": "IV",
63         "\u2162": "III",
64         "\u2161": "II",
65         "\u2160": "I",
66     }.items():
67         text = text.replace(old, new)
68     return text.strip("[]")
69
70
71
72 def split_capabilities(value: Any) -> list[str]:
73     if value is None or pd.isna(value):
74         return []
75     text = str(value)
76     items: list[str] = []
77     current: list[str] = []
78     depth = 0
79     for character in text:
80         if character in "([[" :

```

```

81         depth += 1
82         current.append(character)
83     elif character in ") ] ]":
84         depth = max(depth - 1, 0)
85         current.append(character)
86     elif character in "\ ; ;\r\n" and depth == 0:
87         item = "".join(current).strip()
88         if item:
89             items.append(item)
90         current = []
91     else:
92         current.append(character)
93 item = "".join(current).strip()
94 if item:
95     items.append(item)
96 return items
97
98
99 def load_rules(path: Path) -> pd.DataFrame:
100     rules = pd.read_csv(path)
101     rules = rules[rules["category"].isin(SERVICE_FAMILIES)].copy()
102     rules["priority"] = pd.to_numeric(rules["priority"], errors="raise")
103     rules = rules.sort_values("priority")
104     missing = sorted(set(SERVICE_FAMILIES) - set(rules["category"]))
105     if missing:
106         raise ValueError(f"冻结项目族缺失: {missing}")
107     return rules
108
109
110 def classify_token(normalized: str, rules: pd.DataFrame) -> tuple[str, str, str]:
111     for row in rules.itertuples(index=False):
112         if re.search(row.keyword_regex, normalized, flags=re.IGNORECASE):
113             return row.category, row.keyword_regex, row.classification_basis
114     raise RuntimeError(f"项目未被冻结规则覆盖: {normalized}")
115
116
117 def broad_descriptor_status(normalized: str, category: str) -> tuple[bool, str]:
118     if category != "一般其他":
119         if re.search(r"常规|综合", normalized):
120             return (
121                 False,
122                 "含“常规/综合”字样但同时有明确部位或专项关键词，仅按明确项目族映射。",
123             )
124         return False, "表1列出具体部位或专项项目。"
125     broad = bool(
126         re.fullmatch(
127             r"(?:彩色多普勒)?(?:超声|彩超|B超)?(?:常规检查|常规|综合检查|综合超声|检查)",
128             normalized,
129         )
130         or re.search(r"综合超声|全身超声|常规超声$", normalized)
131         or re.fullmatch(r"带有床旁的超声项目", normalized)
132     )
133     if broad:
134         return (
135             True,
136             "宽泛描述只支持“一般其他”中的未细分常规能力；若原文明确床旁则仅另授予床旁地点约束，不外推至心脏、血管、产科、儿科或介入专项。",
137         )
138     return (
139         False,
140         "冻结规则未命中专项关键词，作为一般其他中的具体残余项目；不代表该房间覆盖一般其他的全部项目。",
141     )
142

```

```

143
144 def build_evidence(equipment: pd.DataFrame, rules: pd.DataFrame) -> pd.DataFrame:
145     rows: list[dict[str, Any]] = []
146     for item in equipment.itertuples(index=False):
147         for evidence in split_capabilities(item.capability_text):
148             normalized = normalize_project_text(evidence)
149             category, pattern, basis = classify_token(normalized, rules)
150             broad, ambiguity = broad_descriptor_status(normalized, category)
151             rows.append(
152                 {
153                     "source_row": item.source_row,
154                     "machine_id": item.machine_id,
155                     "current_room": item.current_room,
156                     "machine_model": item.machine_model,
157                     "purchase_date": item.purchase_date,
158                     "category": category,
159                     "evidence_project_original": evidence,
160                     "evidence_project_normalized": normalized,
161                     "matched_frozen_regex": pattern,
162                     "classification_basis": basis,
163                     "is_broad_descriptor": broad,
164                     "qualifies_for_family_binary": not broad,
165                     "ambiguity_note": ambiguity,
166                     "authority_source": "表1现行设备检查项目原文",
167                 }
168             )
169     return pd.DataFrame(rows)
170
171
172 def build_matrix(
173     equipment: pd.DataFrame,
174     evidence: pd.DataFrame,
175 ) -> tuple[pd.DataFrame, pd.DataFrame]:
176     counts = (
177         evidence.groupby(["machine_id", "current_room", "category"], as_index=False)
178         .agg(
179             evidence_count=("evidence_project_original", "size"),
180             broad_evidence_count=("is_broad_descriptor", "sum"),
181             qualifying_evidence_count=("qualifies_for_family_binary", "sum"),
182             evidence_sample=("evidence_project_original", lambda x: "; ".join(x.head
183 (3))),
184         )
185     )
186     full_index = pd.MultiIndex.from_product(
187         [equipment["machine_id"].tolist(), SERVICE_FAMILIES],
188         names=["machine_id", "category"],
189     ).to_frame(index=False)
190     room_lookup = equipment[["machine_id", "current_room"]].drop_duplicates()
191     long = full_index.merge(room_lookup, on="machine_id", how="left")
192     long = long.merge(counts, on=["machine_id", "current_room", "category"], how="left")
193     long["evidence_count"] = long["evidence_count"].fillna(0).astype(int)
194     long["broad_evidence_count"] = long["broad_evidence_count"].fillna(0).astype(int)
195     long["qualifying_evidence_count"] = long["qualifying_evidence_count"].fillna(0).
196     astype(int)
197     long["capability"] = long["qualifying_evidence_count"].gt(0).astype(int)
198     long["all_evidence_broad"] = (
199         long["evidence_count"].gt(0)
200         & long["qualifying_evidence_count"].eq(0)
201     )
202     long["matrix_semantics"] = np.select(
203         [long["capability"].eq(1), long["all_evidence_broad"]],
204         [
205             "表1至少列出一个属于该族的具体项目；不等于族内全部项目均可做",

```

```

204         "表1只有宽泛描述，严格族矩阵置0；独立地点能力另行保留",
205     ],
206     default="表1未列出该族项目；不能用历史完成记录补为1",
207 )
208 long = long[
209     [
210         "machine_id",
211         "current_room",
212         "category",
213         "capability",
214         "evidence_count",
215         "broad_evidence_count",
216         "qualifying_evidence_count",
217         "all_evidence_broad",
218         "evidence_sample",
219         "matrix_semantics",
220     ]
221 ]
222
223 matrix = (
224     long.pivot(index=["machine_id", "current_room"], columns="category", values="
capability")
225     .reindex(columns=SERVICE_FAMILIES)
226     .reset_index()
227 )
228 matrix.columns.name = None
229 order = {machine: index for index, machine in enumerate(equipment["machine_id"])}
230 matrix["_order"] = matrix["machine_id"].map(order)
231 matrix = matrix.sort_values("_order").drop(columns="_order")
232 return matrix, long
233
234
235 def special_resource_check(
236     equipment: pd.DataFrame,
237     matrix: pd.DataFrame,
238     evidence: pd.DataFrame,
239 ) -> pd.DataFrame:
240     rows: list[dict[str, Any]] = []
241     for resource, category in SPECIAL_RESOURCES:
242         if resource == "床旁":
243             token_hit = evidence["evidence_project_original"].str.contains("床旁", na=
False)
244             room_hit = equipment["current_room"].str.contains("床边|床旁", na=False)
245             feasible = sorted(
246                 set(evidence.loc[token_hit, "machine_id"])
247                 | set(equipment.loc[room_hit, "machine_id"]),
248                 key=lambda x: equipment.set_index("machine_id").loc[x, "source_row"],
249             )
250             evidence_basis = "表1诊室名含床边/床旁，或检查项目原文含床旁"
251         else:
252             feasible = matrix.loc[matrix[category].eq(1), "machine_id"].tolist()
253             evidence_basis = f"22×11矩阵中“{category}”=1，且每个1有表1项目原文证据"
254             room_map = equipment.set_index("machine_id")["current_room"].to_dict()
255             rooms = [room_map[machine] for machine in feasible]
256             count = len(feasible)
257             if count == 0:
258                 status = "零可行房"
259             elif count == 1:
260                 status = "唯一瓶颈"
261             elif count <= 3:
262                 status = "少数房间"
263             else:
264                 status = "多房间"
265             rows.append(

```

```

266         {
267             "resource": resource,
268             "mapped_category": category or "独立床旁约束",
269             "feasible_room_count": count,
270             "feasible_machine_ids": "|".join(feasible),
271             "feasible_rooms": "|".join(rooms),
272             "zero_feasible": count == 0,
273             "unique_bottleneck": count == 1,
274             "status": status,
275             "evidence_basis": evidence_basis,
276         }
277     )
278     return pd.DataFrame(rows)
279
280
281 def constraint_checks(
282     equipment: pd.DataFrame,
283     matrix: pd.DataFrame,
284     long: pd.DataFrame,
285     evidence: pd.DataFrame,
286 ) -> pd.DataFrame:
287     input_sources = "equipment_capability.parquet|category_rules.csv"
288     one_key_count = int(
289         evidence.loc[evidence["qualifies_for_family_binary"]]
290         .drop_duplicates(["machine_id", "category"])
291         .shape[0]
292     )
293     matrix_one_count = int(matrix[SERVICE_FAMILIES].to_numpy().sum())
294     fragment_artifact = evidence["evidence_project_normalized"].str.fullmatch(
295         r"(?:脬|股浅|股深|脛前后|脛前|脛后|腓())?|尺|肱|贵要|附件及周围组织()())?",
296         na=False,
297     )
298     checks = [
299         {
300             "check_id": "C01_machine_rows",
301             "check_description": "表1现行设备行数为22",
302             "passed": len(equipment) == 22,
303             "observed": len(equipment),
304             "expected": 22,
305         },
306         {
307             "check_id": "C02_unique_machine_ids",
308             "check_description": "22个机器ID互不重复",
309             "passed": equipment["machine_id"].nunique() == 22,
310             "observed": equipment["machine_id"].nunique(),
311             "expected": 22,
312         },
313         {
314             "check_id": "C03_matrix_shape",
315             "check_description": "矩阵为22行x11项目族",
316             "passed": matrix.shape == (22, 13),
317             "observed": f"{len(matrix)}x{len(SERVICE_FAMILIES)}",
318             "expected": "22x11",
319         },
320         {
321             "check_id": "C04_binary_values",
322             "check_description": "所有能力值均为0或1",
323             "passed": bool(matrix[SERVICE_FAMILIES].isin([0, 1]).all().all()),
324             "observed": sorted(pd.unique(matrix[SERVICE_FAMILIES].to_numpy().ravel()).
325                 tolist()),
326             "expected": "[0, 1]",
327         },
328         {
329             "check_id": "C05_every_one_has_evidence",

```

```

329         "check_description": "矩阵每个1至少有一条非宽泛表1原文证据",
330         "passed": bool(long.loc[long["capability"].eq(1), "
qualifying_evidence_count"].ge(1).all()),
331         "observed": int(long.loc[long["capability"].eq(1), "
qualifying_evidence_count"].min()),
332         "expected": ">=1",
333     },
334     {
335         "check_id": "C06_no_evidence_for_zero",
336         "check_description": "矩阵每个0均无可授予族能力的具体证据",
337         "passed": bool(long.loc[long["capability"].eq(0), "
qualifying_evidence_count"].eq(0).all()),
338         "observed": int(long.loc[long["capability"].eq(0), "
qualifying_evidence_count"].max()),
339         "expected": 0,
340     },
341     {
342         "check_id": "C07_authority_source",
343         "check_description": "能力证据全部来自表1, 未读取历史完成记录",
344         "passed": evidence["authority_source"].eq("表1现行设备检查项目原文").all(),
345         "observed": input_sources,
346         "expected": "仅表1设备能力+冻结分类规则",
347     },
348     {
349         "check_id": "C08_all_rooms_named",
350         "check_description": "22个现行诊室名称均非空",
351         "passed": equipment["current_room"].notna().all(),
352         "observed": int(equipment["current_room"].notna().sum()),
353         "expected": 22,
354     },
355     {
356         "check_id": "C09_evidence_token_count",
357         "check_description": "拆分后的表1项目证据非空",
358         "passed": len(evidence) > 0,
359         "observed": len(evidence),
360         "expected": ">0",
361     },
362     {
363         "check_id": "C10_one_key_coverage",
364         "check_description": "具有具体证据的机器x项目族键数等于矩阵1的数量",
365         "passed": one_key_count == matrix_one_count,
366         "observed": one_key_count,
367         "expected": matrix_one_count,
368     },
369     {
370         "check_id": "C11_parenthesis_aware_split",
371         "check_description": "括号内血管/器官枚举项未被顿号拆成伪项目",
372         "passed": not fragment_artifact.any(),
373         "observed": int(fragment_artifact.sum()),
374         "expected": 0,
375     },
376 ]
377 return pd.DataFrame(checks)
378
379
380 def safe_json(value: Any) -> Any:
381     if isinstance(value, dict):
382         return {str(key): safe_json(item) for key, item in value.items()}
383     if isinstance(value, list):
384         return [safe_json(item) for item in value]
385     if isinstance(value, (np.integer,)):
386         return int(value)
387     if isinstance(value, (np.bool_,)):
388         return bool(value)

```

```

389     if isinstance(value, (np.floating, float)):
390         return float(value) if math.isfinite(float(value)) else None
391     return value
392
393
394 def markdown_table(frame: pd.DataFrame, columns: list[str]) -> str:
395     selected = frame[columns]
396     output = [
397         "|" + "|".join(columns) + "|",
398         "|" + "|".join(["---"] * len(columns)) + "|",
399     ]
400     for _, row in selected.iterrows():
401         values = []
402         for value in row:
403             if isinstance(value, (list, tuple, set)):
404                 values.append(", ".join(map(str, value)))
405             elif pd.isna(value):
406                 values.append("不适用")
407             elif isinstance(value, (bool, np.bool_)):
408                 values.append("是" if value else "否")
409             else:
410                 values.append(str(value).replace("|", "/"))
411         output.append("|" + "|".join(values) + "|")
412     return "\n".join(output)
413
414
415 def write_report(
416     path: Path,
417     matrix: pd.DataFrame,
418     long: pd.DataFrame,
419     evidence: pd.DataFrame,
420     special: pd.DataFrame,
421     checks: pd.DataFrame,
422 ) -> None:
423     category_counts = (
424         long.groupby("category", as_index=False)
425         .agg(
426             feasible_room_count=("capability", "sum"),
427             evidence_project_count=("evidence_count", "sum"),
428             broad_only_room_count=("all_evidence_broad", "sum"),
429         )
430     )
431     ones = int(matrix[SERVICE_FAMILIES].to_numpy().sum())
432     broad = evidence[evidence["is_broad_descriptor"]]
433     failed = checks[~checks["passed"]]
434     text = f"""## 表1现行设备能力矩阵审计报告
435
436 ## 1. 审计结论
437
438 本审计仅使用表1的22台现行机器及其“检查项目”原文作为能力权威源，冻结的11个服务项目族只承担文本归
439 类。没有读取住院、门诊、体检历史完成记录，也没有用历史机器ID补齐能力。
440
441 最终矩阵为22行×11项目族，共有 {ones} 个能力位为1。每个1均至少对应一条非宽泛的表1原文证据；严格矩阵
442 1的逐条证据见同目录`capability_one_evidence.csv`，完整拆分证据见同目录`
443 capability_evidence.csv`。能力位1的严格语义是“表1至少列出一个属于该项目族的具体项目”，不
444 能解释为该房间能做项目族内全部项目。只有宽泛描述而没有具体项目时，严格族矩阵置0，宽泛证据仍保留
445 在长表。涉及具体排程时，仍应优先用原始项目名与表1项目名匹配；族矩阵适合聚合容量模型和初筛。
446
447 ## 2. 项目族覆盖
448
449 {markdown_table(category_counts, ['category', 'feasible_room_count', '
450 evidence_project_count', 'broad_only_room_count'])}
451
452 `一般其他`是冻结规则的残余类，内部项目异质性最大。它的能力位只能说明房间至少支持表1列出的某个残余项

```

目，不支持把任意新“其他”项目分配给该房间。

3. 宽泛描述处理

表1中含“常规”或“综合”但同时指明腹部、泌尿系等部位的文本，按明确部位映射，不视作宽泛授权。裸“常规超声”“综合超声”“全身超声”等若出现，只映射到`一般其他`，并设置`is\_broad\_descriptor=True`；不外推到心脏、血管、III/IV产科、I/II产科、儿科专项、床旁或介入/定位。表1的“带有床旁的超声项目”同样是宽泛描述：它只确认机器7具备床旁地点能力，不据此赋予任何具体专项能力。

本次拆分证据中宽泛描述条目数为 `{len(broad)}`；详细内容见同目录`broad\_descriptor\_audit.csv`。宽泛条目的`qualifies\_for\_family\_binary=False`，不会把任何项目族矩阵位改成1。上述规则固定在代码中，防止后续表1版本中宽泛文字造成能力扩张。

4. 关键资源瓶颈

```
{markdown_table(special, ['resource', 'mapped_category', 'feasible_room_count', 'feasible_machine_ids', 'feasible_rooms', 'status'])}
```

`唯一瓶颈`表示按表1只有一个现行机器/诊室满足约束，排程时必须设置不可替代资源约束并考虑停机情景；`零可行房`表示项目族定义与表1发生断裂，不能通过历史完成记录自行补1。`少数房间`仍需要容量压力测试，但不等同于单点故障。

床旁能力是独立地点约束：只按表1诊室名含“床边/床旁”或项目原文含“床旁”识别，不因一般项目族能力自动获得床旁资格。

5. 独立约束检查

```
{markdown_table(checks, ['check_id', 'check_description', 'passed', 'observed', 'expected'])}
```

未通过检查数：`{len(failed)}`。这些检查验证矩阵尺寸、二值性、证据完整性和权威源边界，不把某个专项存在零房或唯一房视为代码错误；专项瓶颈由上一节单独报告。

6. 输出说明

- `capability\_matrix.csv`：22×11二值能力矩阵。
- `capability\_matrix\_long.csv`：每个机器×项目族的能力位、证据数和聚合语义。
- `capability\_evidence.csv`：表1拆分后的全部项目证据，含不授予族能力的宽泛描述。
- `capability\_one\_evidence.csv`：只保留严格矩阵中能力位1的合格表1证据，便于逐位复核。
- `equipment\_source\_rows.csv`：表1的22行原始设备与检查项目文本。
- `special\_resource\_bottlenecks.csv`：七类关键资源的零房/唯一瓶颈检查。
- `independent\_constraint\_checks.csv`：独立约束检查结果。
- `broad\_descriptor\_audit.csv`：宽泛描述及映射边界。
- `capability\_audit\_summary.json`：机器可读摘要。

7. 复现

```
``powershell
python supporting_materials/code/audit_capability.py
``
```

脚本只读取表1 Parquet 和冻结项目族规则，所有输出均写入`supporting\_materials/results/capability/`。

```
"""
    path.write_text(text, encoding="utf-8")

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--equipment",
        type=Path,
        default=SUPPORTING_DIR / "processed_data" / "raw_parquet" / "equipment_capability.parquet",
```

```

497 )
498 parser.add_argument(
499     "--rules",
500     type=Path,
501     default=SUPPORTING_DIR / "results" / "p1" / "category_rules.csv",
502 )
503 parser.add_argument(
504     "--output-dir",
505     type=Path,
506     default=SUPPORTING_DIR / "results" / "capability",
507 )
508 args = parser.parse_args()
509 results_dir = args.output_dir
510 results_dir.mkdir(parents=True, exist_ok=True)
511
512 raw = pd.read_parquet(args.equipment)
513 # The parquet schema is fixed by the extraction manifest; positional rename
514 # avoids shell-codepage issues with Chinese column literals.
515 raw = raw.iloc[:, :6].copy()
516 raw.columns = [
517     "source_row",
518     "machine_id",
519     "current_room",
520     "machine_model",
521     "purchase_date",
522     "capability_text",
523 ]
524 raw["machine_id"] = clean_id(raw["machine_id"])
525 for column in ["current_room", "machine_model", "purchase_date", "capability_text"]
526 ]:
527     raw[column] = clean_text(raw[column])
528
529 rules = load_rules(args.rules)
530 evidence = build_evidence(raw, rules)
531 matrix, long = build_matrix(raw, evidence)
532 one_keys = long.loc[long["capability"].eq(1), ["machine_id", "category"]]
533 one_evidence = evidence.loc[evidence["qualifies_for_family_binary"]].merge(
534     one_keys,
535     on=["machine_id", "category"],
536     how="inner",
537     validate="many_to_one",
538 )
539 special = special_resource_check(raw, matrix, evidence)
540 checks = constraint_checks(raw, matrix, long, evidence)
541 broad = evidence[evidence["is_broad_descriptor"]].copy()
542
543 raw.to_csv(results_dir / "equipment_source_rows.csv", index=False, encoding="utf-8-sig")
544 matrix.to_csv(results_dir / "capability_matrix.csv", index=False, encoding="utf-8-sig")
545 long.to_csv(results_dir / "capability_matrix_long.csv", index=False, encoding="utf-8-sig")
546 evidence.to_csv(results_dir / "capability_evidence.csv", index=False, encoding="utf-8-sig")
547 one_evidence.to_csv(results_dir / "capability_one_evidence.csv", index=False, encoding="utf-8-sig")
548 special.to_csv(results_dir / "special_resource_bottlenecks.csv", index=False, encoding="utf-8-sig")
549 checks.to_csv(results_dir / "independent_constraint_checks.csv", index=False, encoding="utf-8-sig")
550 broad.to_csv(results_dir / "broad_descriptor_audit.csv", index=False, encoding="utf-8-sig")
551
552 summary = {

```

```

552     "authority_inputs": [str(args.equipment), str(args.rules)],
553     "historical_completion_records_used": False,
554     "machine_count": int(len(raw)),
555     "service_family_count": len(SERVICE_FAMILIES),
556     "matrix_one_count": int(matrix[SERVICE_FAMILIES].to_numpy().sum()),
557     "evidence_token_count": int(len(evidence)),
558     "matrix_one_evidence_row_count": int(len(one_evidence)),
559     "broad_descriptor_count": int(len(broad)),
560     "all_independent_checks_passed": bool(checks["passed"].all()),
561     "failed_check_ids": checks.loc[~checks["passed"], "check_id"].tolist(),
562     "special_resource_checks": special.to_dict(orient="records"),
563     "matrix_semantics": "1 means Table 1 lists at least one concrete, non-broad
project in the family; it does not guarantee every project in the family.",
564     "broad_descriptor_policy": "Broad-only evidence is retained but produces
family capability 0; an explicit bedside phrase can independently establish only
the bedside location constraint.",
565 }
566 (results_dir / "capability_audit_summary.json").write_text(
567     json.dumps(safe_json(summary), ensure_ascii=False, indent=2),
568     encoding="utf-8",
569 )
570 write_report(
571     results_dir / "CAPABILITY_AUDIT_REPORT.md",
572     matrix,
573     long,
574     evidence,
575     special,
576     checks,
577 )
578 print(json.dumps(safe_json(summary), ensure_ascii=False, indent=2))
579
580
581 if __name__ == "__main__":
582     main()

```

E 医生一项目时长估计完整程序

Listing 4 训练期医生一项目调度时长收缩估计

```

1  """Estimate doctor-project scheduling times from report gaps with shrinkage.
2
3  The source data do not contain scan start/end timestamps. The exported values are
4  therefore scheduling estimates, not measured scan durations.
5  """
6
7  from __future__ import annotations
8
9  import json
10 from pathlib import Path
11
12 import numpy as np
13 import pandas as pd
14
15 import analyze_p1 as p1
16
17
18 ROOT = Path(__file__).resolve().parents[2]
19 RAW = ROOT / "supporting_materials" / "processed_data" / "raw_parquet"
20 P1_RESULTS = ROOT / "supporting_materials" / "results" / "p1"
21 CAPABILITY = ROOT / "supporting_materials" / "results" / "capability"
22 OUT = ROOT / "supporting_materials" / "results" / "doctor_project_time"
23
24

```

```

25 def single_project_episodes() -> pd.DataFrame:
26     all_rows: list[pd.DataFrame] = []
27     for source, filename in p1.SOURCE_FILES.items():
28         frame = p1.load_detail(RAW / filename, source)
29         valid = frame[
30             frame["order_dt"].notna()
31             & frame["report_dt"].notna()
32             & frame["exam_doctor_id"].notna()
33             & frame["machine_id"].notna()
34             & frame["report_dt"].dt.normalize().between(
35                 p1.START_DATE, p1.VALIDATION_START - pd.Timedelta(days=1)
36             )
37         ].copy()
38         valid["patient_key"] = valid["patient_id"].fillna("缺失_")
39         missing = valid["patient_id"].isna()
40         valid.loc[missing, "patient_key"] = "缺失_" + valid.loc[missing, "source_row"
41         ].astype(str)
42         keys = ["source", "patient_key", "order_dt", "exam_doctor_id", "machine_id"]
43         service = valid[~valid["non_service_item"]].drop_duplicates(keys + ["
44         project_norm"])
45         counts = service.groupby(keys).size().rename("unique_service_projects").
46         reset_index()
47         single = service.merge(counts, on=keys, how="left")
48         single = single[single["unique_service_projects"].eq(1)].copy()
49         reports = (
50             valid.groupby(keys, as_index=False)["report_dt"].max()
51             .merge(
52                 single[
53                     keys
54                     + [
55                         "project_norm",
56                         "category",
57                         "duration_lower_min",
58                         "duration_nominal_min",
59                         "duration_upper_min",
60                     ]
61             ).drop_duplicates(keys),
62             on=keys,
63             how="inner",
64             validate="one_to_one",
65         )
66         all_rows.append(reports)
67     episodes = pd.concat(all_rows, ignore_index=True)
68     episodes["report_date"] = episodes["report_dt"].dt.normalize()
69     episodes = episodes.sort_values(["exam_doctor_id", "report_date", "report_dt"])
70     episodes["previous_report_dt"] = episodes.groupby(
71         ["exam_doctor_id", "report_date"]
72     )["report_dt"].shift(1)
73     episodes["report_gap_min"] = (
74         episodes["report_dt"] - episodes["previous_report_dt"]
75     ).dt.total_seconds() / 60
76     hour = episodes["report_dt"].dt.hour + episodes["report_dt"].dt.minute / 60
77     return episodes[
78         episodes["report_gap_min"].gt(0)
79         & episodes["report_gap_min"].le(60)
80         & hour.between(6, 22)
81     ].copy()
82
83 def complete_doctor_project_grid(direct: pd.DataFrame) -> tuple[pd.DataFrame, pd.
84     DataFrame]:
85     doctors = pd.read_csv(P1_RESULTS / "doctor_efficiency_shrunk.csv", encoding="utf
86     -8-sig")

```

```

84 observed_doctors = direct[["exam_doctor_id"]].drop_duplicates()
85 doctors = observed_doctors.merge(doctors, on="exam_doctor_id", how="outer",
86 validate="one_to_one")
87 doctors["has_doctor_factor_evidence"] = doctors["shrunk_relative_throughput_factor"]
88 doctors["valid_days"] = doctors["valid_days"].fillna(0).astype(int)
89 doctors["shrunk_relative_throughput_factor"] = doctors[
90     "shrunk_relative_throughput_factor"
91 ].fillna(1.0)
92 catalog = pd.read_csv(P1_RESULTS / "project_category_catalog.csv", encoding="utf
93 -8-sig")
94 projects = (
95     catalog[~catalog["non_service_item"].astype(str).str.lower().eq("true")]
96     .groupby(
97         [
98             "project_norm",
99             "category",
100             "duration_lower_min",
101             "duration_nominal_min",
102             "duration_upper_min",
103         ],
104         as_index=False,
105     )["record_count"]
106     .sum()
107     .rename(columns={"record_count": "project_record_count"})
108 )
109 project_keys = [
110     "project_norm",
111     "category",
112     "duration_lower_min",
113     "duration_nominal_min",
114     "duration_upper_min",
115 ]
116 direct_projects = direct[project_keys].drop_duplicates().assign(
117     project_record_count=0)
118 projects = (
119     pd.concat([projects, direct_projects], ignore_index=True)
120     .groupby(project_keys, as_index=False)["project_record_count"]
121     .sum()
122 )
123 doctor_columns = [
124     "exam_doctor_id",
125     "valid_days",
126     "shrunk_relative_throughput_factor",
127     "factor_ci95_lower",
128     "factor_ci95_upper",
129     "has_doctor_factor_evidence",
130 ]
131 grid = doctors[doctor_columns].merge(projects, how="cross")
132 return grid, doctors
133
134 def estimate_times(episodes: pd.DataFrame) -> pd.DataFrame:
135     direct = (
136         episodes.groupby(
137             [
138                 "exam_doctor_id",
139                 "project_norm",
140                 "category",
141                 "duration_lower_min",
142                 "duration_nominal_min",
143                 "duration_upper_min",
144             ],
145             as_index=False,

```

```

144     )["report_gap_min"]
145     .agg(
146         proxy_n="size",
147         report_gap_mean_min="mean",
148         report_gap_median_min="median",
149         report_gap_q25_min=lambda values: values.quantile(0.25),
150         report_gap_q75_min=lambda values: values.quantile(0.75),
151     )
152 )
153 grid, _ = complete_doctor_project_grid(direct)
154 grouped = grid.merge(
155     direct,
156     on=[
157         "exam_doctor_id",
158         "project_norm",
159         "category",
160         "duration_lower_min",
161         "duration_nominal_min",
162         "duration_upper_min",
163     ],
164     how="left",
165     validate="one_to_one",
166 )
167 grouped["proxy_n"] = grouped["proxy_n"].fillna(0).astype(int)
168 grouped["doctor_adjusted_nominal_min"] = (
169     grouped["duration_nominal_min"]
170     / grouped["shrunk_relative_throughput_factor"].fillna(1.0)
171 ).clip(grouped["duration_lower_min"], grouped["duration_upper_min"])
172 grouped["gap_proxy_clipped_min"] = grouped["report_gap_median_min"].clip(
173     grouped["duration_lower_min"], grouped["duration_upper_min"]
174 )
175 grouped["gap_weight"] = grouped["proxy_n"] / (grouped["proxy_n"] + 20.0)
176 gap_component = grouped["gap_proxy_clipped_min"].fillna(
177     grouped["doctor_adjusted_nominal_min"]
178 )
179 grouped["estimated_mean_service_min"] = (
180     grouped["gap_weight"] * gap_component
181     + (1 - grouped["gap_weight"]) * grouped["doctor_adjusted_nominal_min"]
182 ).clip(grouped["duration_lower_min"], grouped["duration_upper_min"])
183 grouped["has_direct_pair_evidence"] = grouped["proxy_n"].gt(0)
184 grouped["estimate_basis"] = np.select(
185     [
186         grouped["proxy_n"].ge(20) & grouped["has_doctor_factor_evidence"],
187         grouped["proxy_n"].gt(0) & grouped["has_doctor_factor_evidence"],
188         grouped["proxy_n"].gt(0),
189     ],
190     [
191         "单项目相邻报告间隔与医生吞吐收缩加权",
192         "小样本向医生调整后的题面名义时长收缩",
193         "有直接组合记录但医生因子不足；向题面名义时长收缩",
194     ],
195     default="无直接组合记录；使用可用医生因子调整后的题面名义时长",
196 )
197 grouped["is_measured_scan_duration"] = False
198 return grouped.sort_values(["exam_doctor_id", "category", "proxy_n"], ascending=[
199     True, True, False])
200
201 def category_summary(estimates: pd.DataFrame) -> pd.DataFrame:
202     rows: list[dict[str, object]] = []
203     for category, frame in estimates.groupby("category"):
204         direct = frame[frame["has_direct_pair_evidence"]]
205         weights = direct["proxy_n"].to_numpy(dtype=float)
206         direct_values = direct["estimated_mean_service_min"].to_numpy(dtype=float)

```

```

207     all_values = frame["estimated_mean_service_min"].to_numpy(dtype=float)
208     rows.append(
209         {
210             "category": category,
211             "doctor_project_pairs": int(len(frame)),
212             "direct_evidence_pairs": int(len(direct)),
213             "valid_gap_episodes": int(weights.sum()),
214             "weighted_estimated_mean_min": float(np.average(direct_values, weights
=weights)),
215             "estimate_p10_min": float(np.quantile(direct_values, 0.10)),
216             "estimate_median_min": float(np.median(direct_values)),
217             "estimate_p90_min": float(np.quantile(direct_values, 0.90)),
218             "complete_estimate_p10_min": float(np.quantile(all_values, 0.10)),
219             "complete_estimate_median_min": float(np.median(all_values)),
220             "complete_estimate_p90_min": float(np.quantile(all_values, 0.90)),
221             "minimum_estimate_min": float(all_values.min()),
222             "maximum_estimate_min": float(all_values.max()),
223         }
224     )
225     return pd.DataFrame(rows).sort_values("weighted_estimated_mean_min", ascending=
False)
226
227
228 def machine_profile() -> pd.DataFrame:
229     frame = pd.read_csv(CAPABILITY / "equipment_source_rows.csv", encoding="utf-8-sig"
)
230
231     def parse_purchase(value: object) -> pd.Timestamp:
232         text = str(value).strip()
233         numeric = pd.to_numeric(text, errors="coerce")
234         if pd.isna(numeric) and 30000 <= float(numeric) <= 60000:
235             return pd.Timestamp("1899-12-30") + pd.to_timedelta(float(numeric), unit="
D")
236         return pd.to_datetime(text, errors="coerce")
237
238     frame["purchase_date_parsed"] = frame["purchase_date"].map(parse_purchase)
239     frame["equipment_age_years_2025_03_31"] = (
240         (pd.Timestamp("2025-03-31") - frame["purchase_date_parsed"]).dt.days / 365.25
241     )
242     matrix = pd.read_csv(CAPABILITY / "capability_matrix.csv", encoding="utf-8-sig")
243     family_columns = [column for column in matrix.columns if column not in ["
machine_id", "current_room"]]
244     matrix["compatible_family_count"] = matrix[family_columns].sum(axis=1)
245     return frame.merge(
246         matrix[["machine_id", "compatible_family_count"]],
247         on="machine_id",
248         how="left",
249         validate="one_to_one",
250     )[
251         [
252             "machine_id",
253             "current_room",
254             "machine_model",
255             "purchase_date_parsed",
256             "equipment_age_years_2025_03_31",
257             "compatible_family_count",
258         ]
259     ].sort_values("machine_id")
260
261
262 def main() -> None:
263     OUT.mkdir(parents=True, exist_ok=True)
264     episodes = single_project_episodes()
265     estimates = estimate_times(episodes)

```

```

266 summary = category_summary(estimates)
267 machines = machine_profile()
268 estimates.to_csv(OUT / "doctor_project_time_estimates.csv", index=False, encoding=
"utf-8-sig")
269 estimates[estimates["has_direct_pair_evidence"]].to_csv(
270     OUT / "doctor_project_direct_evidence.csv", index=False, encoding="utf-8-sig"
271 )
272 summary.to_csv(OUT / "category_time_summary.csv", index=False, encoding="utf-8-sig"
")
273 machines.to_csv(OUT / "machine_profile.csv", index=False, encoding="utf-8-sig")
274 (OUT / "summary.json").write_text(
275     json.dumps(
276         {
277             "training_start": str(p1.START_DATE.date()),
278             "training_end": str((p1.VALIDATION_START - pd.Timedelta(days=1)).date
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
)),
        "holdout_report_times_used": False,
        "is_measured_scan_duration": False,
        "valid_single_project_gaps": int(len(episodes)),
        "complete_doctor_project_estimates": int(len(estimates)),
        "direct_evidence_pairs": int(estimates["has_direct_pair_evidence"].sum
()),
        "machine_count": int(len(machines)),
    },
    ensure_ascii=False,
    indent=2,
),
    encoding="utf-8",
)
print(
    f"{len(estimates):,} complete doctor-project estimates, "
    f"{int(estimates['has_direct_pair_evidence'].sum()):,} with direct pair
evidence, "
    f"from {len(episodes):,} valid single-project gaps; {len(machines)} machines",
    flush=True,
)
if __name__ == "__main__":
    main()

```

F 统一患者级输入完整程序

Listing 5 三类患者事件、项目兼容、特殊病例与医生容量构建

```

1  """Build leakage-audited patient-level inputs for the unified scheduler.
2
3  The output has three deliberate boundaries:
4
5  1. Inpatient jobs are released at their order timestamp and have a 48-hour
6  deadline.
7  2. Outpatient and physical-exam jobs use the observed report *date* as a
8  retrospective plan-day scenario and cannot be released before their order
9  time. Historical report time, room and doctor never enter dispatch fields.
10 3. Machine feasibility is recorded per normalized project with an explicit
11   evidence level. A family match is never labelled as an exact project match.
12 """
13
14 from __future__ import annotations
15
16 import argparse
17 import json
18 import math

```

```

19 import re
20 from collections import defaultdict
21 from pathlib import Path
22 from typing import Any, Iterable
23
24 import numpy as np
25 import pandas as pd
26
27 import analyze_p1 as p1
28 import audit_capability as capability
29
30
31 ROOT = Path(__file__).resolve().parents[2]
32 SUPPORTING = ROOT / "supporting_materials"
33 RAW = SUPPORTING / "processed_data" / "raw_parquet"
34 P1_RESULTS = SUPPORTING / "results" / "p1"
35 CAPABILITY_RESULTS = SUPPORTING / "results" / "capability"
36 OUT = SUPPORTING / "processed_data" / "unified_holdout"
37
38 TRAIN_START = pd.Timestamp("2019-03-01")
39 HOLDOUT_START = pd.Timestamp("2024-04-01")
40 HOLDOUT_END_EXCLUSIVE = pd.Timestamp("2025-04-01")
41 # A job can occupy resources for at most 48 hours after release. Therefore
42 # only the two calendar days immediately before the holdout can affect the
43 # state at 2024-04-01; a longer warm-up would add computation without adding a
44 # physically possible carry-over.
45 SIMULATION_START = HOLDOUT_START - pd.Timedelta(hours=48)
46 DAY_START_HOUR = 8
47 DAY_END_HOUR = 17
48 SLOT_MINUTES = 5
49 BLADDER_PREPARATION_MINUTES = 60
50 SEED = 20260824
51
52 SOURCE_CODES = {"住院": "IP", "门诊": "OP", "体检": "PE"}
53
54 EQUIPMENT_RENAME = {
55     "检查超声的机器ID": "machine_id",
56     "彩超诊室名称": "current_room",
57     "机器型号": "machine_model",
58     "设备购置时间": "purchase_date",
59     "检查项目": "capability_text",
60 }
61
62 DOCTOR_DAY_RENAME = {
63     "检查人ID": "doctor_id",
64     "检查日期": "date",
65     "部位数": "project_count",
66     "最早检查时间": "first_report",
67     "最晚检查时间": "last_report",
68 }
69
70 # These descriptors explicitly denote a whole clinically named class in table 1.
71 # They are broader than exact strings, but narrower and better evidenced than a
72 # blanket family matrix.
73 GENERIC_EVIDENCE = {
74     "心脏": ("心脏彩超",),
75     "产科III/IV级": ("III级产科", "IV级产科", "III级产科超声检查", "IV级产科超声检查"),
76     "产科I/II级": ("I级产科", "II级产科", "I级产科超声检查", "II级产科超声检查"),
77     "腹部/胃肠": ("腹部彩超",),
78     "血管": ("血管彩超",),
79 }
80
81
82 def as_bool(series: pd.Series) -> pd.Series:

```

```

83     if pd.api.types.is_bool_dtype(series):
84         return series.fillna(False)
85     return series.astype("string").str.lower().isin(["true", "1", "yes"])
86
87
88 def clean_id(series: pd.Series) -> pd.Series:
89     return p1.clean_id(series).replace("", pd.NA)
90
91
92 def round_up_slot(timestamp: pd.Timestamp) -> pd.Timestamp:
93     if pd.isna(timestamp):
94         return timestamp
95     return timestamp.ceil(f"{SLOT_MINUTES}min")
96
97
98 def release_times(events: pd.DataFrame, bladder_minutes: int) -> pd.Series:
99     """Return the earliest physically available time for each event.
100
101     A background patient whose order already exists can complete preparation
102     before the 08:00 planning window opens. Therefore readiness is
103     max(planning-window start, order time + preparation), not a blanket delay
104     applied after 08:00.
105     """
106     inpatient = events["source"].eq("住院")
107     release = pd.to_datetime(events["order_dt"]).map(round_up_slot)
108     background_start = pd.to_datetime(events["planned_date"]) + pd.Timedelta(hours=
DAY_START_HOUR)
109     release.loc[~inpatient] = np.maximum(
110         release.loc[~inpatient].values.astype("datetime64[ns]"),
111         background_start.loc[~inpatient].values.astype("datetime64[ns]"),
112     )
113     prepared = (pd.to_datetime(events["order_dt"]) + pd.Timedelta(minutes=
bladder_minutes)).map(
114         round_up_slot
115     )
116     bladder = events["bladder"].astype(bool)
117     release.loc[bladder] = np.maximum(
118         release.loc[bladder].values.astype("datetime64[ns]"),
119         prepared.loc[bladder].values.astype("datetime64[ns]"),
120     )
121     return pd.to_datetime(release)
122
123
124 def mode_date(values: pd.Series) -> pd.Timestamp:
125     dates = pd.to_datetime(values, errors="coerce").dropna().dt.normalize()
126     if dates.empty:
127         return pd.NaT
128     counts = dates.value_counts()
129     return pd.Timestamp(counts[counts.eq(counts.max())].index.min())
130
131
132 def unique_join(values: Iterable[Any]) -> str:
133     return "|".join(sorted({str(value) for value in values if pd.notna(value) and str(
value)}))
134
135
136 def load_source(source: str, filename: str) -> pd.DataFrame:
137     frame = p1.load_detail(RAW / filename, source)
138     frame["patient_id"] = clean_id(frame["patient_id"])
139     missing = frame["patient_id"].isna()
140     frame.loc[missing, "patient_id"] = "MISSING_" + frame.loc[missing, "source_row"].
astype(str)
141     frame["report_date"] = frame["report_dt"].dt.normalize()
142

```

```

143     frame = frame.sort_values("source_row", kind="stable")
144
145     duplicate_columns = [
146         "patient_id",
147         "order_dt",
148         "ordered_project",
149         "report_dt",
150         "exam_doctor_id",
151         "machine_id",
152     ]
153     frame["is_exact_duplicate_extra"] = frame.duplicated(duplicate_columns, keep="
first")
154     frame = frame[~frame["is_exact_duplicate_extra"]].copy()
155
156     event_keys = ["source", "patient_id", "order_dt"]
157     event_flags = (
158         frame.groupby(event_keys, dropna=False)
159         .agg(
160             bedside=("床旁标记", "max"),
161             fasting=("空腹上午标记", "max"),
162             bladder=("充盈膀胱标记", "max"),
163             planned_date=("report_date", mode_date),
164         )
165         .reset_index()
166     )
167     frame = frame.merge(event_flags, on=event_keys, how="left", validate="many_to_one"
)
168     frame = frame[~frame["non_service_item"] & frame["project_norm"].notna()].copy()
169     frame = frame.drop_duplicates(event_keys + ["project_norm"], keep="first")
170
171     # Construct the complete application event before applying the simulation
172     # boundary. Filtering detail rows first can truncate a multi-project event
173     # whose reports fall on different dates and can change its modal plan date.
174     if source == "住院":
175         keep = frame["order_dt"].between(
176             SIMULATION_START, HOLDOUT_END_EXCLUSIVE, inclusive="left"
177         )
178     else:
179         keep = frame["planned_date"].between(
180             SIMULATION_START, HOLDOUT_END_EXCLUSIVE, inclusive="left"
181         )
182     frame = frame[keep].copy()
183
184     keys = (
185         frame[event_keys]
186         .drop_duplicates()
187         .sort_values(["order_dt", "patient_id"], kind="stable")
188         .reset_index(drop=True)
189     )
190     code = SOURCE_CODES[source]
191     keys["event_id"] = [f"{code}2024-{index:06d}" for index in range(1, len(keys) + 1)
]
192     frame = frame.merge(keys, on=event_keys, how="left", validate="many_to_one")
193     return frame
194
195
196 def build_patient_tables() -> tuple[pd.DataFrame, pd.DataFrame, dict[str, Any]]:
197     source_frames = [
198         load_source(source, filename) for source, filename in p1.SOURCE_FILES.items()
199     ]
200     items = pd.concat(source_frames, ignore_index=True)
201     items["bedside"] = as_bool(items["bedside"])
202     items["fasting"] = as_bool(items["fasting"])
203     items["bladder"] = as_bool(items["bladder"])

```

```

204 items["item_index"] = items.groupby("event_id").cumcount() + 1
205
206 events = (
207     items.groupby(["event_id", "source", "patient_id", "order_dt"], as_index=False)
208     .agg(
209         planned_date=("planned_date", "first"),
210         bedside=("bedside", "max"),
211         fasting=("fasting", "max"),
212         bladder=("bladder", "max"),
213         service_project_count=("project_norm", "size"),
214         nominal_minutes=("duration_nominal_min", "sum"),
215         upper_minutes=("duration_upper_min", "sum"),
216         project_names=("project_norm", unique_join),
217         categories=("category", unique_join),
218     )
219 )
220
221 inpatient = events["source"].eq("住院")
222 events["mandatory_background"] = ~inpatient
223 events["deadline_dt"] = events["order_dt"] + pd.Timedelta(hours=48)
224 events.loc[~inpatient, "deadline_dt"] = (
225     events.loc[~inpatient, "planned_date"] + pd.Timedelta(hours=DAY_END_HOUR)
226 )
227 events["release_dt"] = release_times(events, BLADDER_PREPARATION_MINUTES)
228 events["deadline_dt"] = pd.to_datetime(events["deadline_dt"])
229
230 events["evaluation_cohort"] = np.where(
231     inpatient,
232     events["order_dt"].between(HOLDOUT_START, HOLDOUT_END_EXCLUSIVE, inclusive="
left"),
233     events["planned_date"].between(HOLDOUT_START, HOLDOUT_END_EXCLUSIVE, inclusive
="left"),
234 )
235 rng = np.random.default_rng(SEED)
236 events["special_case"] = False
237 # Keep the required 5% share independent of the technical warm-up window.
238 # The attachment identifies no individual complex cases, so the selected
239 # event IDs are a fixed scenario sample rather than inferred diagnoses.
240 for cohort_value in [False, True]:
241     eligible = events.index[events["evaluation_cohort"].eq(cohort_value)].to_numpy
242     ()
243     special_count = int(round(0.05 * len(eligible)))
244     if special_count:
245         chosen = rng.choice(eligible, size=special_count, replace=False)
246         events.loc[chosen, "special_case"] = True
247 events["duration_scenario"] = np.where(events["special_case"], "upper", "nominal")
248 event_columns = [
249     "event_id",
250     "source",
251     "patient_id",
252     "order_dt",
253     "planned_date",
254     "release_dt",
255     "deadline_dt",
256     "mandatory_background",
257     "special_case",
258     "duration_scenario",
259     "evaluation_cohort",
260     "bedside",
261     "fasting",
262     "bladder",
263     "service_project_count",
264     "nominal_minutes",

```

```

264     "upper_minutes",
265     "project_names",
266     "categories",
267 ]
268 events = events[event_columns].sort_values(["release_dt", "event_id"], kind="
stable")
269
270 items = items[
271     [
272         "event_id",
273         "item_index",
274         "source_row",
275         "project_norm",
276         "category",
277         "duration_lower_min",
278         "duration_nominal_min",
279         "duration_upper_min",
280         "bedside",
281         "fasting",
282         "bladder",
283     ]
284 ].copy()
285 diagnostics = {
286     "events": {source: int(count) for source, count in events["source"].
value_counts().items()},
287     "items": {source: int(count) for source, count in items.merge(events[["
event_id", "source"]], on="event_id")["source"].value_counts().items()},
288     "special_case_count": int(events["special_case"].sum()),
289     "special_case_share": float(events["special_case"].mean()),
290     "background_scenario_basis": "report date defines the retrospective plan day;
release is no earlier than observed order time; report time, room and doctor are
excluded",
291     "bladder_preparation_minutes": BLADDER_PREPARATION_MINUTES,
292     "bladder_readiness_rule": "max(plan-day start, order time plus preparation)
for background; order time plus preparation for inpatient",
293 }
294 return events, items, diagnostics
295
296
297 def load_equipment() -> pd.DataFrame:
298     frame = pd.read_parquet(RAW / "equipment_capability.parquet").rename(columns=
EQUIPMENT_RENAME)
299     frame["machine_id"] = clean_id(frame["machine_id"])
300     frame["current_room"] = pl.clean_text(frame["current_room"])
301     frame = frame[frame["machine_id"].str.fullmatch(r"\d+", na=False)].copy()
302     if frame["machine_id"].nunique() != 22:
303         raise ValueError("表1现行机器数量不是22台")
304     return frame
305
306
307 def evidence_table(equipment: pd.DataFrame) -> pd.DataFrame:
308     rules = capability.load_rules(P1_RESULTS / "category_rules.csv")
309     rows: list[dict[str, Any]] = []
310     for machine in equipment.itertuples(index=False):
311         for raw_token in capability.split_capabilities(machine.capability_text):
312             token = capability.normalize_project_text(raw_token)
313             category_name, _, _ = capability.classify_token(token, rules)
314             rows.append(
315                 {
316                     "machine_id": str(machine.machine_id),
317                     "current_room": machine.current_room,
318                     "evidence_project": raw_token,
319                     "evidence_norm": token,
320                     "evidence_category": category_name,

```

```

321     }
322     )
323     return pd.DataFrame(rows)
324
325
326 def explicit_generic_match(project: str, category_name: str, evidence: str) -> bool:
327     candidates = GENERIC_EVIDENCE.get(category_name, ())
328     return any(candidate in evidence for candidate in candidates)
329
330
331 def semantic_core(value: str) -> str:
332     """Remove presentation words while retaining anatomy/procedure semantics."""
333     text = capability.normalize_project_text(value)
334     text = re.sub(
335         r"彩色多普勒|浅表器官|超声常规检查|常规检查|超声检查|彩超|B超|"
336         r"临床操作的|主任检查|主任|加收|减免",
337         "",
338         text,
339         flags=re.IGNORECASE,
340     )
341     return re.sub(r"[\(\)\(\),\:\: , \. +\-\_/\s]", "", text)
342
343
344 def project_machine_matches(items: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
345     equipment = load_equipment()
346     evidence = evidence_table(equipment)
347     projects = items[["project_norm", "category"]].drop_duplicates().sort_values("project_norm")
348     matches: list[dict[str, Any]] = []
349
350     for project_row in projects.itertuples(index=False):
351         project = capability.normalize_project_text(project_row.project_norm)
352         category_name = str(project_row.category)
353         category_evidence = evidence[evidence["evidence_category"].eq(category_name)]
354         for row in category_evidence.itertuples(index=False):
355             token = str(row.evidence_norm)
356             rule: str | None = None
357             if project == token:
358                 rule = "exact_project"
359             elif min(len(project), len(token)) >= 6 and (project in token or token in project):
360                 rule = "specific_text_containment"
361             elif min(len(semantic_core(project)), len(semantic_core(token))) >= 4 and
362             (
363                 semantic_core(project) in semantic_core(token)
364                 or semantic_core(token) in semantic_core(project)
365             ):
366                 rule = "semantic_core_match"
367             elif explicit_generic_match(project, category_name, token):
368                 rule = "explicit_generic_descriptor"
369             if rule:
370                 matches.append(
371                     {
372                         "project_norm": project_row.project_norm,
373                         "category": category_name,
374                         "machine_id": str(row.machine_id),
375                         "current_room": row.current_room,
376                         "match_level": "strict",
377                         "match_rule": rule,
378                         "evidence_project": row.evidence_project,
379                     }
380                 )
381
382     # Machine 7 explicitly says it performs bedside ultrasound projects.

```

```

382     matches.append(
383         {
384             "project_norm": project_row.project_norm,
385             "category": category_name,
386             "machine_id": "7",
387             "current_room": equipment.loc[equipment["machine_id"].eq("7"), "
current_room"].iloc[0],
388             "match_level": "bedside_only",
389             "match_rule": "explicit_bedside_descriptor",
390             "evidence_project": "带有床旁的超声项目",
391         }
392     )
393
394     match_frame = pd.DataFrame(matches).drop_duplicates(
395         ["project_norm", "machine_id", "match_level"], keep="first"
396     )
397
398     # A fallback is retained only for projects for which table 1 has no strict
399     # text match. It remains labelled and is tested separately; it is never
400     # described as project-level evidence.
401     strict_counts = (
402         match_frame[match_frame["match_level"].eq("strict")]
403         .groupby("project_norm")["machine_id"]
404         .nunique()
405     )
406     unmatched = projects[~projects["project_norm"].isin(strict_counts.index)]
407     fallback_rows: list[dict[str, Any]] = []
408     for project_row in unmatched.iteruples(index=False):
409         subset = evidence[evidence["evidence_category"].eq(project_row.category)]
410         for row in subset.drop_duplicates("machine_id").iteruples(index=False):
411             fallback_rows.append(
412                 {
413                     "project_norm": project_row.project_norm,
414                     "category": project_row.category,
415                     "machine_id": str(row.machine_id),
416                     "current_room": row.current_room,
417                     "match_level": "category_fallback",
418                     "match_rule": "same_category_without_project_text_match",
419                     "evidence_project": row.evidence_project,
420                 }
421             )
422     if fallback_rows:
423         match_frame = pd.concat([match_frame, pd.DataFrame(fallback_rows)],
ignore_index=True)
424
425     rows: list[dict[str, Any]] = []
426     grouped = match_frame.groupby(["project_norm", "category"], sort=False)
427     for (project, category_name), group in grouped:
428         strict_rooms = sorted(
429             group.loc[group["match_level"].eq("strict"), "machine_id"].unique(), key=
int
430         )
431         fallback_rooms = sorted(
432             group.loc[group["match_level"].eq("category_fallback"), "machine_id"].
unique(), key=int
433         )
434         rows.append(
435             {
436                 "project_norm": project,
437                 "category": category_name,
438                 "strict_rooms": "|".join(strict_rooms),
439                 "fallback_rooms": "|".join(fallback_rooms),
440                 "strict_room_count": len(strict_rooms),
441                 "fallback_room_count": len(fallback_rooms),

```

```

442         "uses_fallback_if_nonbedside": len(strict_rooms) == 0,
443     }
444 )
445 summary = pd.DataFrame(rows)
446 return match_frame.sort_values(["project_norm", "match_level", "machine_id"]),
summary
447
448
449 def build_training_room_scarcity() -> pd.DataFrame:
450     """Estimate room demand pressure from the frozen training catalog only."""
451     catalog = pd.read_csv(
452         P1_RESULTS / "project_category_catalog.csv", encoding="utf-8-sig", low_memory=
False
453     )
454     non_service = catalog["non_service_item"].astype(str).str.lower().eq("true")
455     catalog = catalog[~non_service].copy()
456     _, project_summary = project_machine_matches(catalog[["project_norm", "category"
]])
457     demand = (
458         catalog.groupby(
459             ["project_norm", "category", "duration_nominal_min"], as_index=False
460         )["record_count"]
461         .sum()
462         .merge(
463             project_summary,
464             on=["project_norm", "category"],
465             how="left",
466             validate="many_to_one",
467         )
468     )
469     room_weight: dict[str, float] = defaultdict(float)
470     for row in demand.itertuples(index=False):
471         if "床旁" in str(row.project_norm):
472             rooms = ["7"]
473         else:
474             room_text = row.strict_rooms if int(row.strict_room_count) > 0 else row.
fallback_rooms
475             rooms = [part for part in str(room_text).split("|") if part and part != "
nan"]
476             if not rooms:
477                 continue
478             contribution = float(row.record_count) * float(row.duration_nominal_min) / len
(rooms)
479             for room in rooms:
480                 room_weight[room] += contribution
481     return pd.DataFrame(
482         [
483             {
484                 "room_id": room,
485                 "training_weighted_demand_minutes": weight,
486                 "evidence_period_end": str((HOLDOUT_START - pd.Timedelta(days=1)).date
()),
487             }
488             for room, weight in sorted(room_weight.items(), key=lambda pair: int(pair
[0]))
489         ]
490     )
491
492
493 def slot_clock(slot_index: int) -> tuple[int, int]:
494     if slot_index < 48:
495         minute = 8 * 60 + slot_index * SLOT_MINUTES
496     else:
497         minute = 13 * 60 + (slot_index - 48) * SLOT_MINUTES

```

```

498     return minute // 60, minute % 60
499
500
501 def clock_to_slot(timestamp: pd.Timestamp, side: str) -> int:
502     minute = timestamp.hour * 60 + timestamp.minute + timestamp.second / 60
503     if minute < 12 * 60:
504         raw = (minute - 8 * 60) / SLOT_MINUTES
505         base = 0
506     else:
507         raw = (minute - 13 * 60) / SLOT_MINUTES
508         base = 48
509     value = math.floor(raw) if side == "end" else math.ceil(raw)
510     return int(np.clip(base + value, base, base + 48))
511
512
513 def build_doctor_slot_capacity() -> tuple[pd.DataFrame, pd.DataFrame]:
514     frame = pd.read_parquet(RAW / "doctor_day.parquet").rename(columns=
DOCTOR_DAY_RENAME)
515     frame["doctor_id"] = clean_id(frame["doctor_id"])
516     frame["date"] = pd.to_datetime(frame["date"], errors="coerce", format="mixed").dt.
normalize()
517     frame["first_report"] = pd.to_datetime(frame["first_report"], errors="coerce",
format="mixed")
518     frame["last_report"] = pd.to_datetime(frame["last_report"], errors="coerce",
format="mixed")
519     frame = frame[
520         frame["date"].between(TRAIN_START, HOLDOUT_START, inclusive="left")
521         & frame["doctor_id"].notna()
522         & frame["first_report"].notna()
523         & frame["last_report"].notna()
524     ].copy()
525     frame = (
526         frame.groupby(["date", "doctor_id"], as_index=False)
527         .agg(first_report=("first_report", "min"), last_report=("last_report", "max"))
528     )
529
530     calendar = pd.date_range(TRAIN_START, HOLDOUT_START - pd.Timedelta(days=1), freq="
D")
531     daily_arrays = {date: np.zeros(96, dtype=np.int16) for date in calendar}
532     for row in frame.itertuples(index=False):
533         date = pd.Timestamp(row.date)
534         first_observed = pd.Timestamp(row.first_report)
535         last_observed = pd.Timestamp(row.last_report)
536         first_minute = first_observed.hour * 60 + first_observed.minute
537         last_minute = last_observed.hour * 60 + last_observed.minute
538         if 7 * 60 <= first_minute <= 9 * 60:
539             start = date + pd.Timedelta(hours=8)
540         elif 12 * 60 + 30 <= first_minute <= 14 * 60:
541             start = date + pd.Timedelta(hours=13)
542         else:
543             start = first_observed - pd.Timedelta(minutes=10)
544         if 11 * 60 <= last_minute <= 12 * 60 + 30 and first_minute < 12 * 60:
545             end = date + pd.Timedelta(hours=12)
546         elif 16 * 60 <= last_minute <= 19 * 60:
547             end = date + pd.Timedelta(hours=17)
548         else:
549             end = last_observed
550         for block_start, block_end, offset in [(8, 12, 0), (13, 17, 48)]:
551             block_start_dt = date + pd.Timedelta(hours=block_start)
552             block_end_dt = date + pd.Timedelta(hours=block_end)
553             overlap_start = max(start, block_start_dt)
554             overlap_end = min(end, block_end_dt)
555             if overlap_end <= overlap_start:
556                 continue

```

```

557         first_slot = clock_to_slot(overlap_start, "start")
558         last_slot = clock_to_slot(overlap_end, "end")
559         first_slot = max(first_slot, offset)
560         last_slot = min(last_slot, offset + 48)
561         if last_slot > first_slot:
562             daily_arrays[date][first_slot:last_slot] += 1
563
564     daily_rows: list[dict[str, Any]] = []
565     for date, counts in daily_arrays.items():
566         for slot_index, count in enumerate(counts):
567             hour, minute = slot_clock(slot_index)
568             daily_rows.append(
569                 {
570                     "date": date,
571                     "weekday": date.dayofweek,
572                     "slot_index": slot_index,
573                     "slot_time": f"{hour:02d}:{minute:02d}",
574                     "active_doctor_proxy": int(count),
575                 }
576             )
577     daily = pd.DataFrame(daily_rows)
578     slot = (
579         daily.groupby(["weekday", "slot_index", "slot_time"], as_index=False)
580         .agg(
581             training_days=("date", "nunique"),
582             median_active_doctors=("active_doctor_proxy", "median"),
583             q25_active_doctors=("active_doctor_proxy", lambda values: values.quantile
584 (0.25)),
585             q75_active_doctors=("active_doctor_proxy", lambda values: values.quantile
586 (0.75)),
587             mean_active_doctors=("active_doctor_proxy", "mean"),
588         )
589     )
590     slot["capacity_main"] = np.minimum(
591         np rint(slot["median_active_doctors"]).astype(int), 22
592     )
593     slot["capacity_low"] = np.minimum(
594         np.floor(slot["q25_active_doctors"]).astype(int), 22
595     )
596     return slot, daily
597
598 def attach_room_sets(items: pd.DataFrame, project_summary: pd.DataFrame) -> pd.
599 DataFrame:
600     merged = items.merge(
601         project_summary,
602         on=["project_norm", "category"],
603         how="left",
604         validate="many_to_one",
605     )
606     if merged["strict_room_count"].isna().any():
607         raise ValueError("存在未进入能力匹配的项目")
608     merged["compatible_rooms"] = np.where(
609         merged["bedside"],
610         "7",
611         np.where(
612             merged["strict_room_count"].gt(0),
613             merged["strict_rooms"],
614             merged["fallback_rooms"],
615         ),
616     )
617     merged["capability_level"] = np.where(
618         merged["bedside"],
619         "bedside_explicit",

```

```

618     np.where(merged["strict_room_count"].gt(0), "strict_project", "
category_fallback"),
619 )
620 if merged["compatible_rooms"].fillna("").eq("").any():
621     examples = merged.loc[merged["compatible_rooms"].fillna("").eq(""), "
project_norm"].head().tolist()
622     raise ValueError(f"存在无任何兼容机器的项目: {examples}")
623 return merged
624
625
626 def main() -> None:
627     parser = argparse.ArgumentParser()
628     parser.add_argument("--output-dir", type=Path, default=OUT)
629     args = parser.parse_args()
630     args.output_dir.mkdir(parents=True, exist_ok=True)
631
632     events, items, diagnostics = build_patient_tables()
633     match_rows, project_summary = project_machine_matches(items)
634     items = attach_room_sets(items, project_summary)
635     slot_capacity, daily_capacity = build_doctor_slot_capacity()
636     room_scarcity = build_training_room_scarcity()
637
638     events.to_csv(args.output_dir / "events.csv", index=False, encoding="utf-8-sig")
639     items.to_csv(args.output_dir / "items.csv", index=False, encoding="utf-8-sig")
640     match_rows.to_csv(
641         args.output_dir / "project_machine_evidence.csv", index=False, encoding="utf-
-8-sig"
642     )
643     project_summary.to_csv(
644         args.output_dir / "project_machine_summary.csv", index=False, encoding="utf-8-
sig"
645     )
646     slot_capacity.to_csv(
647         args.output_dir / "doctor_slot_capacity.csv", index=False, encoding="utf-8-sig
"
648     )
649     daily_capacity.to_csv(
650         args.output_dir / "doctor_slot_capacity_training_days.csv",
651         index=False,
652         encoding="utf-8-sig",
653     )
654     room_scarcity.to_csv(
655         args.output_dir / "room_scarcity_training.csv", index=False, encoding="utf-8-
sig"
656     )
657
658     event_source = events[["event_id", "source"]]
659     item_levels = items.merge(event_source, on="event_id", how="left")
660     diagnostics["capability"] = {
661         "unique_projects": int(project_summary["project_norm"].nunique()),
662         "projects_with_strict_match": int(project_summary["strict_room_count"].gt(0).
sum()),
663         "projects_using_category_fallback": int(project_summary["strict_room_count"].
eq(0).sum()),
664         "item_share_strict_or_bedside": float(
665             item_levels["capability_level"].ne("category_fallback").mean()
666         ),
667         "fallback_item_share_by_source": {
668             source: float(group["capability_level"].eq("category_fallback").mean())
669             for source, group in item_levels.groupby("source")
670         },
671     }
672     diagnostics["doctor_capacity"] = {
673         "method": "weekday-by-5-minute median of training-day active session proxy",

```

```

674     "session_reconstruction": "first activity at 07:00-09:00 snaps to 08:00;
12:30-14:00 snaps to 13:00; last activity at 11:00-12:30 snaps to 12:00 and
16:00-19:00 snaps to 17:00; otherwise retain observed interval with a 10-minute
start expansion",
675     "room_cap": 22,
676     "current_doctor_count_from_problem": 33,
677     "main_capacity_min": int(slot_capacity["capacity_main"].min()),
678     "main_capacity_max": int(slot_capacity["capacity_main"].max()),
679 }
680 diagnostics["time_scope"] = {
681     "training_start": str(TRAIN_START.date()),
682     "training_end_exclusive": str(HOLDOUT_START.date()),
683     "holdout_start": str(HOLDOUT_START.date()),
684     "holdout_end_exclusive": str(HOLDOUT_END_EXCLUSIVE.date()),
685     "simulation_start": str(SIMULATION_START.date()),
686     "warmup_days": int((HOLDOUT_START - SIMULATION_START).days),
687 }
688 diagnostics["room_scarcity_boundary"] = {
689     "source": "2019-03-01 to 2024-03-31 frozen training project catalog",
690     "holdout_project_mix_used": False,
691     "measure": "record_count times nominal_minutes divided equally across
compatible rooms",
692 }
693 (args.output_dir / "summary.json").write_text(
694     json.dumps(diagnostics, ensure_ascii=False, indent=2), encoding="utf-8"
695 )
696 print(json.dumps(diagnostics, ensure_ascii=False, indent=2), flush=True)
697
698 if __name__ == "__main__":
699     main()
700

```

G 连续日历联合排程完整程序

Listing 6 共享设备和医生容量下的患者级联合排程

```

1  """Continuous patient-level scheduling of outpatient, physical-exam and inpatient jobs
2  .
3  The annual replay has one calendar state. Outpatient and physical-exam
4  appointment-day demand is placed first because the problem requires it to be
5  met; inpatient policies are compared on the identical residual resources.
6  The JOINT_SCARCITY policy additionally arranges background demand by project
7  scarcity. It is retained as a Pareto alternative, while the authoritative
8  policy is selected by the competition's inpatient 48-hour KPI.
9  """
10
11  from __future__ import annotations
12
13  import argparse
14  import json
15  import math
16  from dataclasses import dataclass
17  from pathlib import Path
18  from typing import Any, Iterable
19
20  import numpy as np
21  import pandas as pd
22
23  import build_unified_inputs as source_config
24
25  ROOT = Path(__file__).resolve().parents[2]
26

```

```

27 SUPPORTING = ROOT / "supporting_materials"
28 INPUT = SUPPORTING / "processed_data" / "unified_holdout"
29 OUT = SUPPORTING / "results" / "unified_schedule"
30
31 SLOT_MINUTES = 5
32 TRANSFER_MINUTES = 10
33 # The holdout closes at 2025-04-01 00:00. Two calendar days are sufficient
34 # to represent the latest admissible 48-hour inpatient deadline (April 2);
35 # no arbitrary extra scheduling horizon is added.
36 FOLLOWUP_DAYS = 2
37
38 POLICIES = {
39     "FCFS_SHARED": {"background": "fcfs", "inpatient": "fcfs"},
40     "SLACK_GUARD_SHARED": {"background": "fcfs", "inpatient": "slack"},
41     "JOINT_SCARCITY": {"background": "scarcity", "inpatient": "slack"},
42 }
43 AUTHORITATIVE_POLICY = "FCFS_SHARED"
44
45
46 def as_bool(series: pd.Series) -> pd.Series:
47     if pd.api.types.is_bool_dtype(series):
48         return series.fillna(False)
49     return series.astype("string").str.lower().isin(["true", "1", "yes"])
50
51
52 def room_tuple(value: Any) -> tuple[str, ...]:
53     if pd.isna(value) or not str(value).strip():
54         return ()
55     return tuple(sorted({part for part in str(value).split("|") if part}, key=int))
56
57
58 @dataclass(frozen=True)
59 class Task:
60     event_id: str
61     item_index: int
62     project_norm: str
63     category: str
64     duration_slots: int
65     compatible_rooms: tuple[str, ...]
66     capability_level: str
67     fasting: bool
68     bladder: bool
69     bedside: bool
70
71
72 @dataclass
73 class PlanStep:
74     task: Task
75     start_index: int
76     room_id: str
77     sequence_position: int
78     transfer_slots_before: int
79
80
81 class WorkCalendar:
82     def __init__(
83         self,
84         start_date: pd.Timestamp,
85         end_date_exclusive: pd.Timestamp,
86         room_ids: list[str],
87         slot_capacity: pd.DataFrame,
88         capacity_column: str,
89         room_scarcity: dict[str, float],
90         room_mode: str,

```

```

91     transfer_minutes: int,
92 ) -> None:
93     dates = pd.date_range(start_date, end_date_exclusive - pd.Timedelta(days=1),
94                             freq="D")
95     timestamps: list[pd.Timestamp] = []
96     slot_in_day: list[int] = []
97     block: list[int] = []
98     for date in dates:
99         for offset in range(48):
100             timestamps.append(date + pd.Timedelta(hours=8, minutes=offset *
101 SLOT_MINUTES))
102             slot_in_day.append(offset)
103             block.append(0)
104         for offset in range(48):
105             timestamps.append(date + pd.Timedelta(hours=13, minutes=offset *
106 SLOT_MINUTES))
107             slot_in_day.append(48 + offset)
108             block.append(1)
109     self.times = pd.DatetimeIndex(timestamps)
110     self.slot_in_day = np.asarray(slot_in_day, dtype=np.int16)
111     self.block = np.asarray(block, dtype=np.int8)
112     self.day_values = self.times.normalize().to_numpy()
113     self.room_ids = room_ids
114     self.room_index = {room: index for index, room in enumerate(room_ids)}
115     self.room_busy = np.zeros((len(room_ids), len(self.times)), dtype=bool)
116     self.doctor_load = np.zeros(len(self.times), dtype=np.int16)
117     self.capacity = self._capacity_array(slot_capacity, capacity_column)
118     self.room_work_slots = {room: 0 for room in room_ids}
119     self.room_scarcity = room_scarcity
120     self.room_mode = room_mode
121     self.transfer_slots = transfer_minutes // SLOT_MINUTES
122
123 def clone(self) -> "WorkCalendar":
124     """Copy mutable allocation state while sharing immutable calendar axes."""
125     other = object.__new__(WorkCalendar)
126     other.times = self.times
127     other.slot_in_day = self.slot_in_day
128     other.block = self.block
129     other.day_values = self.day_values
130     other.room_ids = self.room_ids
131     other.room_index = self.room_index
132     other.room_busy = self.room_busy.copy()
133     other.doctor_load = self.doctor_load.copy()
134     other.capacity = self.capacity
135     other.room_work_slots = self.room_work_slots.copy()
136     other.room_scarcity = self.room_scarcity
137     other.room_mode = self.room_mode
138     other.transfer_slots = self.transfer_slots
139     return other
140
141 def _capacity_array(self, frame: pd.DataFrame, column: str) -> np.ndarray:
142     lookup = {
143         (int(row.weekday), int(row.slot_index)): int(getattr(row, column))
144         for row in frame.itertuples(index=False)
145     }
146     return np.asarray(
147         [lookup[(int(timestamp.dayofweek), int(slot))] for timestamp, slot in zip(
148 self.times, self.slot_in_day)],
149         dtype=np.int16,
150     )
151
152 def search_start(self, timestamp: pd.Timestamp) -> int:
153     return int(self.times.searchsorted(timestamp, side="left"))

```

```

151 def end_timestamp(self, start_index: int, duration_slots: int) -> pd.Timestamp:
152     return self.times[start_index] + pd.Timedelta(minutes=duration_slots *
SLOT_MINUTES)
153
154 def fits_block(self, start_index: int, duration_slots: int) -> bool:
155     if start_index < 0 or start_index + duration_slots > len(self.times):
156         return False
157     end_index = start_index + duration_slots - 1
158     return (
159         self.day_values[start_index] == self.day_values[end_index]
160         and self.block[start_index] == self.block[end_index]
161     )
162
163 def available(self, room_id: str, start_index: int, duration_slots: int) -> bool:
164     if not self.fits_block(start_index, duration_slots):
165         return False
166     room_index = self.room_index[room_id]
167     interval = slice(start_index, start_index + duration_slots)
168     return (
169         not self.room_busy[room_index, interval].any()
170         and np.all(self.doctor_load[interval] < self.capacity[interval])
171     )
172
173 def reserve(self, step: PlanStep) -> None:
174     room_index = self.room_index[step.room_id]
175     interval = slice(step.start_index, step.start_index + step.task.duration_slots
)
176     self.room_busy[room_index, interval] = True
177     self.doctor_load[interval] += 1
178     self.room_work_slots[step.room_id] += step.task.duration_slots
179
180 def release(self, step: PlanStep) -> None:
181     room_index = self.room_index[step.room_id]
182     interval = slice(step.start_index, step.start_index + step.task.duration_slots
)
183     self.room_busy[room_index, interval] = False
184     self.doctor_load[interval] -= 1
185     self.room_work_slots[step.room_id] -= step.task.duration_slots
186
187 def room_choice(self, rooms: list[str], last_room: str | None) -> str:
188     if last_room in rooms:
189         return str(last_room)
190     if self.room_mode == "fcfs":
191         return min(rooms, key=int)
192     return min(
193         rooms,
194         key=lambda room: (
195             self.room_work_slots[room],
196             self.room_scarcity.get(room, 0.0),
197             int(room),
198         ),
199     )
200
201 def find_task(
202     self,
203     task: Task,
204     release: pd.Timestamp,
205     latest_end: pd.Timestamp,
206     last_room: str | None,
207     room_override: str | None = None,
208 ) -> tuple[int, str, int] | None:
209     rooms = [room_override] if room_override is not None else list(task.
compatible_rooms)
210     rooms = [room for room in rooms if room in self.room_index]

```

```

211     if not rooms:
212         return None
213     start_index = self.search_start(release)
214     stop_index = min(
215         int(self.times.searchsorted(latest_end, side="left")),
216         len(self.times) - task.duration_slots + 1,
217     )
218     if stop_index <= start_index:
219         return None
220     indices = np.arange(start_index, stop_index, dtype=np.int64)
221     end_indices = indices + task.duration_slots - 1
222     starts = self.times[indices]
223     ends = starts + pd.Timedelta(minutes=task.duration_slots * SLOT_MINUTES)
224     valid = (
225         (self.day_values[indices] == self.day_values[end_indices])
226         & (self.block[indices] == self.block[end_indices])
227         & (ends <= latest_end)
228     )
229     if task.fasting:
230         valid &= np.asarray(
231             (starts.hour < 10)
232             & (ends <= starts.normalize() + pd.Timedelta(hours=10)),
233             dtype=bool,
234         )
235
236     span_end = stop_index + task.duration_slots - 1
237     doctor_blocked = (
238         self.doctor_load[start_index:span_end] >= self.capacity[start_index:
span_end]
239     ).astype(np.int16)
240     doctor_prefix = np.concatenate([[0], np.cumsum(doctor_blocked)])
241     valid &= (
242         doctor_prefix[task.duration_slots:] - doctor_prefix[:-task.duration_slots]
243     ) == 0
244
245     earliest_by_room: dict[str, int] = {}
246     for room in rooms:
247         room_index = self.room_index[room]
248         room_blocked = self.room_busy[room_index, start_index:span_end].astype(np.
int16)
249         room_prefix = np.concatenate([[0], np.cumsum(room_blocked)])
250         room_free = room_prefix[task.duration_slots:] - room_prefix[:-task.
duration_slots] == 0
251         transfer_slots = 0 if last_room is None or room == last_room else self.
transfer_slots
252         transfer_release = release + pd.Timedelta(minutes=transfer_slots *
SLOT_MINUTES)
253         feasible = valid & room_free & np.asarray(starts >= transfer_release,
dtype=bool)
254         positions = np.flatnonzero(feasible)
255         if positions.size:
256             earliest_by_room[room] = int(indices[positions[0]])
257     if earliest_by_room:
258         earliest = min(earliest_by_room.values())
259         feasible_rooms = [room for room, index in earliest_by_room.items() if
index == earliest]
260         room = self.room_choice(feasible_rooms, last_room)
261         transfer_slots = 0 if last_room is None or room == last_room else self.
transfer_slots
262         return earliest, room, transfer_slots
263     return None
264
265

```

```

266 def load_inputs(
267     special_mode: str,
268     capability_mode: str,
269     bladder_minutes: int,
270 ) -> tuple[pd.DataFrame, dict[str, list[Task]], pd.DataFrame, list[str], dict[str, str
271     ]]:
272     events = pd.read_csv(
273         INPUT / "events.csv",
274         encoding="utf-8-sig",
275         parse_dates=["order_dt", "planned_date", "release_dt", "deadline_dt"],
276     )
277     for column in [
278         "mandatory_background",
279         "special_case",
280         "evaluation_cohort",
281         "bedside",
282         "fasting",
283         "bladder",
284     ]:
285         events[column] = as_bool(events[column])
286     if bladder_minutes != source_config.BLADDER_PREPARATION_MINUTES:
287         events["release_dt"] = source_config.release_times(events, bladder_minutes)
288
289     items = pd.read_csv(INPUT / "items.csv", encoding="utf-8-sig")
290     for column in ["bedside", "fasting", "bladder", "uses_fallback_if_nonbedside"]:
291         items[column] = as_bool(items[column])
292     event_special = events.set_index("event_id")["special_case"].to_dict()
293     tasks: dict[str, list[Task]] = {}
294     for event_id, group in items.groupby("event_id", sort=False):
295         event_tasks: list[Task] = []
296         use_upper = special_mode == "five_percent_upper" and bool(event_special[
297             event_id])
298         for row in group.sort_values("item_index", kind="stable").itertuples(index=
299             False):
300             if capability_mode == "strict" and row.capability_level == "
301                 category_fallback":
302                 rooms: tuple[str, ...] = ()
303             else:
304                 rooms = room_tuple(row.compatible_rooms)
305                 duration = row.duration_upper_min if use_upper else row.
306                 duration_nominal_min
307                 event_tasks.append(
308                     Task(
309                         event_id=str(event_id),
310                         item_index=int(row.item_index),
311                         project_norm=str(row.project_norm),
312                         category=str(row.category),
313                         duration_slots=max(1, int(math.ceil(float(duration) / SLOT_MINUTES
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

```

323     room_names = dict(zip(equipment["machine_id"], equipment["current_room"]))
324     room_ids = sorted(room_names, key=int)
325     slot_capacity = pd.read_csv(INPUT / "doctor_slot_capacity.csv", encoding="utf-8-
326     sig")
327     return events, tasks, slot_capacity, room_ids, room_names
328
329 def task_room_scarcity() -> dict[str, float]:
330     """Read room pressure estimated only from the frozen training catalog."""
331     frame = pd.read_csv(INPUT / "room_scarcity_training.csv", encoding="utf-8-sig")
332     return {
333         str(row.room_id): float(row.training_weighted_demand_minutes)
334         for row in frame.itertuples(index=False)
335     }
336
337 def task_order_candidates(event_tasks: list[Task]) -> list[list[Task]]:
338     keys = [
339         lambda task: (not task.fasting, len(task.compatible_rooms), -task.
340         duration_slots, task.item_index),
341         lambda task: (not task.fasting, -task.duration_slots, len(task.
342         compatible_rooms), task.item_index),
343         lambda task: (len(task.compatible_rooms), not task.fasting, task.item_index),
344         lambda task: (task.item_index,)
345     ]
346     candidates: list[list[Task]] = []
347     seen: set[tuple[int, ...]] = set()
348     for key in keys:
349         ordered = sorted(event_tasks, key=key)
350         signature = tuple(task.item_index for task in ordered)
351         if signature not in seen:
352             candidates.append(ordered)
353             seen.add(signature)
354     return candidates
355
356 def tentative_plan(
357     calendar: WorkCalendar,
358     ordered_tasks: list[Task],
359     release: pd.Timestamp,
360     latest_end: pd.Timestamp,
361     room_override: str | None,
362 ) -> list[PlanStep] | None:
363     steps: list[PlanStep] = []
364     cursor = release
365     last_room: str | None = None
366     for position, task in enumerate(ordered_tasks, start=1):
367         found = calendar.find_task(
368             task,
369             cursor,
370             latest_end,
371             last_room,
372             room_override=room_override,
373         )
374         if found is None:
375             for step in reversed(steps):
376                 calendar.release(step)
377             return None
378         start_index, room_id, transfer_slots = found
379         step = PlanStep(task, start_index, room_id, position, transfer_slots)
380         calendar.reserve(step)
381         steps.append(step)
382         cursor = calendar.end_timestamp(start_index, task.duration_slots)
383         last_room = room_id

```

```

384     return steps
385
386
387 def plan_event(
388     calendar: WorkCalendar,
389     event_tasks: list[Task],
390     release: pd.Timestamp,
391     latest_end: pd.Timestamp,
392 ) -> list[PlanStep] | None:
393     if any(not task.compatible_rooms for task in event_tasks):
394         return None
395     orders = task_order_candidates(event_tasks)
396     common = set(event_tasks[0].compatible_rooms)
397     for task in event_tasks[1:]:
398         common &= set(task.compatible_rooms)
399     if common:
400         candidate_rooms = sorted(
401             common,
402             key=lambda room: (
403                 calendar.room_work_slots[room],
404                 calendar.room_scarcity.get(room, 0.0),
405                 int(room),
406             ),
407         )[:3]
408         for room in candidate_rooms:
409             steps = tentative_plan(calendar, orders[0], release, latest_end, room)
410             if steps is not None:
411                 return steps
412     for ordered in orders:
413         steps = tentative_plan(calendar, ordered, release, latest_end, None)
414         if steps is not None:
415             return steps
416     return None
417
418
419 def event_priority(
420     events: pd.DataFrame,
421     tasks: dict[str, list[Task]],
422     group: str,
423     strategy: str,
424 ) -> list[str]:
425     subset = events[events["mandatory_background"] if group == "background" else ~
426     events["mandatory_background"]].copy()
427     subset["work_slots"] = subset["event_id"].map(
428         lambda event_id: sum(task.duration_slots for task in tasks[event_id])
429     )
430     subset["minimum_room_count"] = subset["event_id"].map(
431         lambda event_id: min((len(task.compatible_rooms) for task in tasks[event_id])),
432         default=0)
433     )
434     subset["has_fasting_task"] = subset["event_id"].map(
435         lambda event_id: any(task.fasting for task in tasks[event_id])
436     )
437     if group == "background" and strategy == "scarcity":
438         subset = subset.sort_values(
439             ["deadline_dt", "has_fasting_task", "minimum_room_count", "work_slots", "
440             release_dt", "event_id"],
441             ascending=[True, False, True, False, True, True],
442             kind="stable",
443         )
444     elif group == "inpatient" and strategy == "slack":
445         subset["slack_key"] = subset["deadline_dt"] - pd.to_timedelta(
446             subset["work_slots"] * SLOT_MINUTES, unit="min"
447         )

```

```

445         subset = subset.sort_values(
446             ["slack_key", "minimum_room_count", "deadline_dt", "event_id"],
447             kind="stable",
448         )
449     else:
450         subset = subset.sort_values(["release_dt", "event_id"], kind="stable")
451     return subset["event_id"].tolist()
452
453
454 def schedule_group(
455     calendar: WorkCalendar,
456     event_ids: list[str],
457     event_lookup: pd.DataFrame,
458     tasks: dict[str, list[Task]],
459 ) -> tuple[dict[str, list[PlanStep]], dict[str, str]]:
460     plans: dict[str, list[PlanStep]] = {}
461     failure: dict[str, str] = {}
462     for count, event_id in enumerate(event_ids, start=1):
463         event = event_lookup.loc[event_id]
464         steps = plan_event(calendar, tasks[event_id], event.release_dt, event.
deadline_dt)
465         if steps is None:
466             if any(not task.compatible_rooms for task in tasks[event_id]):
467                 failure[event_id] = "no_compatible_room_under_scenario"
468             else:
469                 failure[event_id] = "no_complete_plan_by_deadline"
470         else:
471             plans[event_id] = steps
472         if count % 25000 == 0:
473             print(f" on-time pass {count:}/{len(event_ids):}", flush=True)
474     print(
475         f" deadline pass complete: scheduled={len(plans):}, unallocated={len(failure
):},",
476         flush=True,
477     )
478     return plans, failure
479
480
481 def plan_to_schedule(
482     calendar: WorkCalendar,
483     plans: dict[str, list[PlanStep]],
484     events: pd.DataFrame,
485     policy: str,
486     room_names: dict[str, str],
487 ) -> pd.DataFrame:
488     event_meta = events.set_index("event_id")
489     rows: list[dict[str, Any]] = []
490     for event_id, steps in plans.items():
491         event = event_meta.loc[event_id]
492         ordered = sorted(steps, key=lambda step: step.sequence_position)
493         for step in ordered:
494             start = calendar.times[step.start_index]
495             end = calendar.end_timestamp(step.start_index, step.task.duration_slots)
496             rows.append(
497                 {
498                     "policy": policy,
499                     "event_id": event_id,
500                     "source": event.source,
501                     "patient_id": event.patient_id,
502                     "item_index": step.task.item_index,
503                     "sequence_position": step.sequence_position,
504                     "project_norm": step.task.project_norm,
505                     "category": step.task.category,
506                     "start_dt": start,

```

```

507         "end_dt": end,
508         "room_id": step.room_id,
509         "room_name": room_names.get(step.room_id, step.room_id),
510         "duration_minutes": step.task.duration_slots * SLOT_MINUTES,
511         "transfer_minutes_before": step.transfer_slots_before *
SLOT_MINUTES,
512         "room_switch_from_previous": step.transfer_slots_before > 0,
513         "capability_level": step.task.capability_level,
514         "fasting": step.task.fasting,
515         "bladder": step.task.bladder,
516         "bedside": step.task.bedside,
517         "release_dt": event.release_dt,
518         "deadline_dt": event.deadline_dt,
519         "evaluation_cohort": event.evaluation_cohort,
520     }
521 )
522 return pd.DataFrame(rows)
523
524
525 def outcomes_from_schedule(
526     events: pd.DataFrame,
527     schedule: pd.DataFrame,
528     failure: dict[str, str],
529     policy: str,
530 ) -> pd.DataFrame:
531     if schedule.empty:
532         aggregates = pd.DataFrame(columns=["event_id"])
533     else:
534         aggregates = (
535             schedule.groupby("event_id", as_index=False)
536             .agg(
537                 scheduled_items=("item_index", "size"),
538                 first_start_dt=("start_dt", "min"),
539                 completion_dt=("end_dt", "max"),
540                 room_count=("room_id", "nunique"),
541                 room_switch_count=("room_switch_from_previous", "sum"),
542                 transfer_minutes=("transfer_minutes_before", "sum"),
543                 fallback_task_count=("capability_level", lambda values: values.eq("
category_fallback").sum()),
544             )
545         )
546     result = events.merge(aggregates, on="event_id", how="left", validate="one_to_one"
)
547     result["scheduled_items"] = result["scheduled_items"].fillna(0).astype(int)
548     result["complete_event"] = result["scheduled_items"].eq(result["
service_project_count"])
549     result["deadline_met"] = result["complete_event"] & result["completion_dt"].le(
result["deadline_dt"])
550     result["waiting_hours"] = (
551         result["first_start_dt"] - result["release_dt"]
552     ).dt.total_seconds() / 3600
553     result["failure_reason"] = result["event_id"].map(failure)
554     result.loc[
555         ~result["deadline_met"] & result["failure_reason"].isna(), "failure_reason"
556     ] = "completed_after_deadline"
557     result.loc[result["deadline_met"], "failure_reason"] = ""
558     result["policy"] = policy
559     return result
560
561
562 def metrics_from_outcomes(
563     outcomes: pd.DataFrame,
564     schedule: pd.DataFrame,
565     calendar: WorkCalendar,

```

```

566     policy: str,
567     scenario: str,
568 ) -> dict[str, Any]:
569     cohort = outcomes[outcomes["evaluation_cohort"]]
570     inpatient = cohort[cohort["source"].eq("住院")]
571     background = cohort[~cohort["source"].eq("住院")]
572     scheduled_wait = inpatient["waiting_hours"].dropna()
573     room_minutes = float(schedule["duration_minutes"].sum()) if not schedule.empty
574     else 0.0
575     staffed_minutes = float(calendar.capacity.sum() * SLOT_MINUTES)
576     return {
577         "scenario": scenario,
578         "policy": policy,
579         "inpatient_events": int(len(inpatient)),
580         "inpatient_complete_48h": int(inpatient["deadline_met"].sum()),
581         "inpatient_48h_rate": float(inpatient["deadline_met"].mean()),
582         "inpatient_scheduled_complete": int(inpatient["complete_event"].sum()),
583         "inpatient_unscheduled": int((~inpatient["complete_event"]).sum()),
584         "inpatient_wait_p50_hours_conditional": float(scheduled_wait.quantile(0.50)),
585         "inpatient_wait_p90_hours_conditional": float(scheduled_wait.quantile(0.90)),
586         "inpatient_wait_p95_hours_conditional": float(scheduled_wait.quantile(0.95)),
587         "inpatient_room_switch_rate_multi": float(
588             inpatient.loc[inpatient["service_project_count"].gt(1), "room_switch_count"]
589             .fillna(0).gt(0).mean()
590         ),
591         "background_events": int(len(background)),
592         "background_on_planned_day": int(background["deadline_met"].sum()),
593         "background_on_time_rate": float(background["deadline_met"].mean()),
594         "background_scheduled_complete": int(background["complete_event"].sum()),
595         "fallback_scheduled_task_share": float(schedule["capability_level"].eq("category_fallback").mean()),
596         "scheduled_minutes": room_minutes,
597         "staffed_capacity_utilization": room_minutes / staffed_minutes if
598         staffed_minutes else np.nan,
599     }
600
601 def daily_outputs(
602     outcomes: pd.DataFrame,
603     schedule: pd.DataFrame,
604     calendar: WorkCalendar,
605     policy: str,
606 ) -> tuple[pd.DataFrame, pd.DataFrame]:
607     working = outcomes[outcomes["evaluation_cohort"]].copy()
608     working["metric_date"] = working["order_dt"].where(
609         working["source"].eq("住院"), working["planned_date"]
610     ).dt.normalize()
611     daily = (
612         working.groupby(["metric_date", "source"], as_index=False)
613         .agg(
614             events=("event_id", "size"),
615             deadline_met=("deadline_met", "sum"),
616             scheduled_complete=("complete_event", "sum"),
617             demand_nominal_minutes=("nominal_minutes", "sum"),
618             demand_upper_minutes=("upper_minutes", "sum"),
619         )
620     )
621     daily["deadline_rate"] = daily["deadline_met"] / daily["events"]
622     daily["policy"] = policy
623
624     if schedule.empty:
625         utilization = pd.DataFrame()
626     else:
627         schedule = schedule.copy()

```

```

626         schedule["date"] = schedule["start_dt"].dt.normalize()
627         utilization = (
628             schedule.groupby(["date", "room_id", "room_name"], as_index=False)["
duration_minutes"]
629             .sum()
630         )
631         utilization["room_utilization"] = utilization["duration_minutes"] / (8 * 60)
632         utilization["policy"] = policy
633         return daily, utilization
634
635
636 def run_policy(
637     events: pd.DataFrame,
638     tasks: dict[str, list[Task]],
639     slot_capacity: pd.DataFrame,
640     room_ids: list[str],
641     room_names: dict[str, str],
642     policy: str,
643     capacity_column: str,
644     scenario: str,
645     transfer_minutes: int = TRANSFER_MINUTES,
646     background_state: tuple[WorkCalendar, dict[str, list[PlanStep]], dict[str, str]] |
None = None,
647 ) -> tuple[
648     pd.DataFrame,
649     pd.DataFrame,
650     dict[str, Any],
651     pd.DataFrame,
652     pd.DataFrame,
653     tuple[WorkCalendar, dict[str, list[PlanStep]], dict[str, str]],
654 ]:
655     config = POLICIES[policy]
656     scarcity = task_room_scarcity()
657     simulation_start = source_config.SIMULATION_START
658     simulation_end = source_config.HOLDOUT_END_EXCLUSIVE + pd.Timedelta(days=
FOLLOWUP_DAYS)
659     lookup = events.set_index("event_id")
660     inpatient_order = event_priority(events, tasks, "inpatient", config["inpatient"])
661     if background_state is None:
662         calendar = WorkCalendar(
663             simulation_start,
664             simulation_end,
665             room_ids,
666             slot_capacity,
667             capacity_column,
668             scarcity,
669             config["background"],
670             transfer_minutes,
671         )
672         background_order = event_priority(events, tasks, "background", config["
background"])
673         print(f"{policy}: scheduling {len(background_order):,} outpatient/physical-
exam events", flush=True)
674         background_plans, background_failure = schedule_group(
675             calendar, background_order, lookup, tasks
676         )
677         reusable_background = (
678             calendar.clone(),
679             background_plans,
680             background_failure,
681         )
682     else:
683         stored_calendar, background_plans, background_failure = background_state
684         calendar = stored_calendar.clone()

```

```

685     reusable_background = background_state
686     print(f"{policy}: reusing identical background allocation", flush=True)
687     print(f"{policy}: scheduling {len(inpatient_order):,} inpatient events", flush=True)
688     inpatient_plans, inpatient_failure = schedule_group(
689         calendar, inpatient_order, lookup, tasks
690     )
691     plans = {**background_plans, **inpatient_plans}
692     failure = {**background_failure, **inpatient_failure}
693     schedule = plan_to_schedule(calendar, plans, events, policy, room_names)
694     outcomes = outcomes_from_schedule(events, schedule, failure, policy)
695     metrics = metrics_from_outcomes(outcomes, schedule, calendar, policy, scenario)
696     metrics["transfer_minutes_assumption"] = transfer_minutes
697     daily, utilization = daily_outputs(outcomes, schedule, calendar, policy)
698     print(
699         f"{policy}: inpatient48h={metrics['inpatient_48h_rate']:.4%}, "
700         f"background={metrics['background_on_time_rate']:.4%}, tasks={len(schedule):,}"
701     ), flush=True,
702 )
703     return schedule, outcomes, metrics, daily, utilization, reusable_background
704
705 def recommendation_table(schedule: pd.DataFrame, outcomes: pd.DataFrame) -> pd.
706     DataFrame:
707     evaluation = as_bool(outcomes["evaluation_cohort"])
708     inpatient_ids = set(
709         outcomes.loc[outcomes["source"].eq("住院") & evaluation, "event_id"]
710     )
711     result = schedule[schedule["event_id"].isin(inpatient_ids)].copy()
712     result = result.sort_values(["start_dt", "room_id", "event_id", "sequence_position"], kind="stable")
713     event_rank = (
714         result[["event_id"]]
715         .drop_duplicates()
716         .assign(first_start=result.groupby("event_id", sort=False)["start_dt"].first()
717             .values)
718         .sort_values(["first_start", "event_id"], kind="stable")
719     )
720     event_rank["recommended_patient_rank"] = np.arange(1, len(event_rank) + 1)
721     result = result.merge(
722         event_rank[["event_id", "recommended_patient_rank"]], on="event_id", how="left"
723     )
724     columns = [
725         "recommended_patient_rank",
726         "event_id",
727         "patient_id",
728         "sequence_position",
729         "project_norm",
730         "category",
731         "start_dt",
732         "end_dt",
733         "room_id",
734         "room_name",
735         "transfer_minutes_before",
736         "fasting",
737         "bladder",
738         "bedside",
739         "capability_level",
740     ]
741     return result[columns].sort_values(
742         ["recommended_patient_rank", "sequence_position"], kind="stable"

```

```

743
744
745 def main() -> None:
746     parser = argparse.ArgumentParser()
747     parser.add_argument("--output-dir", type=Path, default=OUT)
748     parser.add_argument("--main-only", action="store_true")
749     parser.add_argument("--sensitivity-only", action="store_true")
750     parser.add_argument("--main-policy", choices=list(POLICIES))
751     parser.add_argument("--sensitivity-scenario")
752     args = parser.parse_args()
753     if args.main_only and args.sensitivity_only:
754         parser.error("--main-only and --sensitivity-only cannot be used together")
755     if args.sensitivity_scenario and not args.sensitivity_only:
756         parser.error("--sensitivity-scenario requires --sensitivity-only")
757     if args.main_policy and args.sensitivity_only:
758         parser.error("--main-policy cannot be combined with --sensitivity-only")
759     args.output_dir.mkdir(parents=True, exist_ok=True)
760
761     metrics_rows: list[dict[str, Any]] = []
762     daily_rows: list[pd.DataFrame] = []
763     authoritative_schedule: pd.DataFrame | None = None
764     authoritative_outcomes: pd.DataFrame | None = None
765     authoritative_utilization: pd.DataFrame | None = None
766     if args.sensitivity_only:
767         existing_metrics = pd.read_csv(args.output_dir / "policy_scenario_metrics.csv"
768 )
769         existing_metrics = existing_metrics[existing_metrics["scenario"].eq("
770 main_5pct_upper")]
771         metrics_rows.extend(existing_metrics.to_dict(orient="records"))
772         existing_daily = pd.read_csv(args.output_dir / "daily_service_metrics.csv",
773 low_memory=False)
774         if "scenario" not in existing_daily.columns:
775             existing_daily["scenario"] = "main_5pct_upper"
776             existing_daily["scenario"] = existing_daily["scenario"].fillna("
777 main_5pct_upper")
778         daily_rows.append(existing_daily[existing_daily["scenario"].eq("
779 main_5pct_upper")])
780     else:
781         main_events, main_tasks, slot_capacity, room_ids, room_names = load_inputs(
782             "five_percent_upper", "hierarchical", source_config.
783 BLADDER_PREPARATION_MINUTES
784 )
785         background_cache: dict[
786             str, tuple[WorkCalendar, dict[str, list[PlanStep]], dict[str, str]]
787         ] = {}
788         selected_policies = [args.main_policy] if args.main_policy else list(POLICIES)
789         for policy in selected_policies:
790             background_key = POLICIES[policy]["background"]
791             schedule, outcomes, metrics, daily, utilization, reusable_background =
792 run_policy(
793     main_events,
794     main_tasks,
795     slot_capacity,
796     room_ids,
797     room_names,
798     policy,
799     "capacity_main",
800     "main_5pct_upper",
801     TRANSFER_MINUTES,
802     background_cache.get(background_key),
803 )
804         background_cache.setdefault(background_key, reusable_background)
805         metrics_rows.append(metrics)
806         daily_rows.append(daily.assign(scenario="main_5pct_upper"))

```

```

800         if policy == AUTHORITATIVE_POLICY:
801             authoritative_schedule = schedule
802             authoritative_outcomes = outcomes
803             authoritative_utilization = utilization
804
805         sensitivity_specs = [] if args.main_only else [
806             ("nominal_no_special", "five_percent_upper", "hierarchical", "capacity_main",
807              60, False, 10),
808             ("low_doctor_q25", "five_percent_upper", "hierarchical", "capacity_low", 60,
809              True, 10),
810             ("strict_capability", "five_percent_upper", "strict", "capacity_main", 60,
811              True, 10),
812             ("bladder_45min", "five_percent_upper", "hierarchical", "capacity_main", 45,
813              True, 10),
814             ("bladder_90min", "five_percent_upper", "hierarchical", "capacity_main", 90,
815              True, 10),
816             ("transfer_5min", "five_percent_upper", "hierarchical", "capacity_main", 60,
817              True, 5),
818             ("transfer_15min", "five_percent_upper", "hierarchical", "capacity_main", 60,
819              True, 15),
820         ]
821         if args.sensitivity_scenario:
822             sensitivity_specs = [
823                 spec for spec in sensitivity_specs if spec[0] == args.sensitivity_scenario
824             ]
825             if not sensitivity_specs:
826                 parser.error(f"unknown sensitivity scenario: {args.sensitivity_scenario}")
827             # The first tuple uses a flag to switch all events to nominal duration.
828             for name, special_mode, capability_mode, capacity_column, bladder_minutes,
829                 use_special, transfer_minutes in sensitivity_specs:
830                 effective_special = special_mode if use_special else "all_nominal"
831                 events, tasks, capacity, rooms, names = load_inputs(
832                     effective_special, capability_mode, bladder_minutes
833                 )
834                 _, _, metrics, daily, _, _ = run_policy(
835                     events,
836                     tasks,
837                     capacity,
838                     rooms,
839                     names,
840                     AUTHORITATIVE_POLICY,
841                     capacity_column,
842                     name,
843                     transfer_minutes,
844                 )
845                 metrics_rows.append(metrics)
846                 daily_rows.append(daily.assign(scenario=name))
847
848         metrics_frame = pd.DataFrame(metrics_rows)
849         metrics_frame.to_csv(args.output_dir / "policy_scenario_metrics.csv", index=False,
850                             encoding="utf-8-sig")
851         pd.concat(daily_rows, ignore_index=True).to_csv(
852             args.output_dir / "daily_service_metrics.csv", index=False, encoding="utf-8-
853             sig"
854         )
855         if not args.sensitivity_only:
856             assert authoritative_schedule is not None
857             assert authoritative_outcomes is not None
858             assert authoritative_utilization is not None
859             authoritative_schedule.to_csv(
860                 args.output_dir / "final_patient_task_schedule.csv", index=False, encoding
861                 ="utf-8-sig"
862             )
863             authoritative_outcomes.to_csv(

```

```

853         args.output_dir / "final_patient_outcomes.csv", index=False, encoding="utf
-8-sig"
854     )
855     authoritative_utilization.to_csv(
856         args.output_dir / "final_room_utilization_daily.csv", index=False,
encoding="utf-8-sig"
857     )
858     recommendation_table(authoritative_schedule, authoritative_outcomes).to_csv(
859         args.output_dir / "inpatient_recommended_schedule.csv", index=False,
encoding="utf-8-sig"
860     )
861     summary = {
862         "authoritative_policy": AUTHORITATIVE_POLICY,
863         "annual_state_reset_count": 1,
864         "simulation_start": str(source_config.SIMULATION_START.date()),
865         "simulation_end_exclusive": str(
866             (source_config.HOLDOUT_END_EXCLUSIVE + pd.Timedelta(days=FOLLOWUP_DAYS)).
date()
867         ),
868         "evaluation_start": str(source_config.HOLDOUT_START.date()),
869         "evaluation_end_exclusive": str(source_config.HOLDOUT_END_EXCLUSIVE.date()),
870         "background_interpretation": "retrospective plan-day scenario reconstructed
from report date, with release no earlier than the observed order time; report
time/room/doctor dispatch are excluded",
871         "official_kpi_boundary": "scheduled service completion is used as report-
submission completion proxy because scan-end timestamp is absent",
872         "transfer_minutes_between_rooms": TRANSFER_MINUTES,
873         "metrics": metrics_frame.to_dict(orient="records"),
874     }
875     (args.output_dir / "summary.json").write_text(
876         json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
877     )
878     print(metrics_frame.to_string(index=False), flush=True)
879
880
881 if __name__ == "__main__":
882     main()

```

H 独立可行性核验完整程序

Listing 7 不调用调度器内核的独立约束核验

```

1  """Independent feasibility checks for the authoritative final schedule.
2
3  This verifier deliberately does not import the scheduler. It reconstructs
4  duration, compatibility, interval and capacity checks from the exported input
5  and result tables so that implementation errors are not silently shared.
6  """
7
8  from __future__ import annotations
9
10 import json
11 import math
12 from pathlib import Path
13
14 import numpy as np
15 import pandas as pd
16
17
18 ROOT = Path(__file__).resolve().parents[2]
19 INPUT = ROOT / "supporting_materials" / "processed_data" / "unified_holdout"
20 RESULT = ROOT / "supporting_materials" / "results" / "unified_schedule"
21 SLOT_MINUTES = 5

```

```

22 TRANSFER_MINUTES = 10
23
24
25 def as_bool(series: pd.Series) -> pd.Series:
26     if pd.api.types.is_bool_dtype(series):
27         return series.fillna(False)
28     return series.astype("string").str.lower().isin(["true", "1", "yes"])
29
30
31 def room_set(value: object) -> set[str]:
32     if pd.isna(value):
33         return set()
34     return {part for part in str(value).split("|") if part}
35
36
37 def slot_index(timestamp: pd.Timestamp) -> int | None:
38     minute = timestamp.hour * 60 + timestamp.minute
39     if 8 * 60 <= minute < 12 * 60:
40         return (minute - 8 * 60) // SLOT_MINUTES
41     if 13 * 60 <= minute < 17 * 60:
42         return 48 + (minute - 13 * 60) // SLOT_MINUTES
43     return None
44
45
46 def main() -> None:
47     events = pd.read_csv(
48         INPUT / "events.csv",
49         encoding="utf-8-sig",
50         parse_dates=["order_dt", "planned_date", "release_dt", "deadline_dt"],
51     )
52     items = pd.read_csv(INPUT / "items.csv", encoding="utf-8-sig", low_memory=False)
53     schedule = pd.read_csv(
54         RESULT / "final_patient_task_schedule.csv",
55         encoding="utf-8-sig",
56         parse_dates=["start_dt", "end_dt", "release_dt", "deadline_dt"],
57         dtype={"room_id": "string"},
58         low_memory=False,
59     )
60     outcomes = pd.read_csv(
61         RESULT / "final_patient_outcomes.csv",
62         encoding="utf-8-sig",
63         parse_dates=["release_dt", "deadline_dt", "first_start_dt", "completion_dt"],
64     )
65     capacity = pd.read_csv(INPUT / "doctor_slot_capacity.csv", encoding="utf-8-sig")
66     for frame, columns in [
67         (events, ["special_case", "evaluation_cohort", "bedside", "fasting", "bladder"]),
68         (items, ["bedside", "fasting", "bladder"]),
69         (schedule, ["bedside", "fasting", "bladder", "room_switch_from_previous"]),
70         (outcomes, ["complete_event", "deadline_met", "evaluation_cohort"]),
71     ]:
72         for column in columns:
73             frame[column] = as_bool(frame[column])
74
75     checks: dict[str, dict[str, object]] = {}
76
77     def record(name: str, violation_count: int, detail: object = "") -> None:
78         checks[name] = {
79             "passed": int(violation_count) == 0,
80             "violation_count": int(violation_count),
81             "detail": detail,
82         }
83
84     record(

```

```

85     "unique_exported_task_key",
86     int(schedule.duplicated(["event_id", "item_index"]).sum()),
87     "each scheduled task must occur exactly once",
88 )
89 item_keys = items[["event_id", "item_index"]].drop_duplicates()
90 merged = schedule.merge(
91     items,
92     on=["event_id", "item_index"],
93     how="left",
94     suffixes=("_scheduled", "_input"),
95     indicator=True,
96     validate="many_to_one",
97 )
98 record("scheduled_task_exists_in_input", int(merged["_merge"].ne("both").sum()))
99
100 event_meta = events.set_index("event_id")
101 merged["special_case_input"] = merged["event_id"].map(event_meta["special_case"])
102 merged["expected_duration_minutes"] = np.where(
103     merged["special_case_input"],
104     merged["duration_upper_min"],
105     merged["duration_nominal_min"],
106 )
107 merged["expected_duration_minutes"] = (
108     np.ceil(merged["expected_duration_minutes"].astype(float) / SLOT_MINUTES)
109     .clip(lower=1)
110     * SLOT_MINUTES
111 )
112 duration_bad = (
113     merged["duration_minutes"].ne(merged["expected_duration_minutes"])
114     | (merged["end_dt"] - merged["start_dt"]).dt.total_seconds().div(60).ne(
115         merged["duration_minutes"]
116     )
117 )
118 record("duration_matches_5pct_scenario", int(duration_bad.sum()))
119
120 compat_bad = [
121     room not in room_set(rooms)
122     for room, rooms in zip(merged["room_id"].astype(str), merged["compatible_rooms"])
123 ]
124 record("project_room_compatibility", int(sum(compat_bad)))
125 record(
126     "bedside_uses_room_7",
127     int((merged["bedside_scheduled"] & merged["room_id"].ne("7")).sum()),
128 )
129
130 release = schedule["event_id"].map(event_meta["release_dt"])
131 deadline = schedule["event_id"].map(event_meta["deadline_dt"])
132 record("task_not_before_release", int(schedule["start_dt"].lt(release).sum()))
133 record("task_complete_by_event_deadline", int(schedule["end_dt"].gt(deadline).sum()))
134 record(
135     "five_minute_grid",
136     int(
137         (
138             schedule["start_dt"].dt.minute.mod(SLOT_MINUTES).ne(0)
139             | schedule["end_dt"].dt.minute.mod(SLOT_MINUTES).ne(0)
140             | schedule["start_dt"].dt.second.ne(0)
141             | schedule["end_dt"].dt.second.ne(0)
142         ).sum()
143     ),
144 )
145
146 block_bad = 0

```

```

147 for row in schedule[["start_dt", "end_dt"]].itertuples(index=False):
148     morning = row.start_dt.hour < 12 and row.end_dt <= row.start_dt.normalize() +
pd.Timedelta(hours=12)
149     afternoon = row.start_dt.hour >= 13 and row.end_dt <= row.start_dt.normalize()
+ pd.Timedelta(hours=17)
150     if row.start_dt.normalize() != row.end_dt.normalize() or not (morning or
afternoon):
151         block_bad += 1
152     record("inside_single_work_block", block_bad)
153
154     fasting_bad = schedule["fasting"] & schedule["end_dt"].gt(
155         schedule["start_dt"].dt.normalize() + pd.Timedelta(hours=10)
156     )
157     record("fasting_complete_by_10", int(fasting_bad.sum()))
158
159     room_overlap = 0
160     for _, group in schedule.sort_values(["room_id", "start_dt"]).groupby("room_id",
sort=False):
161         room_overlap += int(group["start_dt"].iloc[1:].reset_index(drop=True).lt(
162             group["end_dt"].iloc[:-1].reset_index(drop=True)
163             ).sum())
164     record("no_room_overlap", room_overlap)
165
166     event_overlap = 0
167     transfer_bad = 0
168     sequence_bad = 0
169     for _, group in schedule.sort_values(["event_id", "sequence_position"]).groupby("
event_id", sort=False):
170         expected_sequence = np.arange(1, len(group) + 1)
171         sequence_bad += int((group["sequence_position"].to_numpy() !=
expected_sequence).sum())
172         if len(group) < 2:
173             continue
174         starts = group["start_dt"].iloc[1:].reset_index(drop=True)
175         ends = group["end_dt"].iloc[:-1].reset_index(drop=True)
176         prior_rooms = group["room_id"].iloc[:-1].reset_index(drop=True)
177         current_rooms = group["room_id"].iloc[1:].reset_index(drop=True)
178         switches = current_rooms.ne(prior_rooms)
179         event_overlap += int(starts.lt(ends).sum())
180         transfer_bad += int((switches & starts.lt(ends + pd.Timedelta(minutes=
TRANSFER_MINUTES))).sum())
181     record("contiguous_event_sequence_numbers", sequence_bad)
182     record("no_same_patient_task_overlap", event_overlap)
183     record("cross_room_transfer_at_least_10min", transfer_bad)
184
185     # Reconstruct doctor concurrency by expanding each scheduled interval to
186     # its occupied 5-minute starts, then compare with weekday-slot capacity.
187     occupied_parts: list[pd.DataFrame] = []
188     for slots, group in schedule.groupby(schedule["duration_minutes"].floordiv(
SLOT_MINUTES).astype(int)):
189         base = group[["start_dt"]].reset_index(drop=True)
190         repeated = pd.DataFrame(
191             {
192                 "slot_dt": np.concatenate(
193                     [
194                         (base["start_dt"] + pd.Timedelta(minutes=offset * SLOT_MINUTES
195                         for offset in range(int(slots)))
196                     ]
197                 )
198             }
199         )
200         occupied_parts.append(repeated)
201     occupied = pd.concat(occupied_parts, ignore_index=True)

```

```

202 load = occupied.groupby("slot_dt", as_index=False).size().rename(columns={"size":
    "doctor_load"})
203 load["weekday"] = load["slot_dt"].dt.dayofweek
204 load["slot_index"] = load["slot_dt"].map(slot_index)
205 cap = capacity[["weekday", "slot_index", "capacity_main"]]
206 load = load.merge(cap, on=["weekday", "slot_index"], how="left", validate="
    many_to_one")
207 record("doctor_capacity_defined", int(load["capacity_main"].isna().sum()))
208 record(
209     "doctor_concurrency_within_capacity",
210     int(load["doctor_load"].gt(load["capacity_main"]).sum()),
211     {
212         "max_observed_load": int(load["doctor_load"].max()),
213         "max_capacity": int(load["capacity_main"].max()),
214     },
215 )
216
217 scheduled_count = schedule.groupby("event_id").size()
218 expected_count = events.set_index("event_id")["service_project_count"]
219 computed_complete = scheduled_count.reindex(expected_count.index, fill_value=0).eq
    (expected_count)
220 outcome_complete = outcomes.set_index("event_id")["complete_event"].reindex(
    expected_count.index)
221 record("outcome_complete_flag_consistency", int(computed_complete.ne(
    outcome_complete).sum()))
222 computed_deadline = computed_complete.copy()
223 completion = schedule.groupby("event_id")["end_dt"].max().reindex(expected_count.
    index)
224 computed_deadline &= completion.le(events.set_index("event_id")["deadline_dt"])
225 outcome_deadline = outcomes.set_index("event_id")["deadline_met"].reindex(
    expected_count.index)
226 record("outcome_deadline_flag_consistency", int(computed_deadline.ne(
    outcome_deadline).sum()))
227
228 passed = all(entry["passed"] for entry in checks.values())
229 report = {
230     "passed": passed,
231     "schedule_rows": int(len(schedule)),
232     "scheduled_events": int(schedule["event_id"].nunique()),
233     "input_events": int(len(events)),
234     "checks": checks,
235 }
236 RESULT.mkdir(parents=True, exist_ok=True)
237 (RESULT / "independent_verification.json").write_text(
238     json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8"
239 )
240 print(json.dumps(report, ensure_ascii=False, indent=2))
241 if not passed:
242     raise SystemExit(1)
243
244
245 if __name__ == "__main__":
246     main()

```

I 论文图表生成完整程序

Listing 8 论文九组图形生成程序

```

1 """Create final paper figures from the authoritative unified result chain."""
2
3 from __future__ import annotations
4
5 from pathlib import Path

```

```

6
7 import matplotlib.dates as mdates
8 import matplotlib.pyplot as plt
9 from matplotlib import font_manager
10 from matplotlib.patches import Patch
11 import numpy as np
12 import pandas as pd
13
14
15 ROOT = Path(__file__).resolve().parents[2]
16 P1 = ROOT / "supporting_materials" / "results" / "p1"
17 UNIFIED = ROOT / "supporting_materials" / "results" / "unified_schedule"
18 INPUT = ROOT / "supporting_materials" / "processed_data" / "unified_holdout"
19 CAPABILITY = ROOT / "supporting_materials" / "results" / "capability"
20 OUT = ROOT / "supporting_materials" / "figures" / "paper_final"
21
22 BLUE = "#276FBF"
23 ORANGE = "#F28E2B"
24 GREEN = "#3A9D5D"
25 RED = "#C93C3C"
26 PURPLE = "#7A5195"
27 GRAY = "#6B7280"
28 LIGHT = "#E8EEF5"
29
30
31 def configure() -> None:
32     candidates = ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "Arial Unicode MS"]
33     available = {font.name for font in font_manager.fontManager.ttflist}
34     family = next((name for name in candidates if name in available), "DejaVu Sans")
35     plt.rcParams.update(
36         {
37             "font.family": family,
38             "axes.unicode_minus": False,
39             "axes.spines.top": False,
40             "axes.spines.right": False,
41             "axes.titleweight": "bold",
42             "axes.labelcolor": "#263238",
43             "text.color": "#263238",
44             "xtick.color": "#455A64",
45             "ytick.color": "#455A64",
46             "figure.dpi": 150,
47             "savefig.dpi": 300,
48         }
49     )
50     OUT.mkdir(parents=True, exist_ok=True)
51
52
53 def save(fig: plt.Figure, name: str) -> None:
54     fig.savefig(OUT / f"{name}.pdf", bbox_inches="tight")
55     plt.close(fig)
56
57
58 def monthly_demand() -> None:
59     frame = pd.read_csv(P1 / "monthly_trends.csv", parse_dates=["period"])
60     frame = frame[frame["basis"].eq("开单日")]
61     colors = {"住院": BLUE, "门诊": ORANGE, "体检": GREEN}
62     fig, ax = plt.subplots(figsize=(10.2, 4.4))
63     for source, group in frame.groupby("source", sort=False):
64         group = group.sort_values("period")
65         ax.plot(
66             group["period"],
67             group["mean_daily_episodes"],
68             lw=1.8,
69             label=source,

```

```

70         color=colors[source],
71     )
72     split = pd.Timestamp("2024-04-01")
73     ax.axvspan(split, pd.Timestamp("2025-04-01"), color=LIGHT, alpha=0.9, zorder=0)
74     ax.axvline(split, color=GRAY, lw=1.1, ls="--")
75     ax.text(split + pd.Timedelta(days=10), ax.get_ylim()[1] * 0.93, "时间顺序留出期",
76            color=GRAY)
77     ax.set_title("三类患者月度日均开单量及时间顺序留出边界")
78     ax.set_ylabel("日均事件数")
79     ax.set_xlabel("月份")
80     ax.legend(ncol=3, frameon=False, loc="upper left")
81     ax.grid(axis="y", color="#D7DEE8", lw=0.7, alpha=0.8)
82     fig.tight_layout()
83     save(fig, "fig01_monthly_demand")
84
85 def doctor_capacity() -> None:
86     frame = pd.read_csv(INPUT / "doctor_slot_capacity.csv")
87     labels = ["周一", "周二", "周三", "周四", "周五", "周六", "周日"]
88     fig, axes = plt.subplots(2, 1, figsize=(10.2, 5.8), sharex=True)
89     for ax, column, title in zip(
90         axes,
91         ["capacity_main", "capacity_low"],
92         ["训练日中位数容量（主情景）", "训练日下四分位容量（偏紧情景）"],
93     ):
94         for weekday, group in frame.groupby("weekday"):
95             x = np.arange(len(group))
96             ax.plot(x, group[column], lw=1.25, label=labels[int(weekday)])
97             ax.set_ylabel("可并发医生数")
98             ax.set_title(title, loc="left", fontsize=11)
99             ax.grid(axis="y", color="#D7DEE8", lw=0.6)
100         tick_index = [0, 12, 24, 36, 47, 48, 60, 72, 84, 95]
101         tick_label = ["08:00", "09:00", "10:00", "11:00", "11:55", "13:00", "14:00", "
15:00", "16:00", "16:55"]
102         axes[-1].set_xticks(tick_index, tick_label)
103         axes[-1].set_xlabel("5 分钟时隙")
104         axes[0].legend(ncol=7, frameon=False, loc="upper center", bbox_to_anchor=(0.5,
1.32))
105     fig.suptitle("由训练期医生工作日记录重构的共享时隙容量", y=1.02, fontweight="bold")
106     fig.tight_layout()
107     save(fig, "fig02_doctor_capacity")
108
109
110 def capability_coverage() -> None:
111     items = pd.read_csv(INPUT / "items.csv", low_memory=False)
112     events = pd.read_csv(INPUT / "events.csv", usecols=["event_id", "source"])
113     frame = items.merge(events, on="event_id", how="left", validate="many_to_one")
114     level = np.where(
115         frame["capability_level"].eq("category_fallback"),
116         "类别回退",
117         np.where(frame["capability_level"].eq("explicit_bedside"), "床旁明确", "严格项目
118         证据"),
119     )
120     frame["evidence_group"] = level
121     table = frame.groupby(["source", "evidence_group"]).size().unstack(fill_value=0)
122     table = table.div(table.sum(axis=1), axis=0).reindex(["住院", "门诊", "体检"])
123     order = ["严格项目证据", "床旁明确", "类别回退"]
124     colors = [BLUE, GREEN, ORANGE]
125     fig, ax = plt.subplots(figsize=(8.8, 3.6))
126     left = np.zeros(len(table))
127     for column, color in zip(order, colors):
128         values = table.get(column, pd.Series(0, index=table.index)).to_numpy()
129         bars = ax.barh(table.index, values, left=left, color=color, height=0.55, label
=column)

```

```

129         for bar, value, start in zip(bars, values, left):
130             if value >= 0.045:
131                 ax.text(start + value / 2, bar.get_y() + bar.get_height() / 2, f"{
value:.1%}", ha="center", va="center", color="white", fontsize=9)
132             left += values
133         ax.set_xlim(0, 1)
134         ax.xaxis.set_major_formatter(lambda value, _: f"{value:.0%}")
135         ax.set_xlabel("项目任务占比")
136         ax.set_title("三类患者项目—设备兼容证据覆盖")
137         ax.legend(ncol=3, frameon=False, loc="lower center", bbox_to_anchor=(0.5, -0.38))
138         fig.tight_layout()
139         save(fig, "fig03_capability_coverage")
140
141
142 def policy_tradeoff() -> None:
143     metrics = pd.read_csv(UNIFIED / "policy_scenario_metrics.csv")
144     frame = metrics[metrics["scenario"].eq("main_5pct_upper")].copy()
145     labels = {
146         "FCFS_SHARED": "共享 FCFS",
147         "SLACK_GUARD_SHARED": "松弛度保护",
148         "JOINT_SCARCITY": "联合稀缺度",
149     }
150     colors = {"FCFS_SHARED": BLUE, "SLACK_GUARD_SHARED": ORANGE, "JOINT_SCARCITY":
GREEN}
151     fig, ax = plt.subplots(figsize=(7.5, 5.2))
152     for row in frame.iterrows(index=False):
153         ax.scatter(
154             row.background_on_time_rate,
155             row.inpatient_48h_rate,
156             s=170,
157             color=colors[row.policy],
158             edgecolor="white",
159             linewidth=1.2,
160             zorder=3,
161         )
162         ax.annotate(
163             labels[row.policy],
164             (row.background_on_time_rate, row.inpatient_48h_rate),
165             xytext=(7, 7),
166             textcoords="offset points",
167             fontsize=10,
168         )
169     ax.set_xlabel("门诊/体检预约日内完成率")
170     ax.set_ylabel("住院 48 小时内完成率")
171     ax.xaxis.set_major_formatter(lambda value, _: f"{value:.1%}")
172     ax.yaxis.set_major_formatter(lambda value, _: f"{value:.1%}")
173     ax.grid(color="#D7DEE8", lw=0.7)
174     ax.set_title("共享资源下三种患者级排程策略比较")
175     ax.set_xlim(frame["background_on_time_rate"].min() - 0.008, frame["
background_on_time_rate"].max() + 0.013)
176     ax.set_ylim(frame["inpatient_48h_rate"].min() - 0.006, frame["inpatient_48h_rate"
].max() + 0.006)
177     fig.tight_layout()
178     save(fig, "fig04_policy_tradeoff")
179
180
181 def sensitivity() -> None:
182     metrics = pd.read_csv(UNIFIED / "policy_scenario_metrics.csv")
183     main = metrics[
184         metrics["scenario"].eq("main_5pct_upper") & metrics["policy"].eq("FCFS_SHARED"
)
185     ].iloc[0]
186     frame = metrics[~metrics["scenario"].eq("main_5pct_upper")].copy()
187     name_map = {

```

```

188     "nominal_no_special": "无 5% 时长上浮",
189     "low_doctor_q25": "医生容量下四分位",
190     "strict_capability": "仅严格设备兼容",
191     "bladder_45min": "憋尿准备 45 分钟",
192     "bladder_90min": "憋尿准备 90 分钟",
193     "transfer_5min": "跨室转运 5 分钟",
194     "transfer_15min": "跨室转运 15 分钟",
195 }
196 frame["label"] = frame["scenario"].map(name_map)
197 frame["inpatient_delta_pp"] = (frame["inpatient_48h_rate"] - main["
inpatient_48h_rate"]) * 100
198 frame["background_delta_pp"] = (frame["background_on_time_rate"] - main["
background_on_time_rate"]) * 100
199 y = np.arange(len(frame))
200 fig, ax = plt.subplots(figsize=(8.8, 5.5))
201 ax.axvline(0, color="#263238", lw=1)
202 ax.hlines(y, frame["background_delta_pp"], frame["inpatient_delta_pp"], color="#
B7C2CF", lw=2)
203 ax.scatter(frame["inpatient_delta_pp"], y, color=BLUE, s=70, label="住院 48 小时率"
)
204 ax.scatter(frame["background_delta_pp"], y, color=ORANGE, s=70, label="背景预约日率
")
205 ax.set_yticks(y, frame["label"])
206 ax.invert_yaxis()
207 ax.set_xlabel("相对主推荐策略的变化（百分点）")
208 ax.set_title("关键证据与参数边界的敏感性")
209 ax.grid(axis="x", color="#D7DEE8", lw=0.7)
210 ax.legend(frameon=False, ncol=2, loc="lower center", bbox_to_anchor=(0.5, -0.32))
211 fig.tight_layout()
212 save(fig, "fig05_sensitivity")
213
214
215 def daily_performance() -> None:
216     daily = pd.read_csv(UNIFIED / "daily_service_metrics.csv")
217     # The result file contains both date-only and timestamp-formatted rows from
218     # different scenarios. Parse the mixed ISO representations explicitly;
219     # otherwise pandas leaves an object series and matplotlib interprets the
220     # row positions as days since 1970.
221     daily["metric_date"] = pd.to_datetime(daily["metric_date"], format="mixed").dt.
normalize()
222     frame = daily[
223         daily["scenario"].eq("main_5pct_upper") & daily["policy"].eq("FCFS_SHARED")
224     ].copy()
225     inpatient = frame[frame["source"].eq("住院")].sort_values("metric_date")
226     background = (
227         frame[~frame["source"].eq("住院")]
228         .groupby("metric_date", as_index=False)
229         .agg(events=("events", "sum"), deadline_met=("deadline_met", "sum"))
230         .sort_values("metric_date")
231     )
232     background["deadline_rate"] = background["deadline_met"] / background["events"]
233     fig, ax = plt.subplots(figsize=(10.2, 4.5))
234     ax.plot(
235         inpatient["metric_date"],
236         inpatient["deadline_rate"].rolling(14, min_periods=1).mean(),
237         color=BLUE,
238         lw=1.7,
239         label="住院 48 小时率（14 日滚动）",
240     )
241     ax.plot(
242         background["metric_date"],
243         background["deadline_rate"].rolling(14, min_periods=1).mean(),
244         color=ORANGE,
245         lw=1.7,

```

```

246     label="门诊/体检预约日率 (14 日滚动)",
247 )
248 ax.axhline(inpatient["deadline_met"].sum() / inpatient["events"].sum(), color=BLUE
, ls="--", lw=0.9)
249 ax.axhline(background["deadline_met"].sum() / background["events"].sum(), color=
ORANGE, ls="--", lw=0.9)
250 ax.set_ylim(0.25, 0.95)
251 ax.yaxis.set_major_formatter(lambda value, _: f"{value:.0%}")
252 ax.set_xlabel("留出期日期")
253 ax.set_ylabel("完成率")
254 ax.xaxis.set_major_locator(mdates.MonthLocator(interval=2))
255 ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
256 ax.tick_params(axis="x", rotation=0)
257 ax.set_title("主推荐策略的日级服务稳定性")
258 ax.grid(axis="y", color="#D7DEE8", lw=0.7)
259 ax.legend(frameon=False, ncol=2, loc="lower center", bbox_to_anchor=(0.5, -0.30))
260 fig.tight_layout()
261 save(fig, "fig06_daily_performance")
262
263
264 def representative_gantt() -> None:
265     schedule = pd.read_csv(
266         UNIFIED / "final_patient_task_schedule.csv",
267         parse_dates=["start_dt", "end_dt"],
268         dtype={"room_id": "string"},
269         low_memory=False,
270     )
271     schedule["evaluation_cohort"] = schedule["evaluation_cohort"].astype(str).str.
lower().eq("true")
272     frame = schedule[schedule["evaluation_cohort"]].copy()
273     frame["date"] = frame["start_dt"].dt.normalize()
274     daily_minutes = frame.groupby("date")["duration_minutes"].sum()
275     target = daily_minutes.idxmax()
276     day = frame[frame["date"].eq(target)].copy()
277     room_order = (
278         day.groupby(["room_id", "room_name"])["duration_minutes"]
279         .sum()
280         .sort_values(ascending=True)
281         .index.tolist()
282     )
283     y_lookup = {room_id: pos for pos, (room_id, _) in enumerate(room_order)}
284     colors = {"住院": BLUE, "门诊": ORANGE, "体检": GREEN}
285     fig, ax = plt.subplots(figsize=(10.8, 7.2))
286     origin = target + pd.Timedelta(hours=8)
287     for row in day.itertuples(index=False):
288         start = (row.start_dt - origin).total_seconds() / 3600
289         width = (row.end_dt - row.start_dt).total_seconds() / 3600
290         ax.barh(
291             y_lookup[str(row.room_id)],
292             width,
293             left=start,
294             height=0.72,
295             color=colors[row.source],
296             edgecolor="white",
297             linewidth=0.25,
298         )
299     ax.axvspan(4, 5, color="#F0F2F5", zorder=0)
300     ax.set_yticks(
301         range(len(room_order)),
302         [f"{room_name} ({room_id})" for room_id, room_name in room_order],
303         fontsize=8,
304     )
305     ax.set_xticks(range(10), [f"{8 + hour:02d}:00" for hour in range(10)])
306     ax.set_xlim(0, 9)

```

```

307 ax.set_xlabel("时刻")
308 ax.set_title(f"高负荷代表日 {target.date()} 的 22 台设备患者级排程")
309 ax.grid(axis="x", color="#D7DEE8", lw=0.6)
310 ax.legend(
311     handles=[Patch(color=colors[key], label=key) for key in ["住院", "门诊", "体检"
312 ]],
313     frameon=False,
314     ncol=3,
315     loc="lower center",
316     bbox_to_anchor=(0.5, -0.13),
317 )
318 fig.tight_layout()
319 save(fig, "fig07_representative_gantt")
320
321 def room_utilization() -> None:
322     schedule = pd.read_csv(
323         UNIFIED / "final_patient_task_schedule.csv",
324         parse_dates=["start_dt"],
325         dtype={"room_id": "string"},
326         low_memory=False,
327     )
328     evaluation = schedule["evaluation_cohort"].astype(str).str.lower().eq("true")
329     frame = schedule[evaluation].copy()
330     frame["date"] = frame["start_dt"].dt.normalize()
331     dates = pd.date_range("2024-04-01", "2025-03-31", freq="D")
332     rooms = frame[["room_id", "room_name"]].drop_duplicates()
333     complete = pd.MultiIndex.from_product(
334         [dates, rooms["room_id"]], names=["date", "room_id"]
335     ).to_frame(index=False)
336     used = frame.groupby(["date", "room_id", as_index=False])["duration_minutes"].sum(
337     )
338     complete = complete.merge(used, on=["date", "room_id", how="left").fillna({"
339 duration_minutes": 0})
340     mean = complete.groupby("room_id")["duration_minutes"].mean().div(480).sort_values(
341     )
342     name_map = rooms.set_index("room_id")["room_name"].to_dict()
343     fig, ax = plt.subplots(figsize=(8.8, 6.2))
344     colors = [RED if room == "7" else BLUE for room in mean.index]
345     bars = ax.barh(range(len(mean)), mean.values, color=colors, height=0.68)
346     ax.set_yticks(range(len(mean)), [f"{name_map[room]} ({room})" for room in mean.
347 index], fontsize=8)
348     ax.xaxis.set_major_formatter(lambda value, _: f"{value:.0%}")
349     ax.set_xlabel("留出期日均占用率（以每日 480 分钟计）")
350     ax.set_title("主推荐策略下各设备日均占用")
351     ax.grid(axis="x", color="#D7DEE8", lw=0.7)
352     ax.legend(handles=[Patch(color=RED, label="床旁唯一设备"), Patch(color=BLUE, label=
353 "普通设备")], frameon=False)
354     fig.tight_layout()
355     save(fig, "fig08_room_utilization")
356
357 def modeling_flow() -> None:
358     fig, ax = plt.subplots(figsize=(10.4, 3.7))
359     ax.axis("off")
360     boxes = [
361         (0.02, "数据审计\n时间边界与事件口径"),
362         (0.22, "P1 参数层\n需求·时长·设备·医生"),
363         (0.42, "统一任务层\n患者—项目—可用机房"),
364         (0.62, "连续日历排程\n房间与医生共享容量"),
365         (0.82, "留出期评价\n约束核验与敏感性"),
366     ]
367     for index, (x, label) in enumerate(boxes):
368         color = [LIGHT, "#DDE9F7", "#DFF1E5", "#FCE8D2", "#E9E0F0"][index]

```

```

365         ax.text(
366             x + 0.08,
367             0.52,
368             label,
369             ha="center",
370             va="center",
371             fontsize=10,
372             bbox=dict(boxstyle="round,pad=0.6", facecolor=color, edgecolor="#607D8B",
linewidth=1.1),
373             transform=ax.transAxes,
374         )
375         if index < len(boxes) - 1:
376             ax.annotate(
377                 "",
378                 xy=(x + 0.19, 0.52),
379                 xytext=(x + 0.17, 0.52),
380                 arrowprops=dict(arrowstyle="->", color="#607D8B", lw=1.4),
381                 xycoords=ax.transAxes,
382             )
383     ax.text(0.50, 0.12, "训练期冻结参数（至 2024-03-31） → 留出期只用于评价（2024-04-01
至 2025-03-31）", ha="center", fontsize=10, color=GRAY, transform=ax.transAxes)
384     ax.set_title("从原始记录到可执行预约方案的建模链", pad=18)
385     fig.tight_layout()
386     save(fig, "fig09_modeling_flow")
387
388
389 def main() -> None:
390     configure()
391     monthly_demand()
392     doctor_capacity()
393     capability_coverage()
394     policy_tradeoff()
395     sensitivity()
396     daily_performance()
397     representative_gantt()
398     room_utilization()
399     modeling_flow()
400     print(f"created 9 vector PDF figures in {OUT}")
401
402
403 if __name__ == "__main__":
404     main()

```

Listing 9 论文结果表生成程序

```

1  """Build reproducible LaTeX tables used by the final paper."""
2
3  from __future__ import annotations
4
5  import json
6  from pathlib import Path
7
8  import pandas as pd
9
10
11  ROOT = Path(__file__).resolve().parents[2]
12  P1 = ROOT / "supporting_materials" / "results" / "p1"
13  CAP = ROOT / "supporting_materials" / "results" / "capability"
14  INPUT = ROOT / "supporting_materials" / "processed_data" / "unified_holdout"
15  UNIFIED = ROOT / "supporting_materials" / "results" / "unified_schedule"
16  OUT = ROOT / "paper" / "tables"
17
18
19 def esc(value: object) -> str:

```

```

20 text = str(value)
21 for old, new in [
22     ("\\", r"\textbackslash{ }"),
23     ("&", r"\&"),
24     ("% ", r"\%"),
25     ("_", r"\_"),
26     ("#", r"\#"),
27 ]:
28     text = text.replace(old, new)
29 return text
30
31
32 def write(name: str, lines: List[str]) -> None:
33     (OUT / name).write_text("\n".join(lines) + "\n", encoding="utf-8")
34
35
36 def pct(value: float) -> str:
37     return f"{float(value) * 100:.2f}\\%"
38
39
40 def table_data_scope() -> None:
41     summary = json.loads((INPUT / "summary.json").read_text(encoding="utf-8"))
42     events = pd.read_csv(INPUT / "events.csv")
43     items = pd.read_csv(INPUT / "items.csv", low_memory=False)
44     rows = []
45     for source in ["住院", "门诊", "体检"]:
46         event_count = int(events[events["source"].eq(source)]["event_id"].nunique())
47         item_count = int(items[items["event_id"].isin(events.loc[events["source"].eq(
48             source), "event_id"])]].shape[0])
49         evaluation = int(
50             events[events["source"].eq(source)]["evaluation_cohort"]
51             .astype(str)
52             .str.lower()
53             .eq("true")
54             .sum()
55         )
56         rows.append((source, event_count, item_count, evaluation))
57     lines = [
58         r"\begin{tabular}{lrrr}",
59         r"\toprule",
60         r"患者来源 & 连续模拟事件数 & 项目任务数 & 留出期评价事件数 \\",
61         r"\midrule",
62     ]
63     lines += [f"{source} & {events[:,]} & {items[:,]} & {evaluation:,} \\\\" for source,
64             events_, items_, evaluation in rows]
65     row_end = r"\\"
66     lines += [
67         r"\midrule",
68         f"合计 & {len(events):,} & {len(items):,} & {int(events['evaluation_cohort'].
69             astype(str).str.lower().eq('true').sum()):,} {row_end}",
70         r"\bottomrule",
71         r"\end{tabular}",
72     ]
73     write("data_scope.tex", lines)
74
75
76 def table_department() -> None:
77     frame = pd.read_csv(P1 / "department_demand_overview_train.csv")
78     lines = [
79         r"\begin{tabular}{lrrrr}",
80         r"\toprule",
81         r"来源 & 训练期事件数 & 可识别科室数 & 前6科室占比 & 前12科室占比 \\",
82         r"\midrule",
83     ]

```

```

81     for row in frame.itertuples(index=False):
82         lines.append(
83             f"{row.source} & {row.service_event_count:,} & {row.known_department_count
84             :,,} & "
85             f"{pct(row.top_6_event_share)} & {pct(row.top_12_event_share)} \\\\"
86         )
87     lines += [r"\bottomrule", r"\end{tabular}"]
88     write("department_demand.tex", lines)
89
90 def table_duration() -> None:
91     frame = pd.read_csv(P1 / "service_duration_parameters.csv")
92     frame = frame[~frame["category"].astype(str).str.startswith("非服务")]
93     lines = [
94         r"\begin{tabular}{lrrrrr}",
95         r"\toprule",
96         r"项目族 & 下界/min & 名义/min & 上界/min & 训练期代理样本数 \\",
97         r"\midrule",
98     ]
99     for row in frame.itertuples(index=False):
100         proxy = "--" if pd.isna(row.proxy_n) else f"{int(row.proxy_n):,}"
101         lines.append(
102             f"{esc(row.category)} & {row.duration_lower_min:g} & {row.
103             duration_nominal_min:g} & "
104             f"{row.duration_upper_min:g} & {proxy} \\\\"
105         )
106     lines += [r"\bottomrule", r"\end{tabular}"]
107     write("duration_parameters.tex", lines)
108
109 def table_capability() -> None:
110     frame = pd.read_csv(CAP / "special_resource_bottlenecks.csv")
111     lines = [
112         r"\begin{tabularx}{\textwidth}{l r X l}",
113         r"\toprule",
114         r"关键资源 & 可用设备数 & 设备编号 & 结构判断 \\",
115         r"\midrule",
116     ]
117     for row in frame.itertuples(index=False):
118         lines.append(
119             f"{esc(row.resource)} & {row.feasible_room_count} & {esc(row.
120             feasible_machine_ids)} & {esc(row.status)} \\\\"
121         )
122     lines += [r"\bottomrule", r"\end{tabularx}"]
123     write("capability_bottlenecks.tex", lines)
124
125 def table_doctor_capacity() -> None:
126     frame = pd.read_csv(INPUT / "doctor_slot_capacity.csv")
127     labels = {0: "周一", 1: "周二", 2: "周三", 3: "周四", 4: "周五", 5: "周六", 6: "周日"}
128     rows = []
129     for weekday, group in frame.groupby("weekday"):
130         rows.append(
131             (
132                 labels[int(weekday)],
133                 int(group["capacity_main"].min()),
134                 float(group["capacity_main"].median()),
135                 int(group["capacity_main"].max()),
136                 int(group["capacity_low"].min()),
137                 float(group["capacity_low"].median()),
138                 int(group["capacity_low"].max()),
139             )
140         )
141     lines = [

```

```

142         r"\begin{tabular}{lrrrrrr}",
143         r"\toprule",
144         r" & \multicolumn{3}{c}{主情景} & \multicolumn{3}{c}{偏紧情景} \\",
145         r"\cmidrule(lr){2-4}\cmidrule(lr){5-7}",
146         r"星期 & 最小 & 中位 & 最大 & 最小 & 中位 & 最大 \\",
147         r"\midrule",
148     ]
149     lines += [f"{day} & {a} & {b:g} & {c} & {d} & {e:g} & {f} \\\\" for day, a, b, c,
150     d, e, f in rows]
151     lines += [r"\bottomrule", r"\end{tabular}"]
152     write("doctor_capacity.tex", lines)
153
154 def table_policy() -> None:
155     frame = pd.read_csv(UNIFIED / "policy_scenario_metrics.csv")
156     frame = frame[frame["scenario"].eq("main_5pct_upper")]
157     names = {
158         "FCFS_SHARED": "共享 FCFS",
159         "SLACK_GUARD_SHARED": "松弛度保护",
160         "JOINT_SCARCITY": "联合稀缺度",
161     }
162     lines = [
163         r"\begin{tabular}{lrrrrrr}",
164         r"\toprule",
165         r"策略 & 住院48h完成数 & 住院率 & 背景完成数 & 背景率 & 等待P90/h \\",
166         r"\midrule",
167     ]
168     for row in frame.itertuples(index=False):
169         marker = r"\textbf{" if row.policy == "FCFS_SHARED" else ""
170         end = "}" if marker else ""
171         lines.append(
172             f"{marker}{names[row.policy]}{end} & {row.inpatient_complete_48h:,} & {pct(
173                 row.inpatient_48h_rate)} & "
174             f"{row.background_on_planned_day:,} & {pct(row.background_on_time_rate)} & "
175             f"{row.inpatient_wait_p90_hours_conditional:.2f} \\\\"
176         )
177     lines += [r"\bottomrule", r"\end{tabular}"]
178     write("policy_metrics.tex", lines)
179
180 def table_sensitivity() -> None:
181     frame = pd.read_csv(UNIFIED / "policy_scenario_metrics.csv")
182     frame = frame[frame["policy"].eq("FCFS_SHARED")]
183     names = {
184         "main_5pct_upper": "主情景: 5%上界时长",
185         "nominal_no_special": "无特殊时长上浮",
186         "low_doctor_q25": "医生容量下四分位",
187         "strict_capability": "仅严格设备兼容",
188         "bladder_45min": "憋尿准备45分钟",
189         "bladder_90min": "憋尿准备90分钟",
190         "transfer_5min": "跨室转运5分钟",
191         "transfer_15min": "跨室转运15分钟",
192     }
193     lines = [
194         r"\begin{tabular}{lrrrrr}",
195         r"\toprule",
196         r"情景 & 住院48h率 & 背景预约日率 & 条件等待P90/h & 排入分钟 \\",
197         r"\midrule",
198     ]
199     for row in frame.itertuples(index=False):
200         lines.append(
201             f"{names[row.scenario]} & {pct(row.inpatient_48h_rate)} & {pct(row.
202                 background_on_time_rate)} & "

```

```

202         f"{row.inpatient_wait_p90_hours_conditional:.2f} & {int(row.
scheduled_minutes):,} \\\\"
203     )
204     lines += [r"\bottomrule", r"\end{tabular}"]
205     write("sensitivity_metrics.tex", lines)
206
207
208 def table_verification() -> None:
209     report = json.loads((UNIFIED / "independent_verification.json").read_text(encoding
="utf-8"))
210     groups = [
211         ("任务键、输入对应、时长", ["unique_exported_task_key", "
scheduled_task_exists_in_input", "duration_matches_5pct_scenario"]),
212         ("设备兼容、床旁位置", ["project_room_compatibility", "bedside_uses_room_7"]),
213         ("释放、截止、工作时段", ["task_not_before_release", "
task_complete_by_event_deadline", "inside_single_work_block"]),
214         ("房间、患者、转运冲突", ["no_room_overlap", "no_same_patient_task_overlap", "
cross_room_transfer_at_least_10min"]),
215         ("医生并发容量", ["doctor_capacity_defined", "
doctor_concurrency_within_capacity"]),
216         ("结果表聚合一致性", ["outcome_complete_flag_consistency", "
outcome_deadline_flag_consistency"]),
217     ]
218     lines = [
219         r"\begin{tabularx}{\textwidth}{X r l}",
220         r"\toprule",
221         r"核验项目 & 违规数 & 结论 \\\",
222         r"\midrule",
223     ]
224     for label, keys in groups:
225         violations = sum(int(report["checks"][key]["violation_count"]) for key in keys
)
226         lines.append(f"{label} & {violations} & {'通过' if violations == 0 else '未通过'
} \\\")
227     lines += [r"\bottomrule", r"\end{tabularx}"]
228     write("verification.tex", lines)
229
230
231 def table_assumption_provenance() -> None:
232     rows = [
233         ("住院48小时全项目完成", "赛题直接给定", "事件硬截止时刻"),
234         ("5%特殊检查取上界时长", "赛题直接给定比例; 附件无个体标签", "评价/预热分别按最近整数抽
取5\%"),
235         ("项目时长区间与空腹10点窗口", "赛题直接给定", "名义/上界时长与硬时窗"),
236         ("平峰$Q_{0.45}$--$Q_{0.55}$、高峰$Q_{0.90}$", "运营分位阈值, 非赛题指定日期", "用真
实日需求识别代表性容量水平"),
237         ("22台设备能力、床旁7号机", "附件表1与设备表文本证据", "项目--设备兼容集"),
238         ("50个项目采用类别级能力回退", "项目名称与表1逐字/语义核心匹配失败时, 仅沿用表1同项目族具
体证据", "主情景逐项留痕; 严格证据情景禁用回退"),
239         ("08:00--12:00、13:00--17:00日班模板", "附件表6训练期首末报告活动重建", "可复现容量模
板, 非未来正式排班"),
240         ("星期--5分钟医生并发容量", "训练日重构活动区间的同刻人数中位数", "仅为总并发代理; 不声称
掌握未来实名班表或专项资质"),
241         ("相邻报告间隔$(0,60]$分钟", "扫查起止缺失下的局部代理阈值", "只用于问题一相对吞吐证据,
不进入主排程时长"),
242         ("5分钟时间网格", "计算离散化, 附件无此字段", "时长向上取整, 不缩短检查"),
243         ("跨室转运10分钟", "附件无转运时间", "主情景; 5/15分钟边界检验"),
244         ("憋尿准备60分钟", "附件无时长; 医院公开流程建议提前1小时\cite{hrhospital2025}", "主
情景; 45/90分钟边界检验"),
245         ("门诊/体检报告日作为计划日", "附件无历史预约日", "回溯压力情景, 不声称还原历史预约"),
246         ("最后任务结束代理报告提交", "附件无扫查起止与报告书写时长", "官方KPI的乐观代理, 明示适用
边界"),
247         ("2日预热与2日截止尾窗", "由最大48小时截止期推出", "只携入跨边界在制任务并覆盖3月31日开
单截止"),

```

```

248 ]
249 lines = [
250     r"\begin{tabularx}{\textwidth}{p{0.24\textwidth} p{0.35\textwidth} X}",
251     r"\toprule",
252     r"设定/参数 & 事实来源 & 模型中的限定 \\",
253     r"\midrule",
254 ]
255 for setting, source, boundary in rows:
256     lines.append(f"{setting} & {source} & {boundary} \\\\")
257 lines += [r"\bottomrule", r"\end{tabularx}"]
258 write("assumption_provenance.tex", lines)
259
260
261 def table_recommendation_sample() -> None:
262     frame = pd.read_csv(UNIFIED / "inpatient_recommended_schedule.csv", low_memory=
False)
263     sample_events = frame["event_id"].drop_duplicates().head(8)
264     sample = frame[frame["event_id"].isin(sample_events)].head(16)
265     lines = [
266         r"\begin{tabularx}{\textwidth}{r l r X l l}",
267         r"\toprule",
268         r"患者序位 & 事件编号 & 项目序位 & 项目族 & 时段 & 设备 \\",
269         r"\midrule",
270     ]
271     for row in sample.itertuples(index=False):
272         start = pd.Timestamp(row.start_dt).strftime("%m-%d %H:%M")
273         end = pd.Timestamp(row.end_dt).strftime("%H:%M")
274         lines.append(
275             f"{row.recommended_patient_rank} & {esc(row.event_id)} & {row.
sequence_position} & "
276             f"{esc(row.category)} & {start}--{end} & {esc(row.room_name)}({esc(row.
room_id)}) \\\\")
277         )
278     lines += [r"\bottomrule", r"\end{tabularx}"]
279     write("recommendation_sample.tex", lines)
280
281
282 def main() -> None:
283     OUT.mkdir(parents=True, exist_ok=True)
284     table_data_scope()
285     table_department()
286     table_duration()
287     table_capability()
288     table_doctor_capacity()
289     table_assumption_provenance()
290     table_policy()
291     table_sensitivity()
292     table_verification()
293     table_recommendation_sample()
294     print(f"created 10 LaTeX tables in {OUT}")
295
296
297 if __name__ == "__main__":
298     main()

```

J 最终结果摘要

Listing 10 联合模型机器可读摘要

```

1 {
2     "authoritative_policy": "FCFS_SHARED",
3     "annual_state_reset_count": 1,
4     "simulation_start": "2024-03-30",

```

```

5 "simulation_end_exclusive": "2025-04-03",
6 "evaluation_start": "2024-04-01",
7 "evaluation_end_exclusive": "2025-04-01",
8 "background_interpretation": "retrospective plan-day scenario reconstructed from
  report date, with release no earlier than the observed order time; report time/
  room/doctor dispatch are excluded",
9 "official_kpi_boundary": "scheduled service completion is used as report-submission
  completion proxy because scan-end timestamp is absent",
10 "transfer_minutes_between_rooms": 10,
11 "metrics": [
12   {
13     "scenario": "main_5pct_upper",
14     "policy": "FCFS_SHARED",
15     "inpatient_events": 70771,
16     "inpatient_complete_48h": 45260,
17     "inpatient_48h_rate": 0.6395274900736178,
18     "inpatient_scheduled_complete": 45260,
19     "inpatient_unscheduled": 25511,
20     "inpatient_wait_p50_hours_conditional": 4.5,
21     "inpatient_wait_p90_hours_conditional": 40.41666666666666,
22     "inpatient_wait_p95_hours_conditional": 45.583333333333336,
23     "inpatient_room_switch_rate_multi": 0.1996234564483083,
24     "background_events": 96046,
25     "background_on_planned_day": 57341,
26     "background_on_time_rate": 0.5970160131603607,
27     "background_scheduled_complete": 57341,
28     "fallback_scheduled_task_share": 0.0877565885313203,
29     "scheduled_minutes": 1819800.0,
30     "staffed_capacity_utilization": 0.7379337611386515,
31     "transfer_minutes_assumption": 10
32   },
33   {
34     "scenario": "main_5pct_upper",
35     "policy": "SLACK_GUARD_SHARED",
36     "inpatient_events": 70771,
37     "inpatient_complete_48h": 45116,
38     "inpatient_48h_rate": 0.6374927583332156,
39     "inpatient_scheduled_complete": 45116,
40     "inpatient_unscheduled": 25655,
41     "inpatient_wait_p50_hours_conditional": 4.5,
42     "inpatient_wait_p90_hours_conditional": 40.25,
43     "inpatient_wait_p95_hours_conditional": 45.583333333333336,
44     "inpatient_room_switch_rate_multi": 0.1995957694224486,
45     "background_events": 96046,
46     "background_on_planned_day": 57341,
47     "background_on_time_rate": 0.5970160131603607,
48     "background_scheduled_complete": 57341,
49     "fallback_scheduled_task_share": 0.0876806269899583,
50     "scheduled_minutes": 1820200.0,
51     "staffed_capacity_utilization": 0.738095962207151,
52     "transfer_minutes_assumption": 10
53   },
54   {
55     "scenario": "main_5pct_upper",
56     "policy": "JOINT_SCARCITY",
57     "inpatient_events": 70771,
58     "inpatient_complete_48h": 44805,
59     "inpatient_48h_rate": 0.6330983029772081,
60     "inpatient_scheduled_complete": 44805,
61     "inpatient_unscheduled": 25966,
62     "inpatient_wait_p50_hours_conditional": 5.416666666666667,
63     "inpatient_wait_p90_hours_conditional": 41.5,
64     "inpatient_wait_p95_hours_conditional": 45.833333333333336,
65     "inpatient_room_switch_rate_multi": 0.2000664488620632,

```

```

66     "background_events": 96046,
67     "background_on_planned_day": 59175,
68     "background_on_time_rate": 0.6161110301313953,
69     "background_scheduled_complete": 59175,
70     "fallback_scheduled_task_share": 0.0911003692843735,
71     "scheduled_minutes": 1947865.0,
72     "staffed_capacity_utilization": 0.7898644607321351,
73     "transfer_minutes_assumption": 10
74 },
75 {
76     "scenario": "nominal_no_special",
77     "policy": "FCFS_SHARED",
78     "inpatient_events": 70771,
79     "inpatient_complete_48h": 45269,
80     "inpatient_48h_rate": 0.6396546608073929,
81     "inpatient_scheduled_complete": 45269,
82     "inpatient_unscheduled": 25502,
83     "inpatient_wait_p50_hours_conditional": 4.5,
84     "inpatient_wait_p90_hours_conditional": 40.41666666666666,
85     "inpatient_wait_p95_hours_conditional": 45.583333333333336,
86     "inpatient_room_switch_rate_multi": 0.199817265629326,
87     "background_events": 96046,
88     "background_on_planned_day": 57339,
89     "background_on_time_rate": 0.5969951898048852,
90     "background_scheduled_complete": 57339,
91     "fallback_scheduled_task_share": 0.0877352059354958,
92     "scheduled_minutes": 1819660.0,
93     "staffed_capacity_utilization": 0.7378769907646766,
94     "transfer_minutes_assumption": 10
95 },
96 {
97     "scenario": "low_doctor_q25",
98     "policy": "FCFS_SHARED",
99     "inpatient_events": 70771,
100    "inpatient_complete_48h": 43850,
101    "inpatient_48h_rate": 0.6196040751155134,
102    "inpatient_scheduled_complete": 43850,
103    "inpatient_unscheduled": 26921,
104    "inpatient_wait_p50_hours_conditional": 8.083333333333334,
105    "inpatient_wait_p90_hours_conditional": 42.0,
106    "inpatient_wait_p95_hours_conditional": 45.583333333333336,
107    "inpatient_room_switch_rate_multi": 0.1923971426989312,
108    "background_events": 96046,
109    "background_on_planned_day": 56621,
110    "background_on_time_rate": 0.5895196051891802,
111    "background_scheduled_complete": 56621,
112    "fallback_scheduled_task_share": 0.0859384602777891,
113    "scheduled_minutes": 1769275.0,
114    "staffed_capacity_utilization": 0.8175248419150858,
115    "transfer_minutes_assumption": 10
116 },
117 {
118     "scenario": "strict_capability",
119     "policy": "FCFS_SHARED",
120     "inpatient_events": 70771,
121     "inpatient_complete_48h": 50652,
122     "inpatient_48h_rate": 0.7157168896864535,
123     "inpatient_scheduled_complete": 50652,
124     "inpatient_unscheduled": 20119,
125     "inpatient_wait_p50_hours_conditional": 5.083333333333333,
126     "inpatient_wait_p90_hours_conditional": 45.333333333333336,
127     "inpatient_wait_p95_hours_conditional": 46.66666666666666,
128     "inpatient_room_switch_rate_multi": 0.2740738689849936,
129     "background_events": 96046,

```

```

"background_on_planned_day": 47705,
"background_on_time_rate": 0.4966890864793953,
"background_scheduled_complete": 47705,
"fallback_scheduled_task_share": 0.0,
"scheduled_minutes": 1664965.0,
"staffed_capacity_utilization": 0.6751477550358363,
"transfer_minutes_assumption": 10
},
{
  "scenario": "bladder_45min",
  "policy": "FCFS_SHARED",
  "inpatient_events": 70771,
  "inpatient_complete_48h": 45218,
  "inpatient_48h_rate": 0.6389340266493337,
  "inpatient_scheduled_complete": 45218,
  "inpatient_unscheduled": 25553,
  "inpatient_wait_p50_hours_conditional": 4.5,
  "inpatient_wait_p90_hours_conditional": 40.25,
  "inpatient_wait_p95_hours_conditional": 45.583333333333336,
  "inpatient_room_switch_rate_multi": 0.1990697159311146,
  "background_events": 96046,
  "background_on_planned_day": 57517,
  "background_on_time_rate": 0.5988484684422047,
  "background_scheduled_complete": 57517,
  "fallback_scheduled_task_share": 0.087424682359633,
  "scheduled_minutes": 1821400.0,
  "staffed_capacity_utilization": 0.7385825654126497,
  "transfer_minutes_assumption": 10
},
{
  "scenario": "bladder_90min",
  "policy": "FCFS_SHARED",
  "inpatient_events": 70771,
  "inpatient_complete_48h": 45376,
  "inpatient_48h_rate": 0.6411665795311638,
  "inpatient_scheduled_complete": 45376,
  "inpatient_unscheduled": 25395,
  "inpatient_wait_p50_hours_conditional": 4.5,
  "inpatient_wait_p90_hours_conditional": 40.166666666666666,
  "inpatient_wait_p95_hours_conditional": 45.416666666666666,
  "inpatient_room_switch_rate_multi": 0.2005925023533972,
  "background_events": 96046,
  "background_on_planned_day": 56833,
  "background_on_time_rate": 0.5917268808695834,
  "background_scheduled_complete": 56833,
  "fallback_scheduled_task_share": 0.0877994673184598,
  "scheduled_minutes": 1815485.0,
  "staffed_capacity_utilization": 0.7361840171122127,
  "transfer_minutes_assumption": 10
},
{
  "scenario": "transfer_5min",
  "policy": "FCFS_SHARED",
  "inpatient_events": 70771,
  "inpatient_complete_48h": 45247,
  "inpatient_48h_rate": 0.6393437990137203,
  "inpatient_scheduled_complete": 45247,
  "inpatient_unscheduled": 25524,
  "inpatient_wait_p50_hours_conditional": 4.583333333333333,
  "inpatient_wait_p90_hours_conditional": 40.5,
  "inpatient_wait_p95_hours_conditional": 45.583333333333336,
  "inpatient_room_switch_rate_multi": 0.2002048839913616,
  "background_events": 96046,
  "background_on_planned_day": 57345,

```

```

194     "background_on_time_rate": 0.5970576598713117,
195     "background_scheduled_complete": 57345,
196     "fallback_scheduled_task_share": 0.0876075583510254,
197     "scheduled_minutes": 1821800.0,
198     "staffed_capacity_utilization": 0.7387447664811492,
199     "transfer_minutes_assumption": 5
200 },
201 {
202     "scenario": "transfer_15min",
203     "policy": "FCFS_SHARED",
204     "inpatient_events": 70771,
205     "inpatient_complete_48h": 45289,
206     "inpatient_48h_rate": 0.6399372624380043,
207     "inpatient_scheduled_complete": 45289,
208     "inpatient_unscheduled": 25482,
209     "inpatient_wait_p50_hours_conditional": 4.5,
210     "inpatient_wait_p90_hours_conditional": 40.333333333333336,
211     "inpatient_wait_p95_hours_conditional": 45.5,
212     "inpatient_room_switch_rate_multi": 0.1995680823965889,
213     "background_events": 96046,
214     "background_on_planned_day": 57400,
215     "background_on_time_rate": 0.597630302146888,
216     "background_scheduled_complete": 57400,
217     "fallback_scheduled_task_share": 0.0880673538062054,
218     "scheduled_minutes": 1819020.0,
219     "staffed_capacity_utilization": 0.7376174690550774,
220     "transfer_minutes_assumption": 15
221 }
222 ]
223 }

```

Listing 11 独立核验机器可读报告

```

1  {
2    "passed": true,
3    "schedule_rows": 151020,
4    "scheduled_events": 102986,
5    "input_events": 167361,
6    "checks": {
7      "unique_exported_task_key": {
8        "passed": true,
9        "violation_count": 0,
10       "detail": "each scheduled task must occur exactly once"
11     },
12     "scheduled_task_exists_in_input": {
13       "passed": true,
14       "violation_count": 0,
15       "detail": ""
16     },
17     "duration_matches_5pct_scenario": {
18       "passed": true,
19       "violation_count": 0,
20       "detail": ""
21     },
22     "project_room_compatibility": {
23       "passed": true,
24       "violation_count": 0,
25       "detail": ""
26     },
27     "bedside_uses_room_7": {
28       "passed": true,
29       "violation_count": 0,
30       "detail": ""
31     },

```

```

32 "task_not_before_release": {
33     "passed": true,
34     "violation_count": 0,
35     "detail": ""
36 },
37 "task_complete_by_event_deadline": {
38     "passed": true,
39     "violation_count": 0,
40     "detail": ""
41 },
42 "five_minute_grid": {
43     "passed": true,
44     "violation_count": 0,
45     "detail": ""
46 },
47 "inside_single_work_block": {
48     "passed": true,
49     "violation_count": 0,
50     "detail": ""
51 },
52 "fasting_complete_by_10": {
53     "passed": true,
54     "violation_count": 0,
55     "detail": ""
56 },
57 "no_room_overlap": {
58     "passed": true,
59     "violation_count": 0,
60     "detail": ""
61 },
62 "contiguous_event_sequence_numbers": {
63     "passed": true,
64     "violation_count": 0,
65     "detail": ""
66 },
67 "no_same_patient_task_overlap": {
68     "passed": true,
69     "violation_count": 0,
70     "detail": ""
71 },
72 "cross_room_transfer_at_least_10min": {
73     "passed": true,
74     "violation_count": 0,
75     "detail": ""
76 },
77 "doctor_capacity_defined": {
78     "passed": true,
79     "violation_count": 0,
80     "detail": ""
81 },
82 "doctor_concurrency_within_capacity": {
83     "passed": true,
84     "violation_count": 0,
85     "detail": {
86         "max_observed_load": 20,
87         "max_capacity": 20
88     }
89 },
90 "outcome_complete_flag_consistency": {
91     "passed": true,
92     "violation_count": 0,
93     "detail": ""
94 },
95 "outcome_deadline_flag_consistency": {

```

```
96     "passed": true,  
97     "violation_count": 0,  
98     "detail": ""  
99   }  
100 }  
101 }
```